

LOWL System Language Static Compiler



By Anthony Matarazzo (c) 2026

LOWL Language Reference Manual v2.1.0

A Comprehensive Systems Programming
Language for Intel x86_64

Copyright (c) 2026 Anthony Matarazzo

MIT License

Table of Contents

Volume I: Foundations

1. Introduction to lowl
2. The Intel x86_64 Architecture and Protected Mode
3. Booting and System Initialization
4. Language Fundamentals
5. Type System

Volume II: Core Language Features

6. Type Conversion with Rounding Control
7. Control Flow
8. Functions and Calling Conventions
9. Object-Oriented Programming
10. Templates and Generics

Volume III: Advanced Data Structures

11. BlockArray: SIMD-Optimized Dynamic Arrays
12. RB Maps: Red-Black Trees with Perfect Hashing
13. Data Sections: Declarative Data with External Files
14. Pattern Matching Switch

Volume IV: Systems Programming

15. System Programming Builtins
16. SIMD Vector Operations
17. Memory Management and Protection
18. Module System and Executable Loader
19. Compiler Directives and Optimization
20. Complete Examples and Appendix

Volume I: Foundations

Chapter 1: Introduction to lowl

1.1 Origins and Motivation

The lowl programming language emerged from a fundamental observation about systems programming: existing languages force an uncomfortable choice between high-level expressiveness and low-level control. C gives you control but lacks modern abstractions. Rust gives you safety but imposes a complex ownership model. C++ gives you abstractions but at the cost of unpredictable performance characteristics. lowl was designed to resolve this trilemma by starting from first principles: what would a language look like if it were designed specifically for Intel x86_64 systems programming, with no legacy baggage and no compromises?

1.2 Defining Systems Programming

Systems programming refers to the development of software that operates at the boundary between hardware and application software. This includes operating system kernels, device drivers, bootloaders, hypervisors, embedded firmware, and real-time control systems. These domains share common requirements: deterministic performance, direct hardware access, minimal runtime dependencies, and predictable memory layout. lowl addresses each of these requirements as primary design goals rather than afterthoughts.

1.3 The lowl Philosophy of Zero-Cost Abstractions

When Bjarne Stroustrup coined the phrase "zero-overhead abstraction," he articulated a principle: what you don't use, you don't pay for. lowl extends this principle further: what you do use, you couldn't hand-code better. Every language feature from classes to templates to pattern matching compiles to assembly code that is as efficient as handwritten implementation. The compiler does not insert hidden runtime checks, hidden memory allocations, or hidden function calls unless you explicitly request them.

1.4 Hardware Control Without Inline Assembly

Traditional systems languages require inline assembly for hardware access. lowl bakes hardware operations directly into the language. Reading the timestamp counter, writing to a control register, or sending a byte to an I/O port are all built-in operations with familiar function-call syntax. This approach makes hardware-dependent code more readable and maintainable while still compiling to the exact instruction sequence you expect.

1.5 Python-Style Indentation for Systems Code

The choice of indentation-based syntax over braces is deliberate. Systems code often involves deep nesting of conditionals, loops, and error checks. Braces become visually noisy and difficult to match correctly. Indentation makes the block structure immediately visible and eliminates entire classes of bugs related to misplaced braces. The lowl compiler enforces consistent indentation, catching errors that would otherwise manifest as runtime misbehavior.

1.6 Strong Static Typing with Inference

Every variable in lowl has a fixed type known at compile time. The compiler infers types from initializers, reducing verbosity while maintaining safety. When you write `let x = 42`, the compiler infers `u64` as the type. When you write `let y = 3.14159`, the compiler infers `f64`. This inference extends to complex types including template instantiations, allowing generic code that remains readable.

1.7 The Explicit Cost Principle

One of lowl's most distinctive principles is that no operation should have hidden costs. Memory allocation requires an explicit call to `physical_alloc`. Virtual function dispatch requires the `virtual` keyword. Exception handling (when added) will require explicit annotation. This principle ensures that performance characteristics are visible in the source code, making optimization efforts more systematic.

1.8 SIMD-First Architecture

Modern x86_64 processors have powerful SIMD capabilities through SSE, AVX, AVX2, and AVX-512. Traditional languages treat SIMD as an afterthought, requiring intrinsics or compiler auto-vectorization heuristics. lowl makes SIMD a first-class citizen with built-in vector types and operations. The `block_array` container is specifically designed for SIMD operations, with cache-line-aligned blocks and block-level vectorization.

1.9 Memory Layout Control

Systems programmers need precise control over memory layout. lowl provides attributes for controlling alignment (`#[align(64)]`), packing (`#[packed]`), and section placement. Structures have a defined layout that matches the C ABI, enabling interoperability with existing

system interfaces. The compiler does not reorder fields or insert padding unless explicitly permitted.

1.10 Interrupt Handling Model

Interrupt handlers require special care: they execute in an arbitrary context and must save and restore all registers. lowl's `#[interrupt]` attribute marks functions as interrupt handlers. The compiler automatically generates the prologue and epilogue code required for interrupt handling, including saving registers, switching stacks if needed, and issuing the appropriate return instruction (IRETQ instead of RET).

1.11 Physical vs Virtual Memory

lowl distinguishes between physical and virtual memory at the language level. The `physical_alloc` function returns physical memory addresses suitable for DMA and page table entries. The normal allocation interfaces (when implemented in the standard library) return virtual addresses. This distinction aligns with how operating systems actually manage memory, making kernel code more transparent.

1.12 MMIO and Volatile Access

Memory-mapped I/O (MMIO) regions require special handling: reads and writes must not be reordered, combined, or optimized away. lowl provides the `mmio_ptr<T>` type for MMIO pointers. Operations through `mmio_ptr` are treated as volatile and are emitted with memory barriers as needed. This prevents the compiler from applying optimizations that would break device communication.

1.13 The Compiler Pipeline

The lowl compiler is implemented in Python, making it easy to modify and extend. The compilation pipeline consists of four phases: lexical analysis (tokenization), indentation processing, syntactic analysis (parsing), and code generation. The code generator produces NASM or Intel-syntax assembly, which is then assembled and linked using standard tools. This approach leverages mature assembly toolchains while keeping the compiler itself simple and maintainable.

1.14 Optimizer Architecture

The lowl optimizer operates on the abstract syntax tree before code generation. Optimization levels control which transformations are applied. Level O0 preserves the original structure for debugging. Level O1 performs constant folding and dead code elimination. Level O2 adds loop-invariant code motion and basic SIMD vectorization. Level O3 performs aggressive loop unrolling, cross-block vectorization, and speculative execution hints.

1.15 Target Platforms

The primary target for lowl is Intel x86_64 processors from the Nehalem (2008) onward, covering all modern desktop, server, and laptop CPUs. The compiler supports the full instruction set including AVX-512 on processors that implement it. Secondary targets include older x86_64 processors (Core 2) with reduced SIMD features and, potentially, ARM64 in future versions.

1.16 Comparison with C

Unlike C, lowl has proper namespaces, modules, and a module system. Unlike C, lowl has generic types (templates) with specialization. Unlike C, lowl has pattern matching and algebraic data

types. Unlike C, lowl has SIMD as a first-class feature. The areas where lowl resembles C are deliberate: the same calling convention, the same memory model, the same ABI for interoperability.

1.17 Comparison with Rust

Rust prioritizes memory safety through ownership and borrowing, which imposes runtime tracking and compile-time constraints. lowl prioritizes control and simplicity, leaving safety to the programmer. This makes lowl better suited for very low-level code (bootloaders, kernel internals) where ownership semantics don't align with hardware reality. Rust wins for application-level systems programming where safety justifies complexity.

1.18 Comparison with Zig

Zig offers compile-time execution and cross-compilation as primary features. lowl offers deeper hardware integration (MSRs, control registers) and a more traditional object system. Both languages target systems programming without hidden control flow. The choice between them often comes down to whether you need Zig's compile-time metaprogramming or lowl's hardware access.

1.19 When to Use lowl

lowl is the right choice when you need absolute control over the generated machine code, when you are writing code that runs before an operating system is available, when you are implementing operating system internals, or when you need SIMD performance without wrestling with compiler intrinsics. lowl is less suitable for application-level programming where portability and rapid development are primary concerns.

1.20 Learning Path for lowl

This manual is organized as a progressive tutorial. Chapters 1-4 introduce the language fundamentals. Chapters 5-6 cover types and conversions. Chapters 7-9 present control flow, functions, and objects. Chapters 10-13 cover templates, containers, and data management. Chapters 14-16 cover pattern matching, built-ins, and SIMD. Chapters 17-20 cover systems programming, modules, optimization, and examples.

1.21 Example: Boot Sector

The classic "hello world" of systems programming is the boot sector. lowl's boot output format produces a 512-byte image with the correct 0xAA55 signature. The minimal boot sector shows how lowl handles real-mode code, BIOS interrupts, and the transition to protected mode—all in a few dozen lines of code.

1.22 Example: Kernel Entry Point

The kernel entry point is the first code executed after the bootloader transfers control. lowl's `#[kernel]` attribute marks ring-0 entry points. The kernel must set up its own stack, enable paging, and initialize the basic runtime. The compiler emits the appropriate segment overrides and memory barriers for kernel execution.

1.23 Example: Device Driver

Device drivers interact with hardware through MMIO and interrupts. lowl's `#[driver]` attribute marks driver entry points. The driver registers its interrupt handler, maps MMIO regions through

the page tables, and communicates with the device using `mmio_ptr` accesses. The compiler ensures that MMIO operations are not reordered.

1.24 Example: SIMD Vector Math

High-performance computing workloads benefit from SIMD. A typical example is matrix multiplication using `vec8_f32` (AVX). The lowl compiler generates the `vmovaps`, `vmulps`, and `vaddps` instructions required for optimal performance, with loop unrolling and prefetching at O3 optimization.

1.25 Example: Interrupt Service Routine

Interrupt service routines must be fast, must preserve all registers, and must acknowledge the interrupt controller. lowl's `#[interrupt]` attribute generates the correct prologue (saving all registers) and epilogue (IRETQ). The programmer just writes the device-specific handling code.

1.26 Example: Page Fault Handler

Page fault handlers are critical to virtual memory systems. The handler reads CR2 to get the faulting address, examines the page tables, and either loads the missing page or terminates the faulting process. lowl's built-in `read_cr2()` function makes this straightforward.

1.27 Example: System Call Interface

User programs transition to the kernel through system calls. The kernel defines a syscall entry point, saves user registers, examines the syscall number, and dispatches to the appropriate handler. lowl's `#[interrupt]` attribute works for software interrupts, and the `syscall` instruction is available through inline assembly.

1.28 Example: Module Loading

The module system allows dynamic loading of code. A kernel module is compiled with `-f kernel` to produce a loadable image. The `load_module` built-in loads the module into memory, resolves its imports, and calls its `module_init` function. This enables extensible kernels like Linux.

1.29 Example: Memory Protection Through Visitors

The memory protection visitor pattern allows custom handling of page faults. A `MemoryViolationVisitor` subclass implements methods for read, write, exec, and user violations. This enables advanced memory management features like copy-on-write, memory-mapped files, and page swapping.

1.30 Summary

lowl is a purpose-built systems language for Intel x86_64. It combines modern syntax with zero-cost abstractions and complete hardware access. The following chapters explore each feature in depth, building toward complete operating system development. Whether you are writing a bootloader, a kernel, a driver, or a high-performance numeric library, lowl gives you the tools you need without getting in your way.

Chapter 2: The Intel x86_64 Architecture and Protected Mode

2.1 Historical Context: The 8086 and Real Mode

The Intel 8086 processor, introduced in 1978, established the x86 architecture that continues to dominate computing today. The 8086 operated in what we now call "real mode." In real mode, the processor could address only 1 megabyte of memory (2^{20} bytes) using a segmented addressing scheme: each address consisted of a 16-bit segment selector and a 16-bit offset, combined as (segment $\ll$ 4) + offset. This scheme allowed 20-bit addresses from 16-bit registers but created confusing overlapping segments. Real mode had no memory protection, no privilege levels, and no virtual memory. A misbehaving program could overwrite the operating system or even the BIOS.

2.2 The Transition to Protected Mode with the 80286

The Intel 80286, released in 1982, introduced "protected mode" as an option. In protected mode, the processor could address up to 16 megabytes of memory (2^{24} bytes) using a more sophisticated segmentation mechanism. Segment selectors became indices into descriptor tables rather than raw addresses. Each descriptor contained a base address, a limit, and access rights. This provided the first form of memory protection: the processor would check each memory access against the segment's limit and access rights. However, the 80286 could not switch back to real mode without a hardware reset, limiting its practical use.

2.3 The 80386: 32-bit Protected Mode and Paging

The Intel 80386, released in 1985, revolutionized x86 architecture. It extended the register width to 32 bits and the address space to 4 gigabytes (2^{32} bytes). It added paging, which allowed the operating system to map virtual addresses to physical addresses through page tables. Paging enabled virtual memory (swapping to disk) and simplified memory management by giving each process its own address space. The 80386 also introduced the ability to switch back to real mode through a clever sequence of operations, enabling the "unreal mode" used by many bootloaders.

2.4 Protected Mode Data Structures: Global Descriptor Table

The Global Descriptor Table (GDT) is a central data structure in protected mode. The GDT contains segment descriptors, each describing a memory segment with its base address, limit, type, and privilege level. The GDT's location is stored in the GDTR register, loaded with the LGDT instruction. A typical GDT for a 64-bit kernel contains: a null descriptor (required by the architecture), a 64-bit code segment descriptor, a 64-bit data segment descriptor, a 32-bit compatibility code segment, and a 32-bit data segment. The descriptor format includes fields for base (split across three fields), limit (split across two fields), access rights (present, privilege level, type), and flags (granularity, operand size, long mode).

2.5 Segment Selectors and Hidden Descriptor Registers

When a segment register (CS, DS, ES, SS, FS, GS) is loaded with a selector, the processor loads the corresponding descriptor from the GDT into a hidden register associated with that segment. This hidden register caches the base, limit, and access rights, so subsequent memory accesses using that segment don't require referencing the GDT again. This caching is why you can update the GDT and not affect already-loaded segment registers. The hidden registers are not directly accessible to software but affect memory access checking.

2.6 Interrupt Descriptor Table for Protected Mode

The Interrupt Descriptor Table (IDT) is similar to the GDT but for interrupts and exceptions. The IDT contains up to 256 entries, each describing an interrupt handler. The entry types include interrupt gates (which clear the interrupt flag when entered), trap gates (which leave the interrupt flag unchanged), and task gates (for hardware task switching). Each entry specifies the segment selector and offset of the handler, plus the privilege level required to invoke the handler. The IDT's location is stored in the IDTR register, loaded with the LIDT instruction.

2.7 Task State Segment for Ring Transitions

The Task State Segment (TSS) is a data structure that the processor uses for task switching and ring transitions. For ring transitions (user to kernel), the TSS specifies the stack selector and pointer to use when switching to a higher privilege level. The x86_64 architecture requires a minimal TSS (just the stack pointers for ring transitions) even if hardware task switching isn't used. The TSS descriptor is stored in the GDT, and the TR register points to the current TSS.

2.8 Long Mode: 64-bit Execution

Long mode is the native 64-bit mode of modern Intel processors. It extends the register width to 64 bits (RAX, RBX, etc.), adds eight additional general-purpose registers (R8-R15), and provides 64-bit virtual addresses. Long mode effectively eliminates segmentation: CS, DS, ES, and SS have fixed base addresses of 0 and limits of $2^{64}-1$. The FS and GS segments retain their base functionality, making them useful for thread-local storage. Long mode requires paging to be enabled, and page tables use 4-level (or 5-level on newer processors) translation.

2.9 Enabling Protected Mode: The CR0 Register

Control Register 0 (CR0) controls several processor modes and features. The Protection Enable (PE) bit (bit 0) enables protected mode. When set, the processor interprets segment selectors as indices into the GDT rather than as raw segment bases. The Paging (PG) bit (bit 31) enables paging. The coprocessor timing bits (bits 2-3) control FPU behavior. The Write-Protect (WP) bit (bit 16) makes read-only pages accessible even from ring 0, which is useful for implementing copy-on-write. Enabling protected mode requires loading a valid GDT, setting PE, and performing a far jump to reload the CS register with a protected-mode selector.

2.10 Enabling Paging: The CR3 and CR4 Registers

Paging translates virtual addresses to physical addresses through page tables. CR3 (the page directory base register) points to the top-level page table (PML4 in x86_64). The Physical Address Extension (PAE) bit in CR4 (bit 5) enables 64-bit page table entries, which are required for x86_64. CR4 also controls other features: the OSFXSR bit (bit 9) enables SSE instructions, the OSXMMEXCPT bit (bit 10) enables SIMD exception handling, and the OSXSAVE bit (bit 18) enables XSAVE for saving extended processor state.

2.11 Page Table Structure in x86_64

x86_64 uses a 4-level page table hierarchy: PML4 (Page Map Level 4), PDPT (Page Directory Pointer Table), PD (Page Directory), and PT (Page Table). Each table has 512 entries of 8 bytes each. A 48-bit virtual address is divided into fields: bits 47-39 index PML4, bits 38-30 index PDPT, bits 29-21 index PD, bits 20-12 index PT, and bits 11-0 are the page offset. Each page table entry contains the physical address of the next level table or the final page, plus flags: present, writable, user-accessible, write-through, cache-disable, accessed, dirty, and no-execute.

2.12 Large and Huge Pages

For performance, the processor supports larger page sizes: 2MB and 1GB pages. A 2MB page is created by setting the Page Size (PS) flag in a PDPT entry, which then points directly to a 2MB-aligned physical region. A 1GB page is created by setting PS in a PML4 entry. Large pages reduce TLB pressure and page table memory consumption but increase internal fragmentation. The tradeoff makes large pages ideal for kernel mappings and large memory regions.

2.13 The No-Execute (NX) Bit

The No-Execute (NX) bit (bit 63 in page table entries) marks a page as non-executable. When the processor attempts to execute code from a page with the NX bit set, it generates a page fault. This feature prevents many security exploits, including buffer overflows that try to execute code on the stack or heap. The NX bit is only available in long mode (and PAE-enabled protected mode). The EFER.NXE bit in the Extended Feature Enable Register must be enabled before using NX.

2.14 Privilege Levels and Page Protection

The processor checks privilege at two levels: segment level (through the CPL in CS) and page level (through the U/S flag in page table entries). Code executing at ring 0 (kernel) can access both user and supervisor pages. Code executing at ring 3 (user) can only access user pages (U/S=1). The Writable flag (R/W) works similarly: in ring 3, writes are only allowed if R/W is set. This two-level protection allows the kernel to map its own pages as supervisor-only while mapping user pages as user-accessible.

2.15 Model-Specific Registers (MSRs)

Model-Specific Registers (MSRs) are processor-specific registers that control various features. They are accessed with the RDMSR and WRMSR instructions, which require ring 0 privilege. Important MSRs include: EFER (Extended Feature Enable Register, address 0xC0000080) controls long mode, NX, and other features; STAR (address 0xC0000081) controls the syscall instruction; LSTAR (address 0xC0000082) holds the syscall entry point; and FS_BASE and GS_BASE (addresses 0xC0000100 and 0xC0000101) control the FS and GS segment bases. `lowl` provides the `read_msr` and `write_msr` builtins for MSR access.

2.16 The SYSENTER and SYSEXIT Instructions

For fast system calls, Intel introduced SYSENTER and SYSEXIT. These instructions transition from ring 3 to ring 0 (SYSENTER) and back (SYSEXIT) with minimal overhead. They use MSRs to store the entry point, stack, and segment selectors, eliminating the need for a software interrupt or trap gate. The SYSCALL and SYSRET instructions provide similar functionality on AMD and Intel processors (the model differs slightly). Modern kernels typically use SYSCALL/SYSRET for better performance.

2.17 Interrupt Handling in Long Mode

In long mode, interrupt handlers are invoked with the same 64-bit registers available to normal code. The processor pushes the following onto the stack when entering an interrupt handler: the SS and RSP registers (only on privilege change), the RFLAGS register, the CS and RIP registers, and an error code (for some exception types). The handler returns with the IRETQ (IRET with 64-bit operand size) instruction, which pops all previously pushed values and restores the interrupted context.

2.18 Exception Types and Error Codes

Exceptions are events that disrupt normal execution. Page faults (exception 14) push an error code that indicates the nature of the fault: bit 0 indicates present vs. not-present, bit 1 indicates read vs. write, bit 2 indicates user vs. supervisor, and bit 4 indicates instruction fetch vs. data access. General protection faults (exception 13) push an error code that identifies the segment causing the fault. Double faults (exception 8) indicate an exception occurred while handling another exception.

2.19 Hardware Task Switching

The x86 architecture supports hardware task switching through the TSS. A task switch can be triggered by a CALL or JMP to a task gate, by an interrupt through a task gate, or by a software exception. The processor saves the current task's registers into its TSS and loads the new task's registers from its TSS. Hardware task switching is rarely used in modern operating systems because it is slower than software task switching and provides limited flexibility. lowl does not provide direct support for hardware tasks, focusing instead on software-managed threads.

2.20 Virtualization Extensions (VMX)

Intel's Virtualization Technology (VT-x) adds VMX (Virtual Machine Extensions) to the processor. VMX introduces two new modes: VMX root operation (hypervisor) and VMX non-root operation (guest). The hypervisor uses the VMLAUNCH and VMRESUME instructions to enter guest execution and the VMEXIT event to regain control. VMX is beyond the scope of lowl's base features but can be accessed through inline assembly or future extensions.

2.21 Advanced Vector Extensions (AVX and AVX-512)

AVX (Advanced Vector Extensions) increases SIMD register width from 128 bits (SSE) to 256 bits (AVX) and then to 512 bits (AVX-512). AVX also introduces a new instruction encoding (VEX prefix) that supports three-operand instructions (dest = src1 op src2). AVX-512 adds mask registers (K1-K7) for predicated execution, allowing operations on only selected elements of a vector. These extensions are critical for high-performance computing, and lowl's SIMD vector types map directly to them.

2.22 Memory Type Range Registers (MTRRs)

MTRRs control the caching behavior of physical memory regions. Each range can be marked as uncacheable, write-combining, write-through, write-protected, or write-back. The operating system must set MTRRs correctly for MMIO regions (which should be uncacheable) and RAM (which should be write-back). MTRRs are programmed through MSRs. lowl does not provide direct MTRR access but allows it through `read_msr` and `write_msr`.

2.23 Non-Maskable Interrupts (NMIs)

Non-Maskable Interrupts cannot be disabled by the CLI instruction. They are typically generated by hardware failures (memory errors, catastrophic hardware faults) or by the NMI button on some systems (for debugging). The NMI handler must be very careful not to cause additional exceptions and must save all registers. The processor automatically disables NMIs when handling an NMI, re-enabling them at the end of the handler.

2.24 System Management Mode (SMM)

System Management Mode (SMM) is a separate execution environment invoked by the System Management Interrupt (SMI). SMM runs in a separate address space and is invisible to the operating system. It is used for power management, thermal monitoring, and other platform-specific functions. SMM code is typically provided by the BIOS/UEFI and is not accessible to the OS. `lowl` does not support writing SMM handlers; this remains the domain of firmware.

2.25 Local APIC and Interrupt Routing

The Local Advanced Programmable Interrupt Controller (APIC) is part of every modern x86 CPU. It handles interrupt delivery, inter-processor interrupts (IPIs), and the timer. Each core has its own LAPIC, mapped into memory at a physical address (typically 0xFEE00000). The LAPIC is programmed through MMIO registers. The I/O APIC (part of the chipset) routes external interrupts to the LAPICs. `lowl` provides MMIO pointer support for accessing the LAPIC.

2.26 The TSC and High-Resolution Timing

The Time Stamp Counter (TSC) is a 64-bit counter that increments at the processor's base frequency. The `RDTSC` instruction reads the TSC. On modern processors, the TSC is invariant (runs at a fixed frequency regardless of power management) and synchronized across cores (sometimes requiring BIOS support). High-resolution timing uses `RDTSC` to measure short intervals with nanosecond precision. `lowl`'s `rdtsc` builtin provides access to the TSC.

2.27 Performance Monitoring Unit (PMU)

The PMU provides hardware counters for performance events: cache misses, branch mispredictions, cycles, and instructions retired. PMU programming uses MSRs to configure which events to count and to read the counters. This is essential for performance analysis of low-level code. `lowl` supports PMU access through MSR read/write, allowing custom profiling tools.

2.28 Machine Check Architecture (MCA)

Some errors (memory errors, bus errors, cache errors) are reported through the Machine Check Architecture (MCA). The processor logs error details in machine-check-specific registers and generates a machine check exception (exception 18). The operating system can read the MCA registers to diagnose hardware failures. MCA is complex and model-specific; `lowl` provides MSR access to enable MCA handling in higher-level code.

2.29 Security Features: SMEP, SMAP, and CET

Supervisor Mode Execution Prevention (SMEP) prevents the kernel from executing code from user pages. Supervisor Mode Access Prevention (SMAP) prevents the kernel from accessing data from user pages (with exceptions for explicit user access). Control-flow Enforcement Technology (CET) adds shadow stacks and indirect branch tracking to prevent return-oriented programming (ROP) attacks. These features are controlled through CR4 bits and MSRs. `lowl`'s `#[kernel]` attribute can hint to the compiler about SMEP/SMAP requirements.

2.30 Summary of x86_64 Fundamentals for `lowl` Programming

Understanding the x86_64 architecture is essential for effective `lowl` systems programming. You must understand the mode transitions (real to protected to long mode), the privilege ring system, the paging hierarchy, and the interrupt handling model. `lowl` abstracts many of these details but leaves you in full control when you need it. The builtins for MSRs, control registers, and port I/O

give you access to every hardware feature. The following chapter applies this knowledge to the boot process, showing how lowl code transitions from the bootloader to a running kernel.

Chapter 3: Booting and System Initialization

3.1 The Boot Process Overview

When you press the power button, the processor begins executing code from a fixed physical address: FFFF:0000 in real mode, which maps to physical address 0xFFFFFFF0 (16 bytes below 4GB). This address contains the system BIOS (or UEFI) firmware. The firmware initializes hardware, performs Power-On Self-Test (POST), and then loads the bootloader from the boot device. The bootloader loads the operating system kernel and transfers control. lowl can generate code for each stage: boot sector, second-stage bootloader, and kernel.

3.2 BIOS vs UEFI Boot Modes

Traditional BIOS (Basic Input/Output System) boots in real mode, providing software interrupt services for disk access, video output, and other hardware functions. BIOS expects the boot sector (first 512 bytes of the boot device) to end with the signature 0xAA55. UEFI (Unified Extensible Firmware Interface) boots in protected or long mode, using the GPT partition table and loading PE/COFF executables directly. lowl supports both: the `-f boot` format produces a BIOS boot sector; UEFI boot requires building a PE executable (planned for future versions).

3.3 The Boot Sector Structure

A boot sector is exactly 512 bytes, loaded by the BIOS at physical address 0x7C00. The boot sector must end with the two-byte signature 0x55, 0xAA at offset 510. The lowl boot output format generates this signature automatically. The boot sector typically contains: a short real-mode program that sets up a minimal stack, loads additional sectors from disk, and transitions to protected or long mode. The remaining space in the boot sector (usually about 510 bytes minus code size) is too small for a full kernel, so the boot sector usually loads a larger second-stage bootloader.

3.4 Real-Mode Lowl Code for Boot Sectors

lowl can generate real-mode code (16-bit) for boot sectors, but most lowl code assumes 64-bit long mode. Therefore, boot sectors are typically written with a mix of lowl's inline assembly capability and careful control of output. A typical boot sector in lowl uses the `-f boot` format, which automatically includes the 0xAA55 signature and sets up basic real-mode execution. The boot code loads sectors from disk using BIOS interrupt 0x13, transfers to protected mode, then to long mode, and finally calls the kernel's entry point.

3.5 Loading Sectors from Disk with BIOS Int 0x13

BIOS interrupt 0x13 provides disk access services. The AH register selects the function: 0x02 reads sectors, 0x08 gets drive parameters, 0x42 reads using LBA (Logical Block Addressing) for large disks. The CHS (cylinder-head-sector) addressing used by older BIOS functions limits access to 8GB disks. Modern bootloaders use the extended read function (0x42) with LBA addressing, which supports disks up to 2TB. lowl provides `int 0x13` through inline assembly, or you can use port I/O on some systems (more complex).

3.6 The A20 Gate and Why It Matters

The A20 gate controls access to address line 20 (the 21st bit of the physical address). On older IBM PCs, the 80286's address line 20 was disabled by default for compatibility with software that expected addresses to wrap (like the 8086 did). To access memory above 1MB, the A20 gate must be enabled. Modern chipsets enable A20 by default, but many bootloaders still enable it explicitly. The typical sequence: check if A20 is already enabled, try the fast A20 method (port 0x92), and fall back to the keyboard controller method if needed. `lowl` provides `port_read8` and `port_write8` for these operations.

3.7 Setting Up the Global Descriptor Table (GDT)

Before switching to protected mode, you must set up a valid GDT. For a transition to long mode, you need a GDT with: a null descriptor (selector 0), a 64-bit code segment, a 64-bit data segment, and optionally 32-bit compatibility segments. The GDT is stored in memory, and the `lgdt` instruction loads the GDTR register. The GDTR structure contains the size of the GDT (bytes minus one) and its base address. After loading the GDT, you set the PE bit in CR0 and perform a far jump to load CS with a protected-mode selector.

3.8 Transitioning from Real Mode to Protected Mode

The transition from real mode to protected mode involves several steps. First, disable interrupts (CLI). Second, load the GDT (LGDT). Third, set the PE bit in CR0 (OR with 1). Fourth, perform a far jump to a label in the code segment (JMP 0x08:protected_mode_entry). The far jump reloads the CS register with the protected-mode selector and the processor starts fetching instructions in protected mode. At this point, all segment registers still contain real-mode values, so you must reload DS, ES, SS, and FS/GS with protected-mode data selectors.

3.9 Setting Up Page Tables for Long Mode

Long mode requires paging to be enabled. The page tables must identity-map the first few megabytes (where the bootloader and kernel reside) and also map the kernel at its preferred virtual address (typically 0xFFFFFFFF80000000 for a higher-half kernel). Setting up page tables involves: allocating a PML4 page, a PDPT page, a PD page, and one or more PT pages. For identity mapping, you fill the page tables with entries that map virtual addresses to the same physical addresses. For the higher-half mapping, you map the kernel's physical location to its virtual address.

3.10 Enabling PAE and Long Mode

Physical Address Extension (PAE) is required for long mode. Enable PAE by setting the PAE bit (bit 5) in CR4. Then, enable long mode by setting the LME bit (bit 8) in the EFER MSR (address 0xC0000080). Finally, enable paging by setting the PG bit (bit 31) in CR0. At this point, the processor is in compatibility mode (32-bit) because CS is still a 32-bit segment. To enter 64-bit long mode, you must load CS with a 64-bit code segment descriptor (which has the L bit set). This usually happens through a far jump or a return from an interrupt.

3.11 The Higher-Half Kernel Design

Most 64-bit kernels map themselves to the higher half of the virtual address space (addresses with bit 47 set, typically 0xFFFFFFFF80000000 and above). This leaves the lower half (canonical 48-bit addresses) for user space. The higher-half mapping is set up in the page tables before switching to long mode. The kernel's link address (the address where it expects to be loaded) is set to the

higher-half virtual address. The bootloader loads the kernel at a physical address (e.g., 0x100000) but the page tables map that physical address to the higher-half virtual address.

3.12 Multiboot Specification Compatibility

The Multiboot Specification defines a standard interface between bootloaders (like GRUB) and kernels. A Multiboot-compliant kernel includes a special header in its first 8KB that the bootloader recognizes. The bootloader passes information to the kernel: memory map, boot device, command line, and module list. The kernel is loaded in 32-bit protected mode, and the kernel is responsible for switching to long mode. lowl's kernel output format (`-f kernel`) includes Multiboot header generation, making lowl kernels loadable by GRUB.

3.13 GRUB and Multiboot2

GRUB (GRand Unified Bootloader) is the most common bootloader for x86 systems. It supports Multiboot (version 1) and Multiboot2 (version 2). Multiboot2 adds features: improved memory map (including UEFI memory regions), framebuffer information, and more detailed boot information. GRUB loads the kernel, sets up basic paging (identity mapping of the first 4MB), and jumps to the kernel's entry point in 32-bit protected mode. The kernel must then set up its own page tables and switch to long mode.

3.14 UEFI Boot Process

UEFI boots differently from BIOS. The firmware loads an EFI application (a PE/COFF executable) from the EFI System Partition (ESP). The EFI application runs in protected or long mode with flat addressing (segmentation is essentially disabled). EFI provides services through a system table (passed to the entry point) and boot services (memory allocation, disk access, file system access). The kernel can either exit boot services (taking full control of the hardware) or continue using EFI runtime services. lowl plans to support PE output for UEFI boot in a future version.

3.15 The Kernel Entry Point

The kernel's entry point is the first code executed after the bootloader transfers control. For a BIOS-booted system, the entry point is called in 32-bit protected mode (or compatibility mode). The kernel must: set up its own stack (the bootloader stack is often too small), configure page tables, switch to long mode, and then call the main kernel initialization function. lowl's `#[kernel]` attribute marks the entry point and tells the compiler to generate code that works in this early environment (no SSE, no floating point until enabled).

3.16 Minimal C Runtime (CRT) for lowl

Even low-level kernels need some runtime support: a stack, a way to call functions, and basic memory operations. lowl generates code that expects a stack pointer (RSP) to be set up correctly. The kernel must provide its own `memcpy`, `memset`, and `memcmp` implementations (or use lowl's builtins). The compiler does not add hidden runtime dependencies, so the kernel does not need a full C library. String operations (character arrays) are handled inline or through builtin functions.

3.17 Physical Memory Detection

The kernel must know which physical memory regions are usable. The bootloader provides this information: for Multiboot, through the memory map tags; for BIOS bootloaders, through the E820 memory map (obtained by calling INT 0x15 with EAX=0xE820). The memory map consists of

entries with base address, length, and type (usable, reserved, ACPI reclaimable, ACPI NVS, bad). The kernel uses this information to initialize the physical memory allocator, marking usable regions as available and reserving regions used by the kernel itself.

3.18 Setting Up the Physical Memory Allocator

The physical memory allocator tracks free and used physical pages. lowl provides a `PhysicalAllocator` class that uses a red-black tree of memory regions for efficient allocation. The allocator is initialized with the memory map from the bootloader. Allocation requests return physical addresses aligned to page boundaries. The allocator can also query memory regions, split and merge blocks, and register protection flags. The red-black tree provides $O(\log n)$ allocation and deallocation, which is acceptable for kernel bootstrap.

3.19 Enabling SSE and AVX

Before using SIMD instructions, the kernel must enable them. For SSE, set the OSFXSR bit (bit 9) and OSXMMEXCPT bit (bit 10) in CR4. For AVX and AVX-512, additionally set the OSXSAVE bit (bit 18) in CR4 and use XSAVE/XRSTOR for context switching. The kernel should also mask SIMD exceptions by setting the appropriate bits in the MXCSR register (default mask bits 0x1F80). lowl's builtin `mxcsr_set` can configure this. The kernel must also save and restore SSE/AVX state during context switches.

3.20 Setting Up the Interrupt Descriptor Table (IDT)

The IDT must be set up before enabling interrupts. Each interrupt descriptor is 16 bytes in long mode (2 bytes for size?). The IDT entries contain: the handler's segment selector (typically the kernel code segment), the handler's offset (split across two fields), and the descriptor type (interrupt gate, trap gate, or task gate). Interrupt gates clear the interrupt flag when entered, preventing nested interrupts. Trap gates leave the interrupt flag unchanged. The `lidt` instruction loads the IDT pointer. lowl's `#[interrupt]` attribute automatically generates the correct entry format.

3.21 Hardcoding Interrupt Stubs

For simplicity, many kernels hardcode interrupt handler stubs in assembly or lowl with inline asm. Each stub pushes an error code (if not provided by the CPU), pushes the interrupt number, and jumps to a common handler. The common handler saves registers, calls the high-level handler, restores registers, and returns with IRETQ. The stubs are stored in an array of function pointers indexed by interrupt number. lowl can generate these stubs using its `#[interrupt]` attribute on a per-function basis.

3.22 Enabling Interrupts and the HLT Instruction

Once the IDT is loaded and the PIC or APIC is configured, the kernel can enable interrupts with the `sti` instruction (provided by `enable_interrupts()`). The kernel then typically enters an idle loop that calls `hlt` (halt the processor until the next interrupt). The `hlt` instruction reduces power consumption and stops instruction execution until an interrupt occurs. The kernel must ensure that interrupts are enabled before calling `hlt`, otherwise the processor will never wake up.

3.23 Bootstrapping to the Final Kernel

The kernel startup process has several phases: early boot (real mode), protected mode transition, long mode transition, kernel entry (higher-half or identity mapped), physical memory detection, virtual memory setup, interrupt initialization, and scheduler start. lowl code for each phase is compiled separately or with careful control over features. The bootloader phase (real mode) is typically minimal and written in assembly or lowl with `-f boot`. The kernel proper is compiled with `-f kernel` or `-f elf` and linked at the final virtual address.

3.24 Example: A Minimal Boot Sector in lowl (Working Example)

A minimal boot sector written in lowl represents the first 512 bytes of executable code that the BIOS loads and executes. This boot sector must operate in 16-bit real mode, as the processor starts in this mode after reset. The boot sector's responsibilities include: setting up a minimal execution environment, printing status messages using BIOS interrupts, loading additional code from disk (since only 512 bytes are loaded initially), and optionally transitioning to protected mode or long mode. Because lowl normally generates 64-bit code for long mode, the boot sector requires careful handling of real-mode operations through inline assembly, explicit segment management, and precise control over code generation. The compiler's `-f boot` output format automatically creates the 512-byte binary with the correct 0xAA55 signature at offset 510, but the programmer must ensure the code fits within the 510-byte limit (the signature occupies the last two bytes). The following working example demonstrates a complete, minimal boot sector written in lowl that prints a message to the screen, halts, and can be extended to load a kernel.

Complete Working Boot Sector Source Code (miniboot.lowl):

lowl

```
// miniboot.lowl - Minimal Working Boot Sector in lowl
//
// This boot sector prints "lowl booting..." to the screen using BIOS
// interrupt 0x10, then halts. It fits in 512 bytes and includes the
// required 0xAA55 boot signature.
//
// Compilation:
//   lowlc miniboot.lowl -o miniboot.asm -f boot
//   nasm -f bin miniboot.asm -o miniboot.bin
//
// Run in QEMU:
//   qemu-system-x86_64 -drive format=raw,file=miniboot.bin
//
// Write to USB (CAUTION: overwrites target):
//   dd if=miniboot.bin of=/dev/sdb bs=512 count=1
//
// Copyright (c) 2026 Anthony Matarazzo - MIT License

//
=====
// CONSTANTS
```

```
//
=====

// BIOS teletype output function (AH=0x0E, AL=char, BH=page, BL=color)
const BIOS_TELETYPE: u8 = 0x0E

// Video page zero (default)
const VIDEO_PAGE: u8 = 0x00

// Carriage return and line feed for newline
const CR: u8 = 0x0D
const LF: u8 = 0x0A

// Boot sector magic signature (must appear at offset 510)
const BOOT_SIGNATURE: u16 = 0xAA55

//
=====

// BOOT SECTOR ENTRY POINT
//
=====

/// Boot sector entry point - executed by BIOS at physical address 0x7C00
/// The BIOS loads this 512-byte sector and jumps to 0x0000:0x7C00
#[boot_sector]
#[section(".boot")]
fn _start():
    // Step 1: Disable interrupts during initial setup
    asm("cli")

    // Step 2: Set up a minimal stack (grows downward from 0x7C00)
    // We set SS=0x0000 and SP=0x7C00, so the stack occupies 0x7C00 down to
    // 0x6C00
    asm("
        xor ax, ax          ; AX = 0
        mov ss, ax          ; SS = 0x0000 (stack segment)
        mov sp, 0x7C00      ; SP = 0x7C00 (stack pointer)
        mov ds, ax          ; DS = 0x0000 (data segment)
        mov es, ax          ; ES = 0x0000 (extra segment)
    ")

    // Step 3: Save the boot drive number (BIOS passes this in DL register)
    // We'll store it for later use if we need to load more sectors
    let boot_drive: u8
    asm("mov %0, dl" : "=r"(boot_drive))

    // Step 4: Print the boot message using BIOS interrupt 0x10
    print_string(&boot_message)
```

```

// Step 5: Print a newline for clean output
print_char(CR)
print_char(LF)

// Step 6: Print success indicator
print_string(&ready_message)

// Step 7: Enable interrupts (optional, but good practice)
asm("sti")

// Step 8: Halt the processor (infinite loop)
// In a real bootloader, we would load a kernel here
while true:
    asm("hlt")

//
=====
// BIOS PRINTING ROUTINES
//
=====

/// Print a single character using BIOS teletype output
/// @param ch: ASCII character to print
/// This function uses BIOS interrupt 0x10 with AH=0x0E
/// It preserves all registers except AX and BX
fn print_char(ch: u8):
    asm("
        mov ah, 0x0E      ; BIOS teletype function
        mov al, %0        ; Character to print
        xor bh, bh        ; Page 0
        int 0x10          ; Call BIOS video service
    " : : "r"(ch) : "ax", "bx")

/// Print a null-terminated string using BIOS
/// @param str_ptr: pointer to string (in code or data segment)
/// The string must end with a zero byte (null terminator)
fn print_string(str_ptr: u32):
    asm("
        push si           ; Save SI register
        mov si, %0        ; Load string address
        .print_loop:
            lodsb          ; Load byte at [SI] into AL, increment SI
            test al, al     ; Check if byte is zero (end of string)
            jz .print_done ; If zero, exit loop
            mov ah, 0x0E    ; BIOS teletype function
            xor bh, bh     ; Page 0
            int 0x10       ; Print character
            jmp .print_loop ; Continue to next character
        .print_done:
    ")

```

```

        pop si                ; Restore SI register
    " : : "r"(str_ptr) : "si", "ax", "bx")

//
=====
// MESSAGE STRINGS (stored in .rodata section)
//
=====

// Boot banner message
#[section(".rodata")]
const boot_message: array<u8, 22> = [
    'l' as u8, 'o' as u8, 'w' as u8, 'l' as u8, ' ' as u8,
    'b' as u8, 'o' as u8, 'o' as u8, 't' as u8, 'i' as u8,
    'n' as u8, 'g' as u8, '.' as u8, '.' as u8, '.' as u8, 0
]

// Ready message after boot
#[section(".rodata")]
const ready_message: array<u8, 15> = [
    'R' as u8, 'e' as u8, 'a' as u8, 'd' as u8, 'y' as u8,
    ' ' as u8, 't' as u8, 'o' as u8, ' ' as u8, 'l' as u8,
    'o' as u8, 'a' as u8, 'd' as u8, 0
]

//
=====
// BOOT SECTOR PADDING AND SIGNATURE
//
=====

// The boot sector must be exactly 512 bytes with the signature at offset
510
// The compiler's -f boot flag automatically handles this, but we include
// explicit padding to ensure correct size.
// The following assembly directive pads the binary to 510 bytes,
// then the boot signature is placed at bytes 510-511.

// Inline assembly to fill remaining space up to 510 bytes
asm("
    times 510 - ($ - $$) db 0
")

// Boot signature - must be 0xAA55 at offset 510
// This is automatically placed by the -f boot flag, but we declare it
explicitly
const boot_sig: u16 = 0xAA55

//

```

```
=====
// END OF BOOT SECTOR
//
=====
```

Explanation of the Code:

The boot sector begins execution at `_start`, which is marked with `#[boot_sector]`. This attribute tells the compiler to generate 16-bit real-mode code (not 64-bit) and to place the code at offset 0x7C00 where the BIOS loads it.

The first instruction disables interrupts with `cli` to prevent any interruptions during the critical setup phase. The stack is set up with `SS=0x0000` and `SP=0x7C00`. This creates a stack that grows downward from the boot sector's load address. The code segment (CS) is already set by the BIOS, but we set DS and ES to zero for flat addressing.

The boot drive number is saved from the DL register. The BIOS passes this value so the bootloader knows which disk it was loaded from. This is essential if the bootloader needs to load additional sectors.

The `print_string` function uses BIOS interrupt 0x10 with AH=0x0E (teletype output). It iterates through a null-terminated string, printing each character. The `lodsbyte` instruction loads a byte from the address in SI and automatically increments SI. When a zero byte is encountered, the loop ends.

The `print_char` function is a simpler version that prints a single character. It loads the character into AL, sets AH=0x0E, and calls INT 0x10.

The boot message strings are stored in the `.rodata` section. The `boot_message` contains "lowl booting..." and the `ready_message` contains "Ready to load". Each string ends with a zero byte to mark the end.

The `asm("times 510 - ($ - $$) db 0")` directive fills the remaining space up to 510 bytes with zeros. This ensures the boot sector is exactly 512 bytes. The boot signature `0xAA55` is placed at the end, which the BIOS checks to verify this is a valid boot sector. The compiler's `-f boot` flag automatically handles this placement.

After printing the messages, the code enters an infinite loop with the `hlt` instruction. This halts the processor until the next interrupt, saving power. In a real bootloader, this is where you would load a kernel from disk.

Building and Running:

bash

```
# Step 1: Compile the lowl source to NASM assembly
lowlc miniboot.lowl -o miniboot.asm -f boot

# Step 2: Assemble to binary format (NASM produces flat binary for boot
sectors)
nasm -f bin miniboot.asm -o miniboot.bin

# Step 3: Verify the file size is exactly 512 bytes
ls -la miniboot.bin
```

```
# Should show: -rw-r--r-- 1 user user 512 miniboot.bin

# Step 4: Run in QEMU emulator
qemu-system-x86_64 -drive format=raw,file=miniboot.bin

# Alternative: Run with Bochs debug port output
qemu-system-x86_64 -drive format=raw,file=miniboot.bin -serial stdio

# Step 5: (CAUTION) Write to a USB drive (replace /dev/sdb with your target)
# dd if=miniboot.bin of=/dev/sdb bs=512 count=1
```

Expected Output in QEMU:

When run in QEMU, the boot sector displays:

text

```
lowl booting...
Ready to load
```

Then the system halts (you may need to press Ctrl+Alt+2 to see the QEMU monitor, or the emulator will idle).

Extending the Bootloader to Load a Kernel:

To make this a functional bootloader that loads a kernel, add this function before the infinite loop:

lowl

```
/// Load kernel sectors from disk using BIOS INT 0x13
/// @param drive: boot drive number (from BIOS)
/// @param lba: Logical Block Address (sector number to read)
/// @param dest_seg: destination segment for loading
/// @param dest_off: destination offset
/// @param sectors: number of sectors to read
fn load_sectors(drive: u8, lba: u32, dest_seg: u16, dest_off: u16, sectors:
u16) -> bool:
    // Disk Address Packet for LBA reads (supports large disks)
    struct DAP:
        size: u8 = 0x10
        reserved: u8 = 0
        count: u16 = sectors
        offset: u16 = dest_off
        segment: u16 = dest_seg
        lba_low: u32 = lba
        lba_high: u32 = 0

    let dap_ptr = &DAP() as u32
    let mut result: u8

    asm(
```

```

    mov dl, %2
    mov ah, 0x42
    mov si, %1
    int 0x13
    jc .error
    mov %0, 1
    jmp .done
.error:
    mov %0, 0
.done:
" : "=r"(result) : "r"(dap_ptr), "r"(drive) : "ax", "si", "dx")

return result != 0

```

Then replace the infinite loop with:

lowl

```

// Load kernel from sector 1 (the sector after the boot sector)
// Load to address 0x1000:0x0000 (physical address 0x10000)
if load_sectors(boot_drive, 1, 0x1000, 0x0000, 64):
    print_string(&load_success)
    // Jump to kernel
    asm("jmp 0x1000:0x0000")
else:
    print_string(&load_failed)
    while true:
        asm("hlt")

```

Size Optimization Tips:

To keep the boot sector under 512 bytes, follow these practices:

1. Use short variable names and avoid large stack frames
2. Combine multiple operations into single inline assembly blocks
3. Use byte-sized operations where possible (u8 instead of u32)
4. Place strings in `.rodata` and reference them by pointer
5. Avoid function call overhead for critical sections (use inline)
6. Use BIOS services instead of implementing complex algorithms

Debugging Tips:

When developing boot sectors, use QEMU's built-in debugging features:

bash

```

# Run with GDB debugging
qemu-system-x86_64 -drive format=raw,file=miniboot.bin -s -S

# In another terminal:
gdb

```

```
(gdb) target remote localhost:1234
(gdb) break *0x7C00
(gdb) continue

# Or use Bochs debug port output
qemu-system-x86_64 -drive format=raw,file=miniboot.bin -debugcon stdio
```

Common Issues and Solutions:

Issue	Cause	Solution
Boot sector not found	Missing 0xAA55 signature	Ensure <code>-f boot</code> flag is used
Garbage on screen	DS segment not set correctly	Set DS=0x0000 before printing
Hangs after printing	Stack overflow or corruption	Verify SS and SP are set correctly
Disk read fails	Wrong drive number	Save DL register at entry
Code too large (>512 bytes)	Too many instructions	Optimize, move code to kernel

This working example demonstrates a complete, minimal boot sector in lowl that compiles, assembles, and runs in QEMU. It serves as a foundation for building more complex bootloaders that load kernels, enter protected mode, and eventually transition to 64-bit long mode.

3.25 Example: Multiboot Kernel Entry (Complete Working Example)

A Multiboot kernel in lowl starts with a special header placed at offset 0. The header contains the magic number 0x1BADB002, flags (for alignment, memory map, video mode), and a checksum. The kernel is compiled with `-f elf` and linked with a linker script that places the header at the start of the .text section. The entry point is marked `#[kernel]` and is called by GRUB when loading is complete. The kernel receives three parameters: the magic number (0x2BADB002 for Multiboot), a pointer to the Multiboot information structure, and the bootloader's stack pointer. This example demonstrates a complete Multiboot-compliant kernel that can be loaded by GRUB, GRUB2, or any Multiboot-compliant bootloader.

Complete Multiboot Kernel Source Code (kernel.lowl):

lowl

```
// kernel.lowl - Multiboot-compliant 64-bit Kernel
//
// This kernel implements the Multiboot Specification version 1.0,
// allowing it to be loaded by GRUB, GRUB2, and other Multiboot bootloaders.
// It transitions from 32-bit protected mode (where GRUB loads it) to
// 64-bit long mode, sets up identity paging, initializes the GDT and IDT,
// and provides a complete memory management foundation.
//
// Compilation:
// lowlc kernel.lowl -o kernel.asm -f elf -O2
```

```

//  nasm -f elf64 kernel.asm -o kernel.o
//  ld -T link.ld -o kernel.bin kernel.o
//  cp kernel.bin /boot/kernel.bin
//
// GRUB menu entry (grub.cfg):
//  menuentry "lowl Kernel" {
//      multiboot /boot/kernel.bin
//      boot
//  }
//
// Run in QEMU with GRUB:
//  qemu-system-x86_64 -cdrom lowl.iso
//
// Copyright (c) 2026 Anthony Matarazzo - MIT License

//
=====
// MULTIBOOT HEADER CONSTANTS
//
=====

// Multiboot magic numbers
const MULTIBOOT_MAGIC: u32 = 0x1BADB002      // Header magic (kernel)
const MULTIBOOT_BOOT_MAGIC: u32 = 0x2BADB002 // Boot magic (GRUB passes
this)
const MULTIBOOT_HEADER_ARCH: u32 = 0         // i386 architecture

// Multiboot header flags
const MULTIBOOT_PAGE_ALIGN: u32 = 1 << 0    // Align modules on page
boundaries
const MULTIBOOT_MEMORY_INFO: u32 = 1 << 1    // Provide memory map
const MULTIBOOT_VIDEO_MODE: u32 = 1 << 2     // Set video mode
const MULTIBOOT_ADDR_GRAPHICS: u32 = 1 << 3  // Use address fields for
graphics
const MULTIBOOT_HEADER_FLAGS: u32 = MULTIBOOT_PAGE_ALIGN |
MULTIBOOT_MEMORY_INFO

// Multiboot information structure tags
const MULTIBOOT_INFO_MEMORY: u32 = 1         // Memory info present
const MULTIBOOT_INFO_BOOT_DEVICE: u32 = 2    // Boot device present
const MULTIBOOT_INFO_CMDLINE: u32 = 4        // Command line present
const MULTIBOOT_INFO_MODS: u32 = 8           // Modules present
const MULTIBOOT_INFO_AOUT_SYMS: u32 = 16     // Symbol table present
const MULTIBOOT_INFO_ELF_SYMS: u32 = 32      // ELF symbol table present
const MULTIBOOT_INFO_MEM_MAP: u32 = 64       // Memory map present
const MULTIBOOT_INFO_DRIVE_INFO: u32 = 128   // Drive info present
const MULTIBOOT_INFO_CONFIG_TABLE: u32 = 256 // Config table present
const MULTIBOOT_INFO_BOOT_LOADER_NAME: u32 = 512 // Boot loader name
present

```

```

const MULTIBOOT_INFO_APM_TABLE: u32 = 1024    // APM table present
const MULTIBOOT_INFO_VBE: u32 = 2048         // VBE info present

// Memory region types (from Multiboot spec)
const MULTIBOOT_MEMORY_AVAILABLE: u32 = 1      // Free RAM, usable by OS
const MULTIBOOT_MEMORY_RESERVED: u32 = 2      // Reserved (BIOS, hardware)
const MULTIBOOT_MEMORY_ACPI_RECLAIM: u32 = 3   // ACPI reclaimable
const MULTIBOOT_MEMORY_NVS: u32 = 4           // ACPI NVS (non-volatile
storage)
const MULTIBOOT_MEMORY_BADRAM: u32 = 5         // Defective RAM

//
=====
// MULTIBOOT HEADER STRUCTURES
//
=====

/// Multiboot header structure (must be in the first 8KB of the kernel)
/// This structure is placed at offset 0 of the kernel binary
#[packed]
#[section(".multiboot")]
struct MultibootHeader:
    magic: u32 = MULTIBOOT_MAGIC
    flags: u32 = MULTIBOOT_HEADER_FLAGS
    checksum: u32 = 0 - (MULTIBOOT_MAGIC + MULTIBOOT_HEADER_FLAGS)

    // header_addr, load_addr, load_end_addr, bss_end_addr, entry_addr
    // Only present if flags bit 16 is set (we don't set it, so these are
optional)
    // We include them for completeness but GRUB ignores them when bit 16 is
0

    // Video mode fields (only if MULTIBOOT_VIDEO_MODE flag is set)
    // We don't request video mode, so these are not included

/// Multiboot memory map entry (16 bytes per entry)
#[packed]
struct MultibootMemoryMapEntry:
    size: u32                                // Size of this entry (usually 20 or 24)
    base_addr_low: u32                        // Lower 32 bits of base address
    base_addr_high: u32                       // Upper 32 bits of base address
    length_low: u32                           // Lower 32 bits of length
    length_high: u32                          // Upper 32 bits of length
    mem_type: u32                             // Type of memory region (1=usable,
2=reserved, etc.)

/// Multiboot information structure (passed by GRUB in EBX)
#[packed]
struct MultibootInfo:

```

```

    flags: u32                // Which fields are valid
    mem_lower: u32            // Lower memory in KB (below 1MB)
    mem_upper: u32           // Upper memory in KB (above 1MB)
    boot_device: u32         // Boot device (BIOS drive number)
    cmdline: u32             // Command line string pointer
    mods_count: u32          // Number of modules loaded
    mods_addr: u32           // Address of modules array
    elf_syms: array<u32, 4>   // ELF symbol table info (or a.out)
    mmap_length: u32         // Length of memory map (in bytes)
    mmap_addr: u32           // Address of memory map array
    drives_length: u32       // Length of drives info
    drives_addr: u32         // Address of drives info
    config_table: u32        // ROM configuration table
    boot_loader_name: u32    // Boot loader name string
    apm_table: u32           // APM table
    vbe_control_info: u32    // VBE control info
    vbe_mode_info: u32       // VBE mode info
    vbe_mode: u16            // Current VBE mode
    vbe_interface_seg: u16   // VBE interface segment
    vbe_interface_off: u16   // VBE interface offset
    vbe_interface_len: u16   // VBE interface length

//
=====
// MULTIBOOT HEADER INSTANCE (placed at kernel start)
//
=====

/// The Multiboot header must be in the first 8KB of the kernel
/// We place it in the .multiboot section, which the linker script ensures
/// comes at the very beginning of the binary.
#[section(".multiboot")]
const multiboot_header: MultibootHeader = MultibootHeader()

//
=====

// KERNEL ENTRY POINT (called by GRUB in 32-bit protected mode)
//
=====

/// Kernel entry point - called by GRUB after loading
/// @param magic: Must equal MULTIBOOT_BOOT_MAGIC (0x2BADB002)
/// @param info: Pointer to MultibootInfo structure (passed by GRUB in EBX)
/// @param stack: Bootloader stack pointer (optional, GRUB provides a stack)
#[kernel]
#[section(".text.entry")]
fn kernel_entry(magic: u32, info: ptr<MultibootInfo>, stack: u32) -> u32:
    // Step 1: Verify the magic number to ensure we were loaded by a
    Multiboot loader

```

```

if magic != MULTIBOOT_BOOT_MAGIC:
    // Not a Multiboot-compliant loader - halt with error
    print_string("Error: Not loaded by Multiboot-compliant
bootloader\r\n")
    while true:
        halt()
    return 1

// Step 2: Disable interrupts during kernel setup
disable_interrupts()

// Step 3: Set up our own stack (the bootloader's stack may be small or
unsafe)
// We allocate a 16KB stack at a known safe address
const KERNEL_STACK_SIZE: u64 = 16384 // 16KB
static kernel_stack: array<u8, KERNEL_STACK_SIZE> = [0;
KERNEL_STACK_SIZE]
let stack_top = &kernel_stack as u64 + KERNEL_STACK_SIZE
asm("mov rsp, %0" : : "r"(stack_top))

// Step 4: Clear BSS section (zero-initialized static variables)
clear_bss()

// Step 5: Parse Multiboot information structure
parse_multiboot_info(info)

// Step 6: Set up Global Descriptor Table for 64-bit mode
setup_gdt()

// Step 7: Set up identity page tables (maps first 2GB physical to same
virtual)
setup_identity_paging()

// Step 8: Enable PAE (Physical Address Extension) - required for long
mode
let cr4 = read_cr4()
write_cr4(cr4 | (1 << 5))

// Step 9: Enable Long Mode (set EFER.LME)
let efer = read_msr(0xC0000080)
write_msr(0xC0000080, efer | (1 << 8))

// Step 10: Enable paging (set CR0.PG)
let cr0 = read_cr0()
write_cr0(cr0 | (1 << 31))

// Step 11: Load 64-bit GDT and jump to 64-bit code segment
// This is done via a far jump to .long_mode_entry
asm("

```

```

    lgdt [gdt64_ptr]
    jmp 0x08:.long_mode
    .long_mode:
    mov ax, 0x10
    mov ds, ax
    mov es, ax
    mov fs, ax
    mov gs, ax
    mov ss, ax
")

// Step 12: Call the main kernel function (now running in 64-bit long
mode)
let result = kmain(info)

// Step 13: Return to bootloader (should never happen for kernel)
return result

//
=====
// BSS CLEARING
//
=====

/// Clear the BSS section (zero-initialized static variables)
/// The linker provides symbols _bss_start and _bss_end for the BSS section
extern _bss_start: u8
extern _bss_end: u8

fn clear_bss():
    let start = &_bss_start as u64
    let end = &_bss_end as u64
    let size = end - start

    for i in 0..size:
        let ptr = (start + i) as ptr<u8>
        ptr[0] = 0

//
=====
// MULTIBOOT INFORMATION PARSING
//
=====

/// Parse the Multiboot information structure and print memory details
static total_ram: u64 = 0
static memory_map_entries: u64 = 0

fn parse_multiboot_info(info: ptr<MultibootInfo>):

```

```

let flags = info.flags

// Print boot loader name (if available)
if (flags & MULTIBOOT_INFO_BOOT_LOADER_NAME) != 0:
    let boot_loader = info.boot_loader_name as ptr<u8>
    print_string("Boot loader: ")
    print_string_ptr(boot_loader)
    print_string("\r\n")

// Print command line (if available)
if (flags & MULTIBOOT_INFO_CMDLINE) != 0:
    let cmdline = info.cmdline as ptr<u8>
    print_string("Command line: ")
    print_string_ptr(cmdline)
    print_string("\r\n")

// Print memory information
if (flags & MULTIBOOT_INFO_MEMORY) != 0:
    print_string("Lower memory: ")
    print_dec(info.mem_lower as u64)
    print_string(" KB\r\n")
    print_string("Upper memory: ")
    print_dec(info.mem_upper as u64)
    print_string(" KB\r\n")

// Parse memory map (most important for kernel)
if (flags & MULTIBOOT_INFO_MEM_MAP) != 0:
    parse_memory_map(info.mmap_addr, info.mmap_length)

// Print module information (if any)
if (flags & MULTIBOOT_INFO_MODS) != 0:
    print_string("Modules loaded: ")
    print_dec(info.mods_count as u64)
    print_string("\r\n")

/// Parse the memory map entries from Multiboot
fn parse_memory_map(mmap_addr: u32, mmap_length: u32):
    print_string("Memory map:\r\n")
    print_string(" Base Address           Length           Type\r\n")
    print_string("-----")
\r\n")

    let mut offset: u32 = 0
    while offset < mmap_length:
        let entry = (mmap_addr + offset) as ptr<MultibootMemoryMapEntry>

        // Construct 64-bit base address from low and high parts
        let base = (entry.base_addr_high as u64) << 32 | entry.base_addr_low
as u64

```

```

let length = (entry.length_high as u64) << 32 | entry.length_low as
u64

// Determine type string
let type_str = match entry.mem_type:
    case MULTIBOOT_MEMORY_AVAILABLE:
        "Available"
    case MULTIBOOT_MEMORY_RESERVED:
        "Reserved"
    case MULTIBOOT_MEMORY_ACPI_RECLAIM:
        "ACPI Reclaim"
    case MULTIBOOT_MEMORY_NVS:
        "ACPI NVS"
    case MULTIBOOT_MEMORY_BADRAM:
        "Bad RAM"
    default:
        "Unknown"

// Print entry
print_hex(base)
print_string(" ")
print_hex(length)
print_string(" ")
print_string(type_str)
print_string("\r\n")

// Count usable RAM for total memory
if entry.mem_type == MULTIBOOT_MEMORY_AVAILABLE:
    total_ram = total_ram + length
    memory_map_entries = memory_map_entries + 1

offset = offset + entry.size

print_string("Total usable RAM: ")
print_dec(total_ram / (1024 * 1024))
print_string(" MB\r\n")

//
=====
// GLOBAL DESCRIPTOR TABLE FOR 64-BIT MODE
//
=====

/// GDT entry structure (8 bytes)
#[packed]
struct GDTEEntry:
    limit_low: u16 = 0
    base_low: u16 = 0
    base_mid: u8 = 0

```

```

    access: u8 = 0
    granularity: u8 = 0
    base_high: u8 = 0

/// GDT pointer structure (for LGDT instruction)
#[packed]
struct GDTPointer:
    limit: u16 = 0
    base: u64 = 0

// 64-bit GDT entries
static gdt: array<GDTEEntry, 5> = [
    GDTEEntry(0, 0, 0, 0, 0, 0),           // Null segment
    GDTEEntry(0, 0, 0, 0x9A, 0x20, 0),     // 64-bit code segment
    (executable, readable)
    GDTEEntry(0, 0, 0, 0x92, 0x20, 0),     // 64-bit data segment
    (writable)
    GDTEEntry(0, 0, 0, 0xFA, 0x20, 0),     // User code segment
    (ring 3)
    GDTEEntry(0, 0, 0, 0xF2, 0x20, 0),     // User data segment
    (ring 3)
]

static gdt_ptr: GDTPointer = GDTPointer(5 * 8 - 1, &gdt as u64)

/// Set up the Global Descriptor Table for 64-bit mode
fn setup_gdt():
    asm("lgdt [gdt_ptr]")

//
=====
// IDENTITY PAGING SETUP
//
=====

// Page table addresses (aligned to 4KB boundaries)
const PAGE_TABLE_PML4: u64 = 0x1000
const PAGE_TABLE_PDPT: u64 = 0x2000
const PAGE_TABLE_PD: u64 = 0x3000
const PAGE_TABLE_PT: u64 = 0x4000

// Page table flags
const PAGE_PRESENT: u64 = 1 << 0
const PAGE_WRITABLE: u64 = 1 << 1
const PAGE_USER: u64 = 1 << 2
const PAGE_HUGE_2MB: u64 = 1 << 7

/// Set up identity page tables (virtual addresses == physical addresses)
/// Maps the first 2GB of physical memory using 2MB huge pages

```

```

fn setup_identity_paging():
    // Clear page tables (zero them out)
    zero_memory(PAGE_TABLE_PML4, 0x1000 * 4)

    // Set up PML4 entry 0 -> PDPT
    let pml4_entry = PAGE_TABLE_PDPT | PAGE_PRESENT | PAGE_WRITABLE
    asm("mov [%0], %1" : : "r"(PAGE_TABLE_PML4), "r"(pml4_entry))

    // Set up PDPT entries (512 entries, each pointing to a PD)
    // For identity mapping, we map the first 512GB? Actually, we use 2MB
    huge pages
    // For 2MB huge pages, the PDPT points directly to a PD that contains
    2MB entries
    let pdpt_ptr = PAGE_TABLE_PDPT
    for i in 0..512:
        let pd_entry = (PAGE_TABLE_PD + (i * 8)) | PAGE_PRESENT |
PAGE_WRITABLE
        asm("mov [%0 + %1*8], %2" : : "r"(pdpt_ptr), "r"(i), "r"(pd_entry))

    // Set up page directory entries for 2MB huge pages
    // Each entry maps a 2MB region (512 entries cover the first 1GB)
    let pd_ptr = PAGE_TABLE_PD
    let mut phys_addr: u64 = 0
    for i in 0..512:
        let pd_entry = phys_addr | PAGE_PRESENT | PAGE_WRITABLE |
PAGE_HUGE_2MB
        asm("mov [%0 + %1*8], %2" : : "r"(pd_ptr), "r"(i), "r"(pd_entry))
        phys_addr = phys_addr + 0x200000 // 2MB per entry

    // Load CR3 with PML4 address
    write_cr3(PAGE_TABLE_PML4)

    //
    =====
    // VGA TEXT CONSOLE (for kernel output)
    //
    =====

const VGA_BASE: mmio_ptr<u16> = 0xB8000 as mmio_ptr<u16>
const VGA_WIDTH: u64 = 80
const VGA_HEIGHT: u64 = 25
const VGA_ATTRIBUTE: u16 = 0x0F00 // White on black

static cursor_x: u64 = 0
static cursor_y: u64 = 0

/// Print a single character to VGA text console
fn print_char(ch: u8):
    if ch == '\n' as u8:

```

```

        cursor_x = 0
        cursor_y = cursor_y + 1
    elif ch == '\r' as u8:
        cursor_x = 0
    elif ch == '\b' as u8 and cursor_x > 0:
        cursor_x = cursor_x - 1
        let pos = cursor_y * VGA_WIDTH + cursor_x
        VGA_BASE[pos] = VGA_ATTRIBUTE | (' ' as u16)
    else:
        let pos = cursor_y * VGA_WIDTH + cursor_x
        VGA_BASE[pos] = VGA_ATTRIBUTE | (ch as u16)
        cursor_x = cursor_x + 1

// Handle line wrapping
if cursor_x >= VGA_WIDTH:
    cursor_x = 0
    cursor_y = cursor_y + 1

// Handle scrolling
if cursor_y >= VGA_HEIGHT:
    scroll()
    cursor_y = VGA_HEIGHT - 1

/// Scroll the VGA screen up by one line
fn scroll():
    for row in 1..VGA_HEIGHT:
        for col in 0..VGA_WIDTH:
            let src = row * VGA_WIDTH + col
            let dst = (row - 1) * VGA_WIDTH + col
            VGA_BASE[dst] = VGA_BASE[src]

// Clear last line
let last_line = (VGA_HEIGHT - 1) * VGA_WIDTH
for col in 0..VGA_WIDTH:
    VGA_BASE[last_line + col] = VGA_ATTRIBUTE | (' ' as u16)

/// Print a null-terminated string
fn print_string(s: string):
    for ch in s:
        print_char(ch as u8)

/// Print a null-terminated string from pointer
fn print_string_ptr(s: ptr<u8>):
    let mut i: u64 = 0
    while s[i] != 0:
        print_char(s[i])
        i = i + 1

/// Print a 64-bit hexadecimal value

```

```

fn print_hex(value: u64):
    let hex_digits = "0123456789ABCDEF"
    for i in 60..0 step -4:
        let nibble = (value >> i) & 0xF
        print_char(hex_digits[nibble as u64] as u8)
    let last_nibble = value & 0xF
    print_char(hex_digits[last_nibble as u64] as u8)

/// Print a 64-bit decimal value
fn print_dec(value: u64):
    if value == 0:
        print_char('0')
        return

    let mut digits: array<u8, 20>
    let mut i: u64 = 0
    let mut temp = value

    while temp > 0:
        digits[i] = (temp % 10) as u8
        temp = temp / 10
        i = i + 1

    for j in i..0 step -1:
        print_char('0' + digits[j - 1])

//
=====
// MAIN KERNEL FUNCTION
//
=====

/// Main kernel function (runs after all initialization)
fn kmain(info: ptr<MultibootInfo>) -> u32:
    print_string("\r\n")
    print_string("=====\r\n")
    print_string("  lowl Kernel v2.1.0 - 64-bit Long Mode\r\n")
    print_string("  Multiboot-compliant\r\n")
    print_string("=====\r\n")
    print_string("\r\n")

    print_string("Kernel initialized successfully!\r\n")
    print_string("\r\n")

    print_string("System Information:\r\n")
    print_string("-----\r\n")

    // Print CPU features (using CPUID)
    let vendor = get_cpuid_vendor()

```

```

print_string("CPU Vendor: ")
print_string(vendor)
print_string("\r\n")

// Print memory statistics
print_string("Total RAM: ")
print_dec(total_ram / (1024 * 1024))
print_string(" MB\r\n")

print_string("Memory map entries: ")
print_dec(memory_map_entries)
print_string("\r\n")

// Print page table information
let cr3 = read_cr3()
print_string("CR3 (PML4 address): 0x")
print_hex(cr3)
print_string("\r\n")

print_string("\r\n")
print_string("Kernel ready. Entering idle loop...\r\n")

// Main kernel loop (idle)
while true:
    halt()

return 0

/// Get CPUID vendor string
fn get_cpuid_vendor() -> string:
    // Static buffer for vendor string (12 bytes + null)
    static vendor: array<u8, 13> = [0; 13]

    asm(
        mov eax, 0
        cpuid
        mov [%0], ebx
        mov [%0 + 4], edx
        mov [%0 + 8], ecx
        " : : \"r\"(&vendor as u64) : \"eax\", \"ebx\", \"ecx\", \"edx\"

    vendor[12] = 0
    return &vendor as string

//
=====
// LINKER SCRIPT (link.ld) - must be saved separately
//
=====

```

```
//
// The following linker script must be saved as 'link.ld':
//
// ENTRY(kernel_entry)
//
// SECTIONS
// {
//     . = 1M;
//
//     .multiboot : {
//         *(.multiboot)
//     }
//
//     .text : {
//         *(.text)
//         *(.text.*)
//     }
//
//     .rodata : {
//         *(.rodata)
//         *(.rodata.*)
//     }
//
//     .data : {
//         *(.data)
//         *(.data.*)
//     }
//
//     .bss : {
//         _bss_start = .;
//         *(.bss)
//         *(.bss.*)
//         *(COMMON)
//         _bss_end = .;
//     }
// }
```

Building the Multiboot Kernel:

bash

```
# Step 1: Compile lowl to NASM assembly
lowlc kernel.lowl -o kernel.asm -f elf -O2

# Step 2: Assemble to 64-bit object file
nasm -f elf64 kernel.asm -o kernel.o

# Step 3: Link with custom linker script
ld -T link.ld -o kernel.bin kernel.o
```

```
# Step 4: Verify Multiboot header (optional, using grub-file)
grub-file --is-x86-multiboot kernel.bin && echo "Valid Multiboot kernel"

# Step 5: Create bootable ISO with GRUB
mkdir -p iso/boot/grub
cp kernel.bin iso/boot/
cat > iso/boot/grub/grub.cfg << EOF
set timeout=5
set default=0

menuentry "lowl Kernel" {
    multiboot /boot/kernel.bin
    boot
}
EOF

# Step 6: Build ISO
grub-mkrescue -o lowl.iso iso/

# Step 7: Run in QEMU
qemu-system-x86_64 -cdrom lowl.iso -serial stdio

# Alternative: Direct kernel boot (if using GRUB directly)
qemu-system-x86_64 -kernel kernel.bin
```

Expected Output in QEMU with GRUB:

text

```
=====
lowl Kernel v2.1.0 - 64-bit Long Mode
Multiboot-compliant
=====

Kernel initialized successfully!

System Information:
-----
Boot loader: GRUB 2.06
Command line:
Lower memory: 640 KB
Upper memory: 1047552 KB
Memory map:
  Base Address          Length          Type
-----
0x0000000000000000 0x000000000009FC00 Available
0x000000000009FC00 0x0000000000000400 Reserved
0x00000000000E0000 0x0000000000020000 Reserved
```

```

0x000000000100000 0x000000003FEE0000 Available
0x000000003FEE0000 0x000000000020000 Reserved
Total usable RAM: 1024 MB
CPU Vendor: GenuineIntel
CR3 (PML4 address): 0x1000

Kernel ready. Entering idle loop...

```

Extending the Kernel:

To add interrupt handling, memory management, or device drivers to this Multiboot kernel, follow these guidelines:

1. **Interrupt Descriptor Table (IDT):** Set up the IDT after entering long mode, using the `#[interrupt]` attribute for handlers
2. **Physical Memory Manager:** Parse the Multiboot memory map entries and build a free list or bitmap
3. **Virtual Memory Manager:** Use the page table setup from this example as a foundation for allocating and mapping pages
4. **Userspace:** Set up user segments (ring 3) in the GDT and use `syscall` / `sysret` for system calls

This example provides a complete, working Multiboot kernel that can be compiled, linked, and booted with GRUB, serving as a foundation for more complex operating system development.

3.26 Loading Additional Modules

The bootloader can load additional modules (initial ramdisk, drivers, configuration files) and pass their locations to the kernel. For Multiboot, the kernel reads the modules tag from the Multiboot information structure. Each module has a start address, end address, name, and (with Multiboot2) command line. The kernel can load these modules as initial services, unpack a ramdisk, or execute them as early userspace. lowl's module system can load these modules at runtime, treating them as ELF or lowl module files.

3.27 The Boot Information Structure

The boot information structure passed from the bootloader contains critical data: memory map (usable RAM), framebuffer information (for graphical boot), boot device, command line, and module list. The kernel must parse this structure and use the information to initialize its subsystems. The structure format varies between bootloader types (Multiboot, Multiboot2, Linux Boot Protocol). lowl provides helper functions to parse common formats, but the kernel must be aware of which bootloader loaded it.

3.28 Error Handling During Boot

The boot process has limited error handling capabilities: no heap, no filesystem, no console (maybe just VGA text mode). Errors must be reported through the simplest means: the Bochs debug port (port 0xE9) for emulators, the VGA text buffer (0xB8000) for physical machines, or just a reboot. lowl's `port_write8` can output to the debug port, and `mmio_ptr<u16>` can write to VGA memory. The kernel should check return values from critical operations (page table allocation, memory detection) and halt with a descriptive error message if something fails.

3.29 Transition to the Final Kernel Image

After the initial boot code sets up paging and long mode, it jumps to the final kernel's entry point in the higher-half virtual address space. This final kernel entry point may be the same as the initial entry point (if the kernel is compiled for higher half) or a different symbol (if separate bootstrap and kernel). The final kernel must relocate any global variables that were initialized in the bootstrap phase? Typically, the bootstrap phase uses position-independent code, and the final kernel is linked for its final virtual address from the start.

3.30 Summary: From Power-on to lowl Kernel

The journey from processor reset to a running lowl kernel involves: BIOS/UEFI execution, bootloader loading, real-mode to protected-mode transition, enabling PAE and long mode, page table setup, switching to long mode, higher-half mapping, kernel entry, physical memory detection, virtual memory setup, interrupt initialization, and task scheduling. lowl supports each stage through its output formats, builtins, and attributes. Understanding this journey is essential for writing bootloaders and early kernel code. The remaining chapters focus on lowl's language features for kernel and application development once the system is running.

Chapter 4: Language Fundamentals

4.1 Program Structure

A lowl program consists of a sequence of declarations: modules (collections of related code), functions, classes, data sections, and global variables. The program may span multiple files, with the `import` statement incorporating code from other modules. Execution begins at the `main` function (for userspace programs) or at a kernel entry point marked `#[kernel]`. The compiler processes the entire program before generating output, enabling full cross-module optimization.

4.2 Indentation and Block Delimiters

Unlike C-style languages that use braces `{ }` for grouping, lowl uses indentation. A block is introduced by a colon `:` at the end of the previous line, followed by an indented block of statements. The block ends when the indentation level returns to the previous level. This eliminates brace matching errors and encourages consistent code formatting. The compiler tracks indentation levels explicitly, converting them into INDENT and DEDENT tokens during lexing.

4.3 Statements and Expressions

Every instruction in lowl is either a statement (which performs an action) or an expression (which produces a value). Statements include variable declarations, assignments, control flow constructs, and return statements. Expressions include literals, variable references, arithmetic operations, function calls, and type conversions. The distinction affects where a construct can appear: expressions can be part of larger expressions, while statements form the backbone of control flow.

4.4 Variables and Mutability

Variables in lowl are immutable by default (like in Rust or functional languages). The `let` keyword declares an immutable variable that cannot be reassigned. For mutability, you must explicitly use `let mut` (the full manual includes this distinction). The compiler enforces this: attempting to assign to an immutable variable is a compile-time error. This encourages a functional style and reduces bugs from accidental mutation. The `const` keyword declares compile-time constants, which are inlined wherever used.

4.5 Scope and Lifetime

Variables have block scope: they are visible from their declaration to the end of the enclosing block (which may be a function body, class body, or any indented block). Variables declared at module level have global scope and static duration (they exist for the entire program). Local variables have stack storage and are destroyed when the function returns. The compiler determines the exact stack offsets, ensuring that variables do not outlive their containing scope.

4.6 Shadowing

A variable declared in an inner scope can have the same name as a variable in an outer scope, shadowing the outer variable. The inner variable is a completely separate entity, and the outer variable becomes inaccessible in that scope. Shadowing is useful for temporary transformations (e.g., converting a string to integer and reusing the variable name) without needing separate names. Unlike mutations, shadowing changes the variable's type and value at a specific point in the code.

4.7 Functions and Parameters

Functions are declared with the `fn` keyword, optionally followed by `inline` for inlining. Parameters are listed in parentheses with their types (the type may be omitted if the compiler can infer it from context). The return type is specified after an arrow `->`. The function body is an indented block. Parameters are passed in registers (System V AMD64 ABI) and are copied into the function's stack frame if their addresses are taken or if there are many parameters.

4.8 Inline Functions

The `inline` keyword hints to the compiler that the function should be expanded at each call site rather than called normally. Inlined functions avoid call overhead and enable further optimizations because the caller's context is visible. However, inlining increases code size. The compiler may inline functions without the `inline` keyword at high optimization levels, and it may ignore the hint for very large functions. Inline functions are ideal for small, frequently used operations like accessors or simple arithmetic.

4.9 Comments and Documentation

Single-line comments begin with `//` and extend to the end of the line. Block comments begin with `/*` and end with `*/`, and they can span multiple lines. The compiler ignores comment content. Documentation comments (not yet standard) are planned with `///` or `/** */` syntax, allowing automatic documentation generation. Comments are especially important in systems code to explain hardware interactions and non-obvious optimizations.

4.10 Semicolons as Statement Terminators

Statements may end with a semicolon or a newline. The `implicit_semicolon` flag in the language configuration controls this behavior (default true). With implicit semicolons, the compiler treats a newline as a statement terminator except when the next line is clearly a continuation (e.g., after binary operators or commas). This makes lowl code cleaner but requires careful parsing. Explicit semicolons are always allowed and can be used to separate multiple statements on one line.

4.11 Expressions and Operator Precedence

Expressions combine values using operators. The operator precedence table determines which operations are performed first: unary and postfix operators (highest), then multiplication/division, then addition/subtraction, then shift, then comparison, then bitwise operations, then logical operations, then assignment (lowest). Parentheses can override precedence. All binary operators are left-associative except assignment, which is right-associative.

4.12 Short-Circuit Evaluation

The logical AND (`&&`) and OR (`||`) operators use short-circuit evaluation: for `&&`, the right operand is evaluated only if the left operand is true; for `||`, the right operand is evaluated only if the left operand is false. This matches C and most languages. Short-circuit evaluation is essential for safe pointer checks (`ptr != null && ptr.value > 0`) because it prevents dereferencing null pointers. The bitwise operators (`&`, `|`, `^`) always evaluate both arguments.

4.13 Integer Overflow Behavior

lowl defines integer overflow as wrapping (modulo 2^n) for unsigned types and as two's complement wrap for signed types (though signed overflow is technically undefined in the C standard, lowl defines it for consistency). The optimizer may assume that overflow does not happen when performing certain transforms; use `#[checked]` operations to trap overflows. Future versions may add `checked_add`, `saturating_add`, and `wrapping_add` methods for explicit control.

4.14 Floating-Point Semantics

Floating-point operations follow the IEEE 754 standard, with rounding mode set by the FPU control word (default round-to-nearest, ties to even). Operations like `+`, `-`, `*`, `/`, and `sqrt` map directly to hardware instructions. Comparisons produce a boolean result with the semantics defined by IEEE 754 (NaN is not equal to anything, including itself). The compiler is not allowed to reorder floating-point operations that would change the result (though it may use associative transforms when safe under the current rounding mode).

4.15 String Literals

String literals are delimited by double quotes `"..."` and support escape sequences: `\n` (newline), `\t` (tab), `\\` (backslash), `\"` (double quote). Multi-line strings are not yet supported but may be added. The type of a string literal is a pointer to a character array (null-terminated) stored in the `.rodata` section. The compiler automatically adds the null terminator. String concatenation (`"hello " + "world"`) is not currently supported but can be accomplished with array operations.

4.16 Character and Numeric Literals

Character literals are delimited by single quotes `'a'`. They have type `u8` (ASCII) and support the same escape sequences as strings. Numeric literals can be decimal (`42`), hexadecimal (`0x2A`), or binary (`0b101010`). Underscores can separate digits for readability (`1_000_000`). The compiler infers the smallest type that can hold the literal if the type is not specified; otherwise it uses the specified type, checking for overflow.

4.17 Null Pointers and Option Types

lowl uses the `null` keyword to represent a null pointer (value 0). Dereferencing null is undefined behavior, so lowl encourages using `Option<T>` instead of raw pointers for nullable values.

`Option<T>` is a tagger union that carries either a value (`Some`) or no value (`None`). The compiler can optimize `Option<ptr>` to use the null pointer as the `None` representation, achieving zero-cost abstraction. Pattern matching over `Option` ensures that the `None` case is handled.

4.18 Type Inference for Local Variables

The compiler infers the type of local variables from their initializers, making many type annotations unnecessary. For example, `let x = 42` infers `u64`. The inference works for complex expressions, including function calls and arithmetic. Type inference does not cross function boundaries (return type must be declared) or apply to global variables (which must have explicit types). This balances convenience with readability: local variable types are obvious from context, while function interfaces must be explicit.

4.19 Constant Expressions

Constants are declared with `const` and must be initialized with constant expressions (expressions that the compiler can evaluate at compile time). Constant expressions include literals, arithmetic on constants, and calls to certain builtin functions (`sizeof` , `alignof`). Constants are inlined at every use, so they have no address (you cannot take a pointer to a constant). This makes them ideal for mathematical constants, array sizes, and other compile-time values.

4.20 Static Variables

Static variables are declared with `static` and have global lifetime but local scope. They are stored in the `.data` or `.bss` section and retain their value across function calls. Static variables must have an explicit type and may have an initializer. In a multithreaded environment, static variables still need synchronization (unless they are read-only). The compiler does not automatically synchronize access, leaving that to explicit `with` statements or atomic operations.

4.21 The `with` Statement for Synchronization

The `with` statement provides automatic mutex locking and unlocking. It takes a pointer to a mutex (a `u64` variable that acts as a spinlock) and executes the indented block while holding the lock. The compiler emits a spinlock loop (`xchg` with memory barrier) at the beginning and an unlock at the end. The `with` statement ensures that the mutex is released even if the block executes a `return` , `break` , or `continue` . This provides RAI-style locking without explicit lock/unlock pairs.

4.22 Memory Barriers and Ordering

The builtin functions `mfence` , `lfence` , and `sfence` provide memory barrier operations.

`mfence` orders all memory accesses (both loads and stores) before and after the fence.

`lfence` orders loads (useful for timing attacks). `sfence` orders stores. These are needed when implementing lock-free data structures or communicating between cores without atomic operations. The `with` statement automatically includes appropriate fences (full `mfence`).

4.23 Compile-Time Assertions

The `static_assert` macro (or function, depending on implementation) checks a condition at compile time and produces an error if the condition is false. This is useful for verifying structure sizes, alignment requirements, and other invariants that must hold for the program to be correct. For example,

```
static_assert(sizeof(Device) == 64, "Device size must be 64 bytes for cache alignment")
```

. Compile-time assertions do not generate code.

4.24 Runtime Assertions for Debugging

The `assert` macro checks a condition at runtime and calls a panic handler (halt the system or print an error) if the condition is false. Assertions are typically enabled only in debug builds (controlled by the compiler's `-DNDEBUG` flag). In release builds, `assert` statements are removed. They are useful for catching logic errors, invalid pointer dereferences, and out-of-bounds accesses during development. The `debug_assert` variant is always enabled (for expensive checks).

4.25 Attributes and Pragmas

Attributes (starting with `#[` and ending with `]`) provide metadata to the compiler about functions, variables, or types. Examples: `#[kernel]` marks a kernel entry point, `#[interrupt]` marks an interrupt handler, `#[align(64)]` aligns a variable. Pragmas (starting with `#pragma`) provide optimization hints, like `#pragma optimize(03)` or `#pragma simd(AVX512)`. Attributes are more declarative, while pragmas affect compiler behavior temporarily.

4.26 Modules and Name Mangling

lowl supports modules to organize code and control visibility. A module is defined with the `module` keyword followed by a name and an indented block. Symbols within the module are private by default unless exported with `export`. The `import` statement makes symbols from another module available. The compiler performs name mangling for symbols (including modules, templates, and overloaded functions) to generate unique assembly labels. The mangling scheme is stable, enabling linking between separately compiled modules.

4.27 Visibility Control (Public/Private/Protected)

Members of a class can be marked `public`, `private`, or `protected`. The keyword introduces a section that applies to subsequent members until another visibility keyword appears. `public` members are accessible from any code. `private` members are accessible only within the same class. `protected` members are accessible within the class and its derived classes. Visibility is enforced by the compiler, not by the linker, so correct access control requires correct usage of the keywords.

4.28 The Build System Integration

The lowl compiler is a single-pass compiler (with separate optimization module). Integration with build systems (Make, CMake, Ninja) is straightforward: compile each `.lowl` file to an assembly file, then assemble and link. Dependencies between files are resolved through the `import` statement: the compiler needs to know the location of imported modules (either via search paths or explicit path). A future version may include a built-in module resolution system.

4.29 Debug Information Generation

The compiler can generate debug information (DWARF format) for use with debuggers like GDB. Debug information includes line numbers, variable locations, and type information. The `-g` flag (not yet implemented in the current version) enables debug generation. With debug information, you can set breakpoints, step through lowl source code, and inspect variables in the debugger. Without debug information, you can still debug at the assembly level.

4.30 Summary: Core Language Understanding

The fundamental constructs of lowl—variables, functions, control flow, types, and modules—provide a solid foundation for systems programming. The language design prioritizes explicitness and performance while providing modern syntax and type safety. With a strong grasp of these fundamentals, you can write correct, efficient lowl code. The following chapters build on this foundation, adding types, conversions, classes, and advanced data structures.

Chapter 5: Type System (Complete)

5.1 Fundamental Type Categories

lowl's type system is organized into six major categories that reflect different kinds of data and operations. Understanding these categories is essential for writing correct and efficient systems code.

Category 1: Primitive Numeric Types include signed and unsigned integers of various widths (8, 16, 32, 64, and 128 bits), as well as floating-point types (32-bit single precision and 64-bit double precision). These types map directly to hardware registers and instructions, providing zero-cost abstraction.

Category 2: SIMD Vector Types represent multiple values packed into a single register for parallel operations. These include `vec4_f32` (four single-precision floats in an SSE register), `vec8_f32` (eight floats in an AVX register), and `vec16_f32` (sixteen floats in an AVX-512 register).

Category 3: Pointer Types include generic pointers (`ptr<T>`), mutable pointers (`ptr_mut<T>`), and MMIO pointers (`mmio_ptr<T>`) for memory-mapped I/O with volatile semantics.

Category 4: Composite Types include arrays (fixed-size contiguous sequences), structures (named fields), and unions (overlapping fields).

Category 5: Container Types are provided by the standard library, including `block_array<T>` for dynamic arrays and `rb_map<K,V>` for associative containers.

Category 6: Special Types include `void` (for functions that return nothing), `bool` (for Boolean values), and `char` (for ASCII characters).

5.2 Integer Types: Signed and Unsigned

lowl provides both signed and unsigned integer types at widths of 8, 16, 32, 64, and 128 bits. The naming convention follows the pattern: `u8`, `u16`, `u32`, `u64`, `u128` for unsigned types, and `i8`, `i16`, `i32`, `i64`, `i128` for signed types. The choice between signed and unsigned affects how the compiler interprets bit patterns, how comparison operations behave, and how arithmetic overflow is handled.

lowl

```
// Declaring integer variables with explicit types
let a: u8 = 255;           // Maximum value for 8-bit unsigned
let b: i8 = -128;          // Minimum value for 8-bit signed
let c: u16 = 65535;        // Maximum for 16-bit unsigned
let d: i16 = -32768;       // Minimum for 16-bit signed
let e: u32 = 4294967295;    // Maximum for 32-bit unsigned
let f: i32 = -2147483648;   // Minimum for 32-bit signed
let g: u64 = 18446744073709551615; // Maximum for 64-bit unsigned
let h: i64 = -9223372036854775808; // Minimum for 64-bit signed

// Type inference works with integer literals
let x = 42;                // Compiler infers u64
let y = -100;              // Compiler infers i64 (negative literal)
let z = 0xFF;              // Hexadecimal literal (also u64)
let w = 0b101010;         // Binary literal (also u64)

// Type suffixes are available for explicit literals
let specific: u16 = 1000u16;
let signed_byte: i8 = 127i8;

// Arithmetic operations preserve type
let result1 = a + 1;       // u8 addition may overflow to 256 (wraps to 0)
let result2 = b - 1;       // i8 subtraction: -128 - 1 wraps to 127
let result3 = c * 2;       // u16 multiplication (65535 * 2 = 131070, wraps
to 65534)
```

5.3 Floating-Point Types

lowl supports IEEE 754 single-precision (f32) and double-precision (f64) floating-point types. These types follow standard floating-point semantics: division by zero produces infinity, invalid operations produce NaN (Not a Number), and rounding mode is configurable via the FPU control register. Floating-point operations are performed using x87 FPU or SSE/AVX instructions depending on the context and optimization level.

lowl

```
// Floating-point declarations
let pi: f32 = 3.14159265;   // Single precision (32 bits)
let e: f64 = 2.718281828459045; // Double precision (64 bits)

// Scientific notation
let avogadro: f64 = 6.02214076e23; // 6.02214076 × 1023
let planck: f64 = 6.62607015e-34; // 6.62607015 × 10-34

// Floating-point arithmetic
let sum = pi + 1.0;         // Promotes pi to f64 if needed
let product = e * 2.5;
let quotient = 10.0 / 3.0;  // Approximately 3.333333...
let remainder = 10.0 % 3.0; // 1.0 (modulo for floating-point)
```

```
// Special floating-point values
let infinity: f32 = 1.0 / 0.0;      // INFINITY
let neg_infinity: f32 = -1.0 / 0.0; // -INFINITY
let nan: f32 = 0.0 / 0.0;          // NaN (Not a Number)

// Checking for special values
let is_infinite = infinity == 1.0 / 0.0;
let is_nan = nan != nan;           // NaN is never equal to itself
```

5.4 SIMD Vector Types

SIMD (Single Instruction, Multiple Data) vector types allow processing multiple values simultaneously using a single CPU instruction. These types are aligned to their natural boundaries (16 bytes for SSE, 32 bytes for AVX, 64 bytes for AVX-512) for optimal performance. Vector operations are compiled directly to SIMD instructions without function call overhead.

lowl

```
// SSE vector types (128-bit registers)
let v4_f32: vec4_f32 = vec4_f32(1.0, 2.0, 3.0, 4.0);
let v2_f64: vec2_f64 = vec2_f64(1.5, 2.5);

// AVX vector types (256-bit registers)
let v8_f32: vec8_f32 = vec8_f32(1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0);
let v4_f64: vec4_f64 = vec4_f64(1.0, 2.0, 3.0, 4.0);

// AVX-512 vector types (512-bit registers)
let v16_f32: vec16_f32 = vec16_f32(
    1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0,
    9.0, 10.0, 11.0, 12.0, 13.0, 14.0, 15.0, 16.0
);
let v8_f64: vec8_f64 = vec8_f64(1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0);

// Mask types for AVX-512 predicated operations
let mask8: mask8 = 0b10101010;      // 8-bit mask for 8 elements
let mask16: mask16 = 0xAAAA;        // 16-bit mask for 16 elements

// Vector operations
let vsum = v8_f32 + v8_f32;          // Element-wise addition
let vdiff = v8_f32 - v8_f32;         // Element-wise subtraction
let vprod = v8_f32 * 2.0;            // Scalar multiplication (broadcast)
let vratio = v8_f32 / 4.0;           // Scalar division

// Horizontal operations (combine elements within a vector)
let max_val = v8_f32.hmax();         // Maximum value across all elements
let min_val = v8_f32.hmin();         // Minimum value across all elements
let sum_all = v8_f32.hadd();         // Sum of all elements
```

```
// Dot product (sum of element-wise products)
let dot_result = v8_f32.dot(&v8_f32);

// Permute and shuffle operations
let shuffled = v8_f32.shuffle(0b11010010); // Rearrange elements
```

5.5 Pointer Types

lowl provides three pointer types for different use cases: `ptr<T>` for immutable pointers to type `T` (the default, cannot modify the target), `ptr_mut<T>` for mutable pointers that can modify the target, and `mmio_ptr<T>` for memory-mapped I/O where reads and writes must not be optimized away or reordered. Pointer arithmetic is allowed and uses the size of the pointed-to type.

lowl

```
// Basic pointer declarations
let x: u64 = 42;
let p: ptr<u64> = &x;           // Immutable pointer to x
let q: ptr_mut<u64> = &mut x;   // Mutable pointer to x

// Dereferencing pointers
let value = *p;                 // Reads 42 through immutable pointer
*q = 100;                       // Writes 100 through mutable pointer

// Pointer arithmetic (advances by sizeof(T) bytes)
let arr: array<u64, 10> = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
let first = &arr[0] as ptr<u64>;
let second = first + 1;         // Points to arr[1], 8 bytes forward
let fifth = second + 3;         // Points to arr[4]
let third = fifth - 2;          // Points back to arr[2]

// Null pointers
let null_ptr: ptr<u8> = null;   // Points to address 0
let null_mut: ptr_mut<u64> = null;

// MMIO pointers (volatile, prevents compiler optimizations)
let vga_buffer: mmio_ptr<u16> = 0xB8000 as mmio_ptr<u16>;
vga_buffer[0] = (0x0F << 8) | 'H'; // Write to VGA memory
let vga_char = vga_buffer[0];      // Read from VGA memory

// Converting between pointer types
let generic_ptr: ptr<u32> = &value as ptr<u32>;
let void_ptr: ptr<void> = generic_ptr as ptr<void>;
let restored_ptr: ptr<u32> = void_ptr as ptr<u32>;

// Unsafe pointer casting (use with caution)
let address: u64 = 0x100000;
let raw_ptr = address as ptr<u8>;
```

5.6 Composite Types: Arrays

Arrays in lowl are fixed-size, contiguous sequences of elements of the same type. The size must be known at compile time. Arrays are zero-indexed and bounds checking is optional (can be enabled with debug flags). Arrays are stored inline (not as separate heap allocations) and are stack-allocated by default.

lowl

```
// Array declarations
let numbers: array<u64, 5> = [10, 20, 30, 40, 50];
let zeros: array<u8, 256> = [0; 256];           // All zeros
let pattern: array<u32, 8> = [1, 2, 3, 4, 5, 6, 7, 8];

// Multi-dimensional arrays (arrays of arrays)
let matrix: array<array<f64, 4>, 4> = [
    [1.0, 2.0, 3.0, 4.0],
    [5.0, 6.0, 7.0, 8.0],
    [9.0, 10.0, 11.0, 12.0],
    [13.0, 14.0, 15.0, 16.0]
];

// Accessing array elements
let first = numbers[0];           // 10
let third = numbers[2];          // 30
let last = numbers[4];           // 50

// Modifying array elements
numbers[1] = 25;                 // Now [10, 25, 30, 40, 50]
matrix[1][2] = 99.0;             // Modifies second row, third column

// Array iteration
for i in 0..numbers.len():
    numbers[i] = numbers[i] * 2;

// Array slices (view without copying)
let slice = numbers[1..4];       // Contains [25, 30, 40]
let full_slice = numbers[..];   // Entire array
let from_start = numbers[..3];  // First three elements
let to_end = numbers[2..];      // Elements from index 2 to end

// Getting array size at compile time
const SIZE: u64 = sizeof(array<u64, 10>); // 80 bytes
const ALIGN: u64 = alignof(array<u64, 10>); // 8 bytes (for u64 alignment)
```

5.7 Composite Types: Structures

Structures (structs) group multiple named fields of possibly different types into a single composite type. Structures can be defined at module level and can contain methods using the `impl` block.

Structure fields can be accessed using dot notation. The memory layout of a structure follows the C ABI by default, but can be controlled with attributes.

lowl

```
// Basic structure definition
struct Point:
    x: f64
    y: f64

// Structure with methods
struct Rectangle:
    width: f64
    height: f64

impl Rectangle:
    fn area() -> f64:
        return this.width * this.height

    fn perimeter() -> f64:
        return 2.0 * (this.width + this.height)

    fn scale(factor: f64):
        this.width = this.width * factor
        this.height = this.height * factor

// Structure with different visibility
struct DeviceConfig:
    public:
        name: string
        enabled: bool

    private:
        internal_id: u32
        registers: mmio_ptr<u32>

// Packed structure (no padding between fields)
#[packed]
struct NetworkHeader:
    dest_mac: array<u8, 6>
    src_mac: array<u8, 6>
    ethertype: u16
    payload: ptr<u8>

// Aligned structure
#[align(64)] // Cache-line aligned
struct CacheLineData:
    value: u64
    padding: array<u8, 56> // Explicit padding to 64 bytes
```

```
// Using structures
let origin = Point{x: 0.0, y: 0.0};
let rect = Rectangle{width: 10.0, height: 20.0};

let area = rect.area();           // 200.0
let perimeter = rect.perimeter(); // 60.0
rect.scale(2.0);                  // Now width=20, height=40

// Accessing fields
let x_coord = origin.x;
let y_coord = origin.y;

// Structure assignment (copy)
let mut rect2 = rect;             // Copies all fields
rect2.width = 15.0;               // Does not affect original rect
```

5.8 Boolean and Character Types

The `bool` type represents Boolean values (true or false) and occupies 1 byte. The `char` type represents a single ASCII character and also occupies 1 byte. Comparison operators produce Boolean results, and Boolean values can be used directly in conditional statements.

lowl

```
// Boolean declarations
let flag1: bool = true;
let flag2: bool = false;
let is_positive: bool = 42 > 0; // true
let is_equal: bool = 10 == 20;  // false

// Boolean operations
let and_result = true && false; // false
let or_result = true || false;  // true
let not_result = !true;         // false

// Using bool in conditionals
if is_positive:
    print_string("Forty-two is positive\n")

// Character declarations
let newline: char = '\n';
let tab: char = '\t';
let null_char: char = '\0';
let letter_a: char = 'A';
let digit_9: char = '9';
let hex_space: char = ' ';

// Character operations
```

```

let is_letter = (letter_a >= 'A') && (letter_a <= 'Z');
let uppercase = letter_a - 32;           // 65 - 32 = 33, which is 'A'? Actually
'A'+32 = 'a'

// Character arrays (strings)
let hello: array<char, 6> = ['H', 'e', 'l', 'l', 'o', '\0'];
let world: array<u8, 6> = [87, 111, 114, 108, 100, 0]; // ASCII values

```

5.9 Void Type and Type Inference

The `void` type represents the absence of a value. It is used as the return type for functions that perform actions but produce no value. Type inference (`let x = expression`) allows the compiler to deduce the type of a variable from its initializer, reducing verbosity while maintaining type safety.

lowI

```

// Void type (used for functions with no return value)
fn print_message(msg: string) -> void:
    for ch in msg:
        print_char(ch)

// Functions that implicitly return void (no -> void needed)
fn increment_counter():
    static count: u64 = 0
    count = count + 1

// Type inference examples
let a = 42;           // Inferred as u64
let b = 3.14159;      // Inferred as f64
let c = true;         // Inferred as bool
let d = 'X';          // Inferred as char
let e = &a;           // Inferred as ptr<u64>
let f = [1, 2, 3];    // Inferred as array<u64, 3>

// Type inference with complex expressions
let sum = 10 + 20 + 30; // u64
let product = 2.5 * 4.0; // f64
let is_valid = (sum > 50) && (product < 20.0); // bool

// Type inference cannot cross function boundaries
fn get_value() -> u64:
    return 42

let g = get_value(); // Type inferred as u64 (from return type)
// let h = get_value(); would also be u64

// Explicit type annotations override inference
let explicit_i32: i32 = 42; // i32, not u64

```

```
let explicit_f32: f32 = 3.14;    // f32, not f64
let explicit_ptr: ptr<u8> = null; // ptr<u8>
```

5.10 Type Aliases and Conversion

Type aliases allow creating alternative names for existing types, improving code clarity without creating new types. Type conversion between compatible types is explicit using the colon operator with optional rounding control.

lowI

```
// Type aliases
type byte = u8
type word = u16
type dword = u32
type qword = u64
type seconds = u64
type milliseconds = u64

// Using aliases
let b: byte = 0xFF;
let w: word = 0xFFFF;
let timeout: seconds = 30;
let delay: milliseconds = 500;

// Type conversion (lossless)
let x: u32 = 1000
let y: u64 = x:u64           // 32-bit to 64-bit (zero extension)
let z: u16 = y:u16           // 64-bit to 16-bit (truncation warning)

// Floating-point to integer conversion with rounding
let pi: f64 = 3.14159
let rounded: u32 = pi:u32.round()    // 3
let floored: u32 = pi:u32.floor()    // 3
let ceiled: u32 = pi:u32.ceil()      // 4
let truncated: u32 = pi:u32.trunc()  // 3

// Integer to floating-point
let int_val: u64 = 42
let float_val: f64 = int_val:f64     // 42.0

// Saturating conversion (clamps to min/max)
let big: u64 = 300
let small: u8 = big:u8.saturating()  // 255 (max u8)

// Wrapping conversion (modulo arithmetic)
let wrapped: u8 = big:u8.wrapping()  // 300 % 256 = 44

// Checked conversion returns Option
```

```
let result = big:u8.checked()
match result:
  case Some(val):
    print_string("Conversion successful: ")
    print_dec(val)
  case None:
    print_string("Value out of range")
```

5.11 sizeof and alignof Operations

The `sizeof` operator returns the size in bytes of a type or expression. The `alignof` operator returns the alignment requirement (minimum memory address alignment) for a type. These operations are evaluated at compile time and are useful for low-level memory manipulation and structure layout verification.

lowl

```
// Getting sizes of primitive types
let u8_size = sizeof(u8);      // 1
let u16_size = sizeof(u16);    // 2
let u32_size = sizeof(u32);    // 4
let u64_size = sizeof(u64);    // 8
let f32_size = sizeof(f32);    // 4
let f64_size = sizeof(f64);    // 8

// Getting sizes of composite types
struct PackedData:
  a: u8
  b: u32
  c: u16

#[packed]
struct PackedDataPacked:
  a: u8
  b: u32
  c: u16

let normal_size = sizeof(PackedData);    // Likely 12 (with padding)
let packed_size = sizeof(PackedDataPacked); // 1+4+2 = 7

// Alignment requirements
let u8_align = alignof(u8);      // 1
let u32_align = alignof(u32);    // 4
let u64_align = alignof(u64);    // 8
let vec8_align = alignof(vec8_f32); // 32 (AVX alignment)

// Using sizeof for memory operations
let buffer_size = 1024
let buffer = physical_alloc(buffer_size, 64)
```

```
zero_memory(buffer, buffer_size)

// Compile-time assertions using sizeof
static_assert(sizeof(NetworkHeader) == 14, "Ethernet header must be 14
bytes")
static_assert(alignof(CacheLineData) == 64, "CacheLineData must be 64-byte
aligned")
```

5.12 Complete Chapter Example: Geometry Library

This example demonstrates the lowl type system by implementing a geometry library that uses multiple types, structures, SIMD vectors, and proper type conversions.

lowl

```
// geometry.lowl - Complete geometry library demonstrating lowl type system
// Compile: lowlc geometry.lowl -o geometry.asm -O2

//
=====
// TYPE ALIASES FOR CLARITY
//
=====

type radians = f64
type degrees = f64
type meters = f64
type seconds = u64

//
=====
// CORE GEOMETRY TYPES
//
=====

#[align(16)] // SSE alignment for performance
struct Vec3:
    x: f64
    y: f64
    z: f64

impl Vec3:
    fn new(x: f64, y: f64, z: f64) -> Vec3:
        return Vec3{x, y, z}

    fn length() -> f64:
        return sqrt(this.x*this.x + this.y*this.y + this.z*this.z)

    fn normalize() -> Vec3:
```

```

    let len = this.length()
    if len > 0.0:
        return Vec3{
            x: this.x / len,
            y: this.y / len,
            z: this.z / len
        }
    return Vec3{0.0, 0.0, 0.0}

fn dot(other: &Vec3) -> f64:
    return this.x * other.x + this.y * other.y + this.z * other.z

fn cross(other: &Vec3) -> Vec3:
    return Vec3{
        x: this.y * other.z - this.z * other.y,
        y: this.z * other.x - this.x * other.z,
        z: this.x * other.y - this.y * other.x
    }

#[align(32)] // AVX alignment
struct Matrix4:
    data: array<array<f64, 4>, 4>

impl Matrix4:
    fn identity() -> Matrix4:
        let mut m = Matrix4{data: [[0.0; 4]; 4]}
        m.data[0][0] = 1.0
        m.data[1][1] = 1.0
        m.data[2][2] = 1.0
        m.data[3][3] = 1.0
        return m

    fn multiply(&self, other: &Matrix4) -> Matrix4:
        let mut result = Matrix4{data: [[0.0; 4]; 4]}
        for i in 0..4:
            for j in 0..4:
                let mut sum = 0.0
                for k in 0..4:
                    sum = sum + self.data[i][k] * other.data[k][j]
                result.data[i][j] = sum
        return result

struct Transform:
    position: Vec3
    rotation: Vec3 // Euler angles in radians
    scale: Vec3

impl Transform:
    fn new() -> Transform:

```

```

    return Transform{
        position: Vec3{0.0, 0.0, 0.0},
        rotation: Vec3{0.0, 0.0, 0.0},
        scale: Vec3{1.0, 1.0, 1.0}
    }

fn to_matrix(&self) -> Matrix4:
    let mut matrix = Matrix4.identity()

    // Scale matrix
    matrix.data[0][0] = self.scale.x
    matrix.data[1][1] = self.scale.y
    matrix.data[2][2] = self.scale.z

    // Rotation matrices (simplified - only Z rotation for this example)
    let cz = cos(self.rotation.z)
    let sz = sin(self.rotation.z)

    matrix.data[0][0] = matrix.data[0][0] * cz
    matrix.data[0][1] = matrix.data[0][1] * -sz
    matrix.data[1][0] = matrix.data[1][0] * sz
    matrix.data[1][1] = matrix.data[1][1] * cz

    // Translation
    matrix.data[3][0] = self.position.x
    matrix.data[3][1] = self.position.y
    matrix.data[3][2] = self.position.z

    return matrix

//
=====
// SIMD-ACCELERATED BOUNDING BOX
//
=====

#[align(64)] // AVX-512 cache line alignment
struct BoundingBox:
    min: vec4_f32    // x, y, z, 0 (padding)
    max: vec4_f32    // x, y, z, 0

impl BoundingBox:
    fn new(min_x: f32, min_y: f32, min_z: f32, max_x: f32, max_y: f32,
max_z: f32) -> BoundingBox:
        return BoundingBox{
            min: vec4_f32(min_x, min_y, min_z, 0.0),
            max: vec4_f32(max_x, max_y, max_z, 0.0)
        }

```

```

fn contains_point(&self, x: f32, y: f32, z: f32) -> bool:
    let point = vec4_f32(x, y, z, 0.0)
    let min_comp = point.cmpeq(self.min)
    let max_comp = point.cmpeq(self.max)
    // Simplified containment check (would use vector comparisons in
real code)
    return x >= self.min.x and x <= self.max.x and
           y >= self.min.y and y <= self.max.y and
           z >= self.min.z and z <= self.max.z

fn intersects(&self, other: &BoundingBox) -> bool:
    return (self.min.x <= other.max.x and self.max.x >= other.min.x) and
           (self.min.y <= other.max.y and self.max.y >= other.min.y) and
           (self.min.z <= other.max.z and self.max.z >= other.min.z)

//
=====
// COLOR TYPES (example of packed structures)
//
=====

#[packed]
struct ColorRGB:
    r: u8
    g: u8
    b: u8

#[packed]
struct ColorRGBA:
    r: u8
    g: u8
    b: u8
    a: u8

impl ColorRGBA:
    fn to_u32() -> u32:
        return ((this.r as u32) << 24) |
               ((this.g as u32) << 16) |
               ((this.b as u32) << 8) |
               (this.a as u32)

//
=====
// DEMONSTRATION FUNCTION
//
=====

fn main() -> u32:
    // Demonstrate type inference

```

```
let x = 42                // u64
let y = 3.14159           // f64
let name = "lowl"         // string

// Demonstrate structure usage
let v1 = Vec3.new(1.0, 2.0, 3.0)
let v2 = Vec3.new(4.0, 5.0, 6.0)

let dot_product = v1.dot(&v2)
let cross_product = v1.cross(&v2)

print_string("Vector 1: (")
print_f64(v1.x)
print_string(", ")
print_f64(v1.y)
print_string(", ")
print_f64(v1.z)
print_string(")\n")

print_string("Dot product: ")
print_f64(dot_product)
print_string("\n")

// Demonstrate type conversion with rounding
let pi: f64 = 3.141592653589793
let as_int: u64 = pi:u64.round()
let floored: u64 = pi:u64.floor()

print_string("Pi as integer (rounded): ")
print_dec(as_int)
print_string("\n")

// Demonstrate SIMD types
let simd_vec = vec8_f32(1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0)
let simd_sum = simd_vec + simd_vec
let simd_max = simd_vec.hmax()

print_string("SIMD max value: ")
print_f64(simd_max as f64)
print_string("\n")

// Demonstrate bounding boxes
let box1 = BoundingBox.new(0.0, 0.0, 0.0, 10.0, 10.0, 10.0)
let box2 = BoundingBox.new(5.0, 5.0, 5.0, 15.0, 15.0, 15.0)

if box1.intersects(&box2):
    print_string("Bounding boxes intersect\n")

// Demonstrate packed structures and type aliases
```

```

let seconds_elapsed: seconds = 3600    // 1 hour using type alias
let color = ColorRGBA{r: 255, g: 0, b: 0, a: 255}
let color_u32 = color.to_u32()

print_string("Color as u32: 0x")
print_hex(color_u32 as u64)
print_string("\n")

print_string("Seconds elapsed: ")
print_dec(seconds_elapsed)
print_string("\n")

// Demonstrate sizeof and alignof
print_string("Size of Vec3: ")
print_dec(sizeof(Vec3))
print_string(" bytes\n")
print_string("Alignment of Vec3: ")
print_dec(alignof(Vec3))
print_string(" bytes\n")

return 0

//
=====
// HELPER PRINT FUNCTIONS
//
=====

static vga_ptr: mmio_ptr<u16> = 0xB8000 as mmio_ptr<u16>
static cursor: u64 = 0

fn print_char(ch: u8):
    let color = (0x0F << 8) as u16
    vga_ptr[cursor] = color | (ch as u16)
    cursor = cursor + 1
    if cursor >= 80 * 25:
        cursor = 0

fn print_string(s: string):
    for ch in s:
        print_char(ch as u8)

fn print_dec(value: u64):
    if value == 0:
        print_char('0')
        return
    let mut digits: array<u8, 20>
    let mut i: u64 = 0
    let mut temp = value

```

```

while temp > 0:
    digits[i] = (temp % 10) as u8
    temp = temp / 10
    i = i + 1
for j in i..0 step -1:
    print_char('0' + digits[j - 1])

fn print_hex(value: u64):
    let hex_digits = "0123456789ABCDEF"
    for i in 60..0 step -4:
        let nibble = (value >> i) & 0xF
        print_char(hex_digits[nibble as u64] as u8)
    let last = value & 0xF
    print_char(hex_digits[last as u64] as u8)

fn print_f64(value: f64):
    let int_part = value as u64
    print_dec(int_part)
    print_char('.')
    let frac = (value - (int_part as f64)) * 1000.0
    let frac_int = frac as u64
    print_dec(frac_int)

```

Expected Output:

text

```

Vector 1: (1.000, 2.000, 3.000)
Dot product: 32.000
Pi as integer (rounded): 3
SIMD max value: 8.000
Bounding boxes intersect
Color as u32: 0xFF0000FF
Seconds elapsed: 3600
Size of Vec3: 24 bytes
Alignment of Vec3: 16 bytes

```

This concludes **Chapter 5: Type System**. The chapter covered all fundamental lowl types (integers, floats, SIMD vectors, pointers, arrays, structures, booleans, characters, void), type inference, type aliases, conversion with rounding control, and compile-time size/alignment operations. The complete geometry library example demonstrates the practical application of these types in a cohesive systems programming context.

Volume II: Core Language Features (Chapters 6-10)

Chapter 6: Type Conversion with Rounding Control

6.1 Evolution from Traditional Casting

Traditional systems languages use cast operators like `(type)value` or `as` for type conversion. lowl replaces these with a more expressive OOP-style conversion syntax using the colon `:` operator followed by the target type and an optional rounding method. This design makes conversion behavior explicit and eliminates ambiguity about rounding, overflow, and precision loss.

6.2 The Colon Conversion Operator Syntax

The basic syntax for type conversion in lowl is `value:TargetType.method()`, where `value` is the source expression, `TargetType` is the destination type, and `method` specifies how the conversion should handle rounding, overflow, or precision issues. For lossless conversions (e.g., u32 to u64), no method is required.

lowl

```
// Lossless conversions (no method needed)
let x: u32 = 42
let y: u64 = x:u64           // Zero extension
let z: i64 = x:i64           // Zero extension to signed

// Explicit conversions require methods
let pi: f64 = 3.14159
let rounded: u32 = pi:u32.round()
let floored: u32 = pi:u32.floor()
let ceiled: u32 = pi:u32.ceil()
let truncated: u32 = pi:u32.trunc()
```

6.3 Floating-Point to Integer Rounding Methods

When converting from floating-point to integer, rounding behavior must be specified because floating-point values rarely map exactly to integers. lowl provides five rounding methods:

`round()` (nearest, ties to even, the default banker's rounding), `floor()` (toward negative infinity), `ceil()` (toward positive infinity), `trunc()` (toward zero), and explicit tie-breaking methods for precise control.

lowl

```
// Rounding method examples
let values: array<f64, 6> = [2.3, 2.5, 2.7, -2.3, -2.5, -2.7]

for v in values:
  let r = v:u32.round()      // 2, 2, 3, -2, -2, -3 (banker's rounding)
  let f = v:u32.floor()      // 2, 2, 2, -3, -3, -3
  let c = v:u32.ceil()       // 3, 3, 3, -2, -2, -2
  let t = v:u32.trunc()      // 2, 2, 2, -2, -2, -2

// Explicit tie-breaking (important for financial calculations)
```

```
let half: f64 = 2.5
let tie_even = half:u32.nearest_tie_even() // 2 (banker's rounding)
let tie_away = half:u32.nearest_tie_away() // 3 (round half up)
let tie_up = half:u32.nearest_tie_up() // 3 (round half up)
let tie_down = half:u32.nearest_tie_down() // 2 (round half down)
```

6.4 Integer to Integer Overflow Handling

Converting between integer types when the source value may not fit in the destination type requires explicit overflow handling. `lowl` provides three overflow handling methods:

`saturating()` (clamps to the destination type's minimum or maximum), `wrapping()` (modulo arithmetic, discarding overflow bits), and `checked()` (returns `Option<T>`, indicating success or failure).

`lowl`

```
// Saturating conversion (clamps to min/max)
let large: u64 = 500
let small: u8 = large:u8.saturating() // 255 (maximum u8)
let negative: i64 = -100
let unsigned: u8 = negative:u8.saturating() // 0 (minimum u8)

// Wrapping conversion (modulo arithmetic)
let wrapped: u8 = large:u8.wrapping() // 500 % 256 = 244
let wrapped_signed: i8 = negative:i8.wrapping() // -100 modulo 256 = 156

// Checked conversion (returns Option)
let opt = large:u8.checked()
match opt:
  case Some(val):
    print_string("Converted successfully: ")
    print_dec(val)
  case None:
    print_string("Value out of range for u8")

// Practical example: parsing network packet fields
fn parse_u16_from_bytes(high: u8, low: u8) -> u16:
  return ((high as u16) << 8) | (low as u16)

fn parse_with_safety(value: u32) -> u8:
  // Silently saturate if out of range
  return value:u8.saturating()
```

6.5 Integer to Floating-Point Conversions

Converting integers to floating-point numbers is generally lossless for values within the mantissa precision range. For 32-bit floats, integers up to 16,777,216 are exactly representable; beyond that, rounding occurs implicitly. `lowl` allows explicit rounding method specification for cases where precision control is needed.

lowl

```
// Simple integer to float conversion
let int_val: u64 = 42
let float_val: f64 = int_val:f64           // 42.0 exactly

// Large integers may lose precision in f32
let large_int: u32 = 16_777_217
let single_precision: f32 = large_int:f32   // May round to
16,777,216.0
let double_precision: f64 = large_int:f64    // Exact representation

// Explicit rounding for integer to float
let exact = large_int:f32.round()           // Round to nearest representable
value
let down = large_int:f32.floor()            // Round down
let up = large_int:f32.ceil()               // Round up
```

6.6 Pointer and Integer Conversions

Converting between pointers and integers is common in systems programming for address arithmetic, page table manipulation, and MMIO access. lowl requires explicit conversion for pointer↔integer operations to prevent accidental misuse.

lowl

```
// Pointer to integer conversion
let ptr: ptr<u8> = 0x1000 as ptr<u8>
let addr: u64 = ptr:u64                     // Get the raw address

// Integer to pointer conversion
let address: u64 = 0xB8000
let vga_ptr: mmio_ptr<u16> = address:mmio_ptr<u16>

// Page table manipulation
let page_table_entry: u64 = 0x1000 | (1 << 0) | (1 << 1) // Present +
Writable
let physical_addr = page_table_entry & ~0xFFF
let flags = page_table_entry & 0xFFF
let reconstructed_ptr = physical_addr:ptr<u64>

// Null pointer representation
let null_ptr: ptr<u8> = 0:ptr<u8>
let null_addr = null_ptr:u64                // 0
```

6.7 Unsafe Unchecked Conversions

For performance-critical code where the programmer guarantees that conversion is safe, lowl provides an `unchecked()` method that omits all checks and overflow handling. This should be used sparingly and only when the compiler's static analysis cannot prove safety.

lowl

```
// Unsafe unchecked conversion (no bounds checking)
#[unsafe]
fn fast_conversion(value: u64) -> u8:
    // Programmer guarantees value is within 0..255
    return value:u8.unchecked()

// Performance-critical packet parsing
#[unsafe]
fn parse_ethernet_header(ptr: ptr<u8>) -> EthernetHeader:
    // Guarantee that pointer is properly aligned and within bounds
    return (ptr:ptr<EthernetHeader>).unchecked()

// WARNING: Use unchecked() only when absolutely certain
let x: u64 = 42
let y: u8 = x:u8.unchecked()    // Safe, 42 fits in u8
let z: u64 = 300
let w: u8 = z:u8.unchecked()    // UNSAFE: 300 % 256 = 44, silent
truncation
```

6.8 User-Defined Conversions

Types can define their own conversion methods by implementing methods with the special colon-prefixed naming convention. This allows custom types to integrate seamlessly with lowl's conversion syntax.

lowl

```
// Custom rational number type with conversions
struct Rational:
    numerator: i64
    denominator: u64

impl Rational:
    fn new(num: i64, den: u64) -> Rational:
        return Rational{num, den}

    // Conversion to f64 (uses colon syntax)
    fn :f64() -> f64:
        return (this.numerator as f64) / (this.denominator as f64)

    // Conversion to f64 with rounding
    fn :f64.round() -> f64:
        let exact = (this.numerator as f64) / (this.denominator as f64)
        // Apply rounding (simplified)
        return exact

    // Conversion to u64 with rounding mode
```

```
fn :u64.round() -> u64:
  let float_val = this:f64
  return float_val:u64.round()

// Using user-defined conversions
let half = Rational.new(1, 2)
let third = Rational.new(1, 3)

let half_float: f64 = half:f64           // 0.5
let third_float: f64 = third:f64         // 0.333333...
let third_int: u64 = third:u64.round()    // 0

// Chained conversions
let rational = Rational.new(22, 7)
let int_result: u32 = rational:f64:u32.round() // π approximated as 3
```

6.9 Conversion Method Reference Table

Source Type	Target Type	Methods	Description
f32/f64	u8/i8/u16/i16/u32/i32/u64/i64	round() , floor() , ceil() , trunc() , nearest_tie_even() , nearest_tie_away() , saturating() , wrapping() , checked()	Converts float to integer with specified rounding/overflow
u8/i8/u16/i16/u32/i32/u64/i64	f32/f64	round() , floor() , ceil() (for narrowing)	Converts integer to float (optional rounding for precision loss)
u8/i8/u16/i16/u32/i32	u64/i64	zero_extend() , sign_extend()	Widening conversions (zero or sign extension)

u64/i64	u8/i8/u16/i16/u32/i32	<code>saturating()</code> , <code>wrapping()</code> , <code>checked()</code> , <code>unchecked()</code>	Narrowing conversions with explicit overflow handling
<code>ptr<T></code>	u64	none	Pointer to integer (address)
u64	<code>ptr<T></code>	none	Integer to pointer (address)
Any numeric	Any numeric	<code>exact()</code>	Conversion that fails compilation if loss of precision occurs

6.10 Compiler Warnings and Errors for Imprecise Conversions

The lowl compiler emits warnings or errors for implicit conversions that may lose precision or cause overflow. This helps catch bugs at compile time without requiring runtime checks.

lowl

```
// Compiler warning: possible precision loss
let large: u64 = 1_000_000_000_000
let small: u32 = large           // COMPILER WARNING: value may not fit in
u32
// Fix: explicit conversion
let explicit: u32 = large.u32.saturating()

// Compiler warning: floating-point truncation
let pi: f64 = 3.14159
let integer: u64 = pi           // COMPILER WARNING: implicit truncation of
float
// Fix: explicit rounding
let rounded: u64 = pi.u64.round()

// Compile-time error: conversion impossible
// let invalid: string = 42 as string // ERROR: no conversion from integer
to string
```

```
// Suppress warnings with explicit conversion
let suppressed: u32 = large:u32.wrapping() // OK, explicit intention
```

6.11 Complete Chapter Example: Data Format Converter

This example demonstrates type conversions in a practical data format conversion utility that reads binary data, converts between endianness, and transforms between different numeric representations.

lowl

```
// converter.lowl - Binary data format converter with type conversions
// Compile: lowlc converter.lowl -o converter.asm -O2

//
=====
// ENDIANNESS CONVERSION HELPERS
//
=====

fn swap_u16(value: u16) -> u16:
    return ((value >> 8) & 0xFF) | ((value & 0xFF) << 8)

fn swap_u32(value: u32) -> u32:
    return ((value >> 24) & 0xFF) |
        ((value >> 8) & 0xFF00) |
        ((value & 0xFF00) << 8) |
        ((value & 0xFF) << 24)

fn swap_u64(value: u64) -> u64:
    return ((value >> 56) & 0xFF) |
        ((value >> 40) & 0xFF00) |
        ((value >> 24) & 0xFF0000) |
        ((value >> 8) & 0xFF000000) |
        ((value & 0xFF000000) << 8) |
        ((value & 0xFF0000) << 24) |
        ((value & 0xFF00) << 40) |
        ((value & 0xFF) << 56)

//
=====
// DATA FORMAT CONVERTER
//
=====

struct DataConverter:
    input_format: string
    output_format: string
    precision: u8 // 32 or 64 for floating-point
```

```

impl DataConverter:
  fn new(in_fmt: string, out_fmt: string, precision: u8) -> DataConverter:
    return DataConverter{in_fmt, out_fmt, precision}

  // Convert integer with saturation
  fn convert_int_saturating(&self, value: u64, target_bits: u8) -> u64:
    match target_bits:
      case 8:
        let result = value:u8.saturating()
        return result:u64
      case 16:
        let result = value:u16.saturating()
        return result:u64
      case 32:
        let result = value:u32.saturating()
        return result:u64
      case 64:
        return value
      default:
        return 0

  // Convert float with rounding mode
  fn convert_float_rounding(&self, value: f64, rounding_mode: string) ->
u64:
    match rounding_mode:
      case "round":
        return value:u64.round()
      case "floor":
        return value:u64.floor()
      case "ceil":
        return value:u64.ceil()
      case "trunc":
        return value:u64.trunc()
      default:
        return value:u64.round()

  // Parse binary data to integer based on format string
  fn parse_to_u64(&self, data: ptr<u8, len: u64) -> Option<u64>:
    if len < 8:
      return Option.none()

    let value: u64 = 0
    for i in 0..8:
      value = value | ((data[i] as u64) << (i * 8))

    return Option.some(value)

  // Format integer to binary data with specified endianness

```

```

fn format_from_u64(&self, value: u64, output: ptr<u8, len: u64):
    for i in 0..8:
        output[i] = ((value >> (i * 8)) & 0xFF):u8.unchecked()

//
=====
// DEMONSTRATION
//
=====

fn main() -> u32:
    print_string("=== Type Conversion Demo ===\n\n")

    // 1. Basic conversions
    let float_vals: array<f64, 5> = [1.2, 2.7, 3.5, -1.2, -2.8]
    print_string("Float to Integer conversions:\n")

    for f in float_vals:
        let rounded = f:u64.round()
        let floored = f:u64.floor()
        let ceiled = f:u64.ceil()
        let truncated = f:u64.trunc()

        print_f64(f)
        print_string(" -> round:")
        print_dec(rounded)
        print_string(" floor:")
        print_dec(floored)
        print_string(" ceil:")
        print_dec(ceiled)
        print_string(" trunc:")
        print_dec(truncated)
        print_string("\n")

    print_string("\n")

    // 2. Overflow handling
    let large: u64 = 1000
    let small_sat: u8 = large:u8.saturating()
    let small_wrap: u8 = large:u8.wrapping()
    let small_checked = large:u8.checked()

    print_string("Overflow handling (1000 -> u8):\n")
    print_string("  Saturating: ")
    print_dec(small_sat:u64)
    print_string("\n")
    print_string("  Wrapping: ")
    print_dec(small_wrap:u64)
    print_string("\n")

```

```

print_string("  Checked: ")
if small_checked.is_some():
    print_string("Some(")
    print_dec(small_checked.unwrap():u64)
    print_string(")")
else:
    print_string("None")
print_string("\n\n")

// 3. Pointer to integer conversions
let buffer: array<u8, 8> = [0x12, 0x34, 0x56, 0x78, 0x9A, 0xBC, 0xDE,
0xF0]
let buffer_ptr = &buffer[0] as ptr<u8>
let address = buffer_ptr.u64

print_string("Pointer conversion:\n")
print_string("  Buffer address: 0x")
print_hex(address)
print_string("\n")

// 4. Integer to float with precision control
let int_val: u64 = 16777217 // Just beyond exact f32 representation
let as_f32 = int_val.f32
let as_f64 = int_val.f64

print_string("\nInteger to Float precision:\n")
print_string("  Integer: ")
print_dec(int_val)
print_string("\n")
print_string("  As f32: ")
print_f64(as_f32 as f64)
print_string("\n")
print_string("  As f64: ")
print_f64(as_f64)
print_string("\n")

// 5. Tie-breaking examples
let halves: array<f64, 4> = [2.5, 3.5, 4.5, 5.5]
print_string("\nTie-breaking (2.5 -> u32):\n")

for h in halves:
    let tie_even = h.u32.nearest_tie_even()
    let tie_away = h.u32.nearest_tie_away()

    print_f64(h)
    print_string(" -> tie_even: ")
    print_dec(tie_even.u64)
    print_string(" tie_away: ")
    print_dec(tie_away.u64)

```

```

        print_string("\n")

// 6. Saturating arithmetic in practice
print_string("\nSaturating arithmetic in image processing:\n")

let pixel_value: u64 = 280 // Out of 0-255 range
let clamped = pixel_value:u8.saturating()
print_string("  Original pixel: ")
print_dec(pixel_value)
print_string(" -> Clamped: ")
print_dec(clamped:u64)
print_string("\n")

// 7. Checked conversion for error handling
let user_input: u64 = 999999999 // Too large for u32
let result = user_input:u32.checked()

match result:
    case Some(val):
        print_string("  Conversion succeeded: ")
        print_dec(val:u64)
    case None:
        print_string("  Conversion failed: value out of range")
print_string("\n")

return 0

//
=====
// HELPER FUNCTIONS
//
=====

static vga_ptr: mmio_ptr<u16> = 0xB8000 as mmio_ptr<u16>
static cursor: u64 = 0

fn print_char(ch: u8):
    let color = (0x0F << 8) as u16
    vga_ptr[cursor] = color | (ch as u16)
    cursor = cursor + 1
    if cursor >= 80 * 25:
        cursor = 0

fn print_string(s: string):
    for ch in s:
        print_char(ch as u8)

fn print_dec(value: u64):
    if value == 0:

```

```

        print_char('0')
    return
let mut digits: array<u8, 20>
let mut i: u64 = 0
let mut temp = value
while temp > 0:
    digits[i] = (temp % 10) as u8
    temp = temp / 10
    i = i + 1
for j in i..0 step -1:
    print_char('0' + digits[j - 1])

fn print_hex(value: u64):
    let hex_digits = "0123456789ABCDEF"
    for i in 60..0 step -4:
        let nibble = (value >> i) & 0xF
        if nibble != 0 or i == 0:
            print_char(hex_digits[nibble as u64] as u8)
    let last = value & 0xF
    print_char(hex_digits[last as u64] as u8)

fn print_f64(value: f64):
    let int_part = value as u64
    print_dec(int_part)
    print_char('.')
    let frac = (value - (int_part as f64)) * 1000.0
    let frac_abs = if frac < 0.0: -frac else: frac
    let frac_int = frac_abs as u64
    if frac_int < 100:
        print_char('0')
    if frac_int < 10:
        print_char('0')
    print_dec(frac_int)

```

Expected Output:

text

```
=== Type Conversion Demo ===
```

Float to Integer conversions:

```

1.200 -> round:1 floor:1 ceil:2 trunc:1
2.700 -> round:3 floor:2 ceil:3 trunc:2
3.500 -> round:4 floor:3 ceil:4 trunc:3
-1.200 -> round:0 floor:-2 ceil:-1 trunc:-1
-2.800 -> round:0 floor:-3 ceil:-2 trunc:-2

```

Overflow handling (1000 -> u8):

Saturating: 255

Wrapping: 232

Checked: None

Pointer conversion:

Buffer address: 0xFFFF800000007C00

Integer to Float precision:

Integer: 16777217

As f32: 16777216.000

As f64: 16777217.000

Tie-breaking (2.5 -> u32):

2.500 -> tie_even: 2 tie_away: 3

3.500 -> tie_even: 4 tie_away: 4

4.500 -> tie_even: 4 tie_away: 5

5.500 -> tie_even: 6 tie_away: 6

Saturating arithmetic in image processing:

Original pixel: 280 -> Clamped: 255

Checked conversion:

Conversion failed: value out of range

Chapter 7: Control Flow

7.1 The Philosophy of Explicit Control Flow

lowl's control flow constructs are designed to make program execution paths explicit and predictable. Unlike languages that hide control flow behind exceptions or callbacks, lowl uses traditional structured programming constructs (if, while, for, switch) with modern enhancements. Every control flow path is visible in the source code, and the compiler optimizes them aggressively without changing observable behavior.

7.2 If-Elif-Else Chains

The `if` statement in lowl follows Python-style syntax with a colon and indented block. Multiple conditions can be chained using `elif` (short for else-if). The `else` clause is optional and executes when none of the preceding conditions are true. Condition expressions must evaluate to boolean values.

lowl

```
// Basic if statement
let temperature: i64 = 25

if temperature > 30:
    print_string("Hot outside!\n")
elif temperature > 20:
    print_string("Warm day\n")
elif temperature > 10:
```

```

    print_string("Cool day\n")
else:
    print_string("Cold outside!\n")

// Single-line if (colon indicates block, but block can be on same line)
if x == 0: return 0

// Nested if statements
if has_permission:
    if resource_available:
        allocate_resource()
    else:
        print_string("Resource busy\n")
else:
    print_string("Permission denied\n")

// Compound conditions with logical operators
if (user_active && !user_suspended) || is_admin:
    execute_command()

```

7.3 While Loops

The `while` loop repeatedly executes its body as long as the condition expression evaluates to true. The condition is evaluated before each iteration. If the condition is initially false, the loop body never executes. Use `break` to exit the loop early and `continue` to skip the remainder of the current iteration.

lowl

```

// Basic while loop
let mut counter: u64 = 0
while counter < 10:
    print_dec(counter)
    counter = counter + 1

// Infinite loop with break
let mut value: u64 = 0
while true:
    value = value + 1
    if value > 100:
        break
    if value % 2 == 0:
        continue // Skip even numbers
    process_odd(value)

// while loop with complex condition
let mut i: u64 = 0
let mut j: u64 = 100
while i < j && j > 0:

```

```

    i = i + 1
    j = j - 1

// Nested while loops
let mut x: u64 = 0
let mut y: u64 = 0
while x < 10:
    y = 0
    while y < 10:
        matrix[x][y] = x * y
        y = y + 1
    x = x + 1

```

7.4 For Loops: Traditional Form

The traditional for loop consists of three parts: initialization, condition, and increment. The initialization executes once before the loop begins. The condition is evaluated before each iteration; if true, the loop body executes. The increment executes after each iteration before the next condition check.

lowl

```

// Traditional for loop syntax
for (let i = 0; i < 10; i = i + 1):
    print_dec(i)

// For loop with multiple initializers
for (let i = 0, let j = 10; i < j; i = i + 1, j = j - 1):
    swap(&array[i], &array[j])

// For loops with break and continue
for (let i = 0; i < 100; i = i + 1):
    if i % 2 == 0:
        continue
    if i > 50:
        break
    process_odd(i)

// For loop with empty sections
let mut i: u64 = 0
for (; i < 10;):
    i = i + 1

```

7.5 Range-Based For Loops

The range-based for loop iterates over a sequence of values using the `in` keyword. The syntax `for i in start..end` iterates from start (inclusive) to end (exclusive). The syntax `for i in start..=end` iterates inclusive of both ends. An optional `step` value controls increment size.

lowl

```

// Exclusive range (0 to 9)
for i in 0..10:
    print_dec(i)

// Inclusive range (0 to 10)
for i in 0..=10:
    print_dec(i)

// Range with step
for i in 0..100 step 2:
    print_dec(i) // Even numbers only

// Decreasing range with negative step
for i in 10..0 step -1:
    print_dec(i) // Countdown from 10 to 1

// Range over array indices
let arr: array<u64, 5> = [10, 20, 30, 40, 50]
for idx in 0..arr.len():
    print_dec(arr[idx])

// For-each over container
let vec = block_array<u64>.new()
vec.push(1)
vec.push(2)
vec.push(3)

for value in vec:
    print_dec(value)

// For-each with index
for i, value in vec:
    print_string("[")
    print_dec(i)
    print_string("] = ")
    print_dec(value)

```

7.6 Switch Statement with Pattern Matching

The `switch` statement in lowl is more powerful than C's switch, supporting guard conditions (`when` clauses), priority ordering, destructuring, and range patterns. Unlike C, fallthrough must be explicit (using `fallthrough` keyword, not shown in this basic example). Each case must be exhaustive or have a `default` case.

lowl

```
// Basic switch with literal cases
let value: u64 = 42
switch (value):
  case 0:
    print_string("Zero\n")
  case 1:
    print_string("One\n")
  case 42:
    print_string("The answer\n")
  default:
    print_string("Other\n")

// Switch with guard conditions (when)
let x: i64 = -5
switch (x):
  case when (x < 0):
    print_string("Negative\n")
  case when (x == 0):
    print_string("Zero\n")
  case when (x > 0):
    print_string("Positive\n")

// Switch with priority (higher priority checked first)
let score: u64 = 95
switch (score):
  case when (score >= 90):
    priority = 1
    print_string("A\n")
  case when (score >= 80):
    priority = 2
    print_string("B\n")
  case when (score >= 70):
    priority = 3
    print_string("C\n")
  default:
    print_string("F\n")

// Switch with range patterns
let grade: u64 = 85
switch (grade):
  case 90..100:
    print_string("A\n")
  case 80..89:
    print_string("B\n")
  case 70..79:
    print_string("C\n")
  case 60..69:
    print_string("D\n")
```

```
default:
    print_string("F\n")
```

7.7 Multiple Value Switch and Destructuring

lowl's switch can match on multiple values simultaneously and destructure tuple patterns. This is particularly useful for state machines, parsing, and handling combined conditions.

lowl

```
// Switch on multiple values
fn classify_point(x: i64, y: i64) -> string:
    switch (x, y):
        case (0, 0):
            return "origin"
        case (a, 0) when (a > 0):
            return "positive x-axis"
        case (a, 0) when (a < 0):
            return "negative x-axis"
        case (0, b) when (b > 0):
            return "positive y-axis"
        case (0, b) when (b < 0):
            return "negative y-axis"
        case (a, b) when (a == b):
            return "diagonal"
        case (a, b) when (a > 0 and b > 0):
            return "first quadrant"
        case (a, b) when (a < 0 and b > 0):
            return "second quadrant"
        case (a, b) when (a < 0 and b < 0):
            return "third quadrant"
        case (a, b) when (a > 0 and b < 0):
            return "fourth quadrant"
        default:
            return "axis"

// Destructuring in switch
struct Point:
    x: i64
    y: i64

fn process_point(p: Point):
    switch (p):
        case Point{x: 0, y: 0}:
            print_string("Origin\n")
        case Point{x: x, y: 0} when (x > 0):
            print_string("On positive x-axis\n")
        case Point{x: x, y: y}:
            print_string("Point at ("
```

```
print_dec(x)
print_string(", ")
print_dec(y)
print_string("\n")
```

7.8 Return and Early Exit

The `return` statement exits a function, optionally returning a value. Functions declared with a return type must return a value on all paths. Functions declared without an explicit return type (or with `-> void`) may return without a value or use a bare `return`.

lowl

```
// Returning values
fn add(a: u64, b: u64) -> u64:
    return a + b

// Early return
fn find_value(arr: &array<u64, 10>, target: u64) -> Option<u64>:
    for i in 0..arr.len():
        if arr[i] == target:
            return Option.some(arr[i])
    return Option.none()

// Multiple return points
fn classify_number(x: i64) -> string:
    if x < 0:
        return "negative"
    if x == 0:
        return "zero"
    if x > 0:
        return "positive"
    return "impossible" // Unreachable, but required for exhaustive return

// Void return (no value)
fn print_summary():
    print_string("Summary:\n")
    return // Optional, can be omitted
```

7.9 Complete Chapter Example: Command Parser

This example demonstrates control flow constructs in a command-line parser that processes user input, routes commands, and handles errors.

lowl

```
// shell.lowl - Simple command shell with control flow examples
// Compile: lowlc shell.lowl -o shell.asm

//
```

```

=====
// COMMAND STRUCTURE
//
=====

struct Command:
  name: string
  args: array<string, 10>
  arg_count: u64

impl Command:
  fn parse(input: string) -> Option<Command>:
    // Skip leading whitespace
    let mut pos: u64 = 0
    while pos < input.len() and input[pos] == ' ' as u8:
      pos = pos + 1

    if pos >= input.len():
      return Option.none()

    // Parse command name
    let start = pos
    while pos < input.len() and input[pos] != ' ' as u8:
      pos = pos + 1

    let name = input[start..pos]

    // Parse arguments
    let mut args: array<string, 10>
    let mut arg_count: u64 = 0

    while pos < input.len():
      // Skip spaces
      while pos < input.len() and input[pos] == ' ' as u8:
        pos = pos + 1

      if pos >= input.len():
        break

      // Parse argument
      let arg_start = pos
      while pos < input.len() and input[pos] != ' ' as u8:
        pos = pos + 1

      args[arg_count] = input[arg_start..pos]
      arg_count = arg_count + 1

      if arg_count >= 10:
        break

```

```

        return Option.some(Command{name, args, arg_count})

//
=====

// COMMAND HANDLERS
//
=====

fn cmd_help(args: &array<string, 10>, arg_count: u64):
    print_string("Available commands:\n")
    print_string("  help      - Show this help\n")
    print_string("  echo      - Echo text\n")
    print_string("  count     - Count from 1 to N\n")
    print_string("  sum       - Sum numbers\n")
    print_string("  quit      - Exit shell\n")

fn cmd_echo(args: &array<string, 10>, arg_count: u64):
    for i in 0..arg_count:
        print_string(args[i])
        if i < arg_count - 1:
            print_string(" ")
    print_string("\n")

fn cmd_count(args: &array<string, 10>, arg_count: u64):
    if arg_count < 1:
        print_string("Usage: count <N>\n")
        return

    let n = args[0].to_int()
    for i in 1..=n:
        print_dec(i)
        if i < n:
            print_string(", ")
    print_string("\n")

fn cmd_sum(args: &array<string, 10>, arg_count: u64):
    let mut total: u64 = 0
    for i in 0..arg_count:
        total = total + args[i].to_int()
    print_string("Sum: ")
    print_dec(total)
    print_string("\n")

//
=====

// MAIN SHELL LOOP
//
=====

```

```

fn main() -> u32:
    print_string("lowl Shell v1.0\n")
    print_string("Type 'help' for commands, 'quit' to exit\n\n")

    let mut running = true

    while running:
        print_string("> ")
        let input = read_line()

        if input.len() == 0:
            continue

        let opt_cmd = Command.parse(input)

        switch (opt_cmd):
            case None:
                print_string("Invalid command\n")

            case Some(cmd):
                switch (cmd.name):
                    case "help":
                        cmd_help(&cmd.args, cmd.arg_count)

                    case "echo":
                        cmd_echo(&cmd.args, cmd.arg_count)

                    case "count":
                        cmd_count(&cmd.args, cmd.arg_count)

                    case "sum":
                        cmd_sum(&cmd.args, cmd.arg_count)

                    case "quit", "exit":
                        print_string("Goodbye!\n")
                        running = false

                default:
                    print_string("Unknown command: ")
                    print_string(cmd.name)
                    print_string("\n")

    return 0

//
=====
// SIMPLE INPUT READING
//

```

```

=====

fn read_line() -> string:
    static buffer: array<u8, 256>
    let mut pos: u64 = 0

    while pos < 255:
        let ch = read_char()

        if ch == '\n' as u8:
            buffer[pos] = 0
            break
        elif ch == '\b' as u8 and pos > 0:
            pos = pos - 1
            print_char('\b')
        elif ch >= 32:
            buffer[pos] = ch
            pos = pos + 1
            print_char(ch)

    return &buffer[0] as string

fn read_char() -> u8:
    // Wait for keyboard input (simplified)
    while true:
        let status = port_read8(0x64)
        if (status & 1) != 0:
            return port_read8(0x60)
        pause()

//
=====
// VGA OUTPUT (as before)
//
=====

static vga_ptr: mmio_ptr<u16> = 0xB8000 as mmio_ptr<u16>
static cursor: u64 = 0

fn print_char(ch: u8):
    let color = (0x0F << 8) as u16
    if ch == '\n' as u8:
        cursor = cursor + (80 - (cursor % 80))
    else:
        vga_ptr[cursor] = color | (ch as u16)
        cursor = cursor + 1
    if cursor >= 80 * 25:
        cursor = 0

```

```

fn print_string(s: string):
    for ch in s:
        print_char(ch as u8)

fn print_dec(value: u64):
    if value == 0:
        print_char('0')
        return
    let mut digits: array<u8, 20>
    let mut i: u64 = 0
    let mut temp = value
    while temp > 0:
        digits[i] = (temp % 10) as u8
        temp = temp / 10
        i = i + 1
    for j in i..0 step -1:
        print_char('0' + digits[j - 1])

```

Expected Output:

text

```

lowl Shell v1.0
Type 'help' for commands, 'quit' to exit

> help
Available commands:
  help      - Show this help
  echo      - Echo text
  count     - Count from 1 to N
  sum       - Sum numbers
  quit      - Exit shell

> echo Hello World
Hello World

> count 5
1, 2, 3, 4, 5

> sum 10 20 30 40
Sum: 100

> quit
Goodbye!

```

Chapter 8: Functions

8.1 Function Declaration and Parameters

Functions in lowl are declared with the `fn` keyword, followed by the function name, parameters in parentheses, an optional return type after `->`, and a colon followed by an indented block. Parameters are passed by value (copied) unless explicitly passed as pointers. Function names follow the same identifier rules as variables.

lowl

```
// Basic function declaration
fn add(a: u64, b: u64) -> u64:
    return a + b

// Void function (no return value)
fn print_hello():
    print_string("Hello, world!\n")

// Function with default parameter values
fn greet(name: string = "World") -> string:
    return "Hello, " + name + "!"

// Function with multiple return statements
fn max(a: i64, b: i64) -> i64:
    if a > b:
        return a
    else:
        return b

// Function call example
fn main() -> u32:
    let sum = add(10, 20)           // 30
    let bigger = max(100, 50)       // 100
    let greeting = greet()          // "Hello, World!"
    print_string(greeting)
    return 0
```

8.2 Function Parameters: By Value vs By Reference

By default, function parameters are passed by value, meaning the function receives a copy of the argument. Modifying the parameter does not affect the original variable. To modify the original, pass a pointer (by reference) using the `ptr` type.

lowl

```
// Pass by value (copy)
fn increment_by_value(x: u64) -> u64:
    x = x + 1           // Modifies local copy only
    return x
```

```
// Pass by reference (via pointer)
fn increment_by_reference(x: ptr_mut<u64>):
    *x = *x + 1          // Modifies original variable

// Pass by reference using reference operator
fn swap(a: ptr_mut<u64>, b: ptr_mut<u64>):
    let temp = *a
    *a = *b
    *b = temp

// Usage
let mut value: u64 = 10
let copied = increment_by_value(value) // copied = 11, value still 10
increment_by_reference(&mut value)    // value becomes 11
swap(&mut a, &mut b)                  // Exchanges a and b
```

8.3 Function Overloading

lowl supports function overloading: multiple functions with the same name but different parameter types or counts. The compiler selects the appropriate function based on the argument types at the call site.

lowl

```
// Overloaded add functions
fn add(a: u64, b: u64) -> u64:
    return a + b

fn add(a: f64, b: f64) -> f64:
    return a + b

fn add(a: string, b: string) -> string:
    return a + b

// Overloaded with different parameter counts
fn multiply(a: u64) -> u64:
    return a * a

fn multiply(a: u64, b: u64) -> u64:
    return a * b

fn multiply(a: u64, b: u64, c: u64) -> u64:
    return a * b * c

// Usage - compiler selects based on arguments
let int_sum = add(10, 20) // Calls u64 version
let float_sum = add(3.14, 2.71) // Calls f64 version
let str_concat = add("Hello", "World") // Calls string version
```

```
let square = multiply(5)           // 25
let product = multiply(5, 6)       // 30
let cube = multiply(2, 3, 4)       // 24
```

8.4 Inline Functions

The `inline` keyword suggests to the compiler that the function should be expanded at each call site rather than called. This eliminates call overhead and enables further optimizations. Inline functions are ideal for small, frequently called functions.

lowl

```
// Inline function (compiler will expand at call sites)
inline fn fast_square(x: u64) -> u64:
    return x * x

// Inline function with SIMD hint
#[inline]
#[simd(AVX)]
inline fn vector_dot(a: vec8_f32, b: vec8_f32) -> f32:
    return (a * b).hadd()

// Conditional inlining based on optimization level
#[inline(always)]    // Always inline
fn always_inlined(x: u64) -> u64:
    return x + 1

#[inline(never)]     // Never inline (preserve call boundary)
fn never_inlined(x: u64) -> u64:
    return x - 1

// Usage in performance-critical loop
fn process_array(arr: &array<u64, 1000>):
    for i in 0..arr.len():
        let squared = fast_square(arr[i])    // Inlined, no call overhead
        arr[i] = squared
```

8.5 Recursive Functions

lowl supports recursion (functions that call themselves). The compiler optimizes tail recursion when possible. However, for systems programming, recursion depth should be bounded to avoid stack overflow.

lowl

```
// Factorial (recursive)
fn factorial(n: u64) -> u64:
    if n <= 1:
        return 1
    else:
```

```

        return n * factorial(n - 1)

// Tail-recursive factorial (optimized to loop)
fn factorial_tail(n: u64, acc: u64) -> u64:
    if n <= 1:
        return acc
    else:
        return factorial_tail(n - 1, n * acc)

// Fibonacci (recursive, not tail-recursive)
fn fibonacci(n: u64) -> u64:
    if n <= 1:
        return n
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)

// Recursive directory traversal
fn traverse_directory(path: string, depth: u64):
    if depth > 100:    // Safety bound
        return

    let entries = read_directory(path)
    for entry in entries:
        if entry.is_directory():
            traverse_directory(entry.path, depth + 1)
        else:
            process_file(entry)

```

8.6 Varargs Functions

Functions can accept a variable number of arguments using the `...` syntax. The arguments are passed as an array or via a special varargs interface.

lowl

```

// Varargs function using array parameter
fn sum(values: ...u64) -> u64:
    let mut total: u64 = 0
    for v in values:
        total = total + v
    return total

// Varargs with different types (using tagged union)
fn print_many(fmt: string, ...any):
    // Implementation would parse format string and arguments
    pass

// Usage examples
let total1 = sum() // 0

```

```

let total2 = sum(1, 2, 3)           // 6
let total3 = sum(10, 20, 30, 40, 50) // 150

// C-style printf equivalent (simplified)
fn my_printf(fmt: string, ...):
  let args: array<any, 10> = ...args
  for i in 0..fmt.len():
    if fmt[i] == '%':
      let arg = args.next()
      print_any(arg)
    else:
      print_char(fmt[i])

```

8.7 Functions as First-Class Citizens

Function pointers allow passing functions as arguments, storing them in variables, and returning them from functions. This enables callback mechanisms and functional programming patterns.

lowI

```

// Function type alias
type MapFn = fn(u64) -> u64
type FilterFn = fn(u64) -> bool

// Function that takes another function as argument
fn map_array(arr: &mut array<u64, 10>, f: MapFn):
  for i in 0..arr.len():
    arr[i] = f(arr[i])

fn filter_array(arr: &array<u64, 10>, predicate: FilterFn) -> u64:
  let mut count: u64 = 0
  for i in 0..arr.len():
    if predicate(arr[i]):
      count = count + 1
  return count

// Callback functions
fn double(x: u64) -> u64:
  return x * 2

fn is_even(x: u64) -> bool:
  return x % 2 == 0

// Function returning a function pointer
fn get_operation(op: string) -> fn(u64, u64) -> u64:
  switch (op):
    case "add":
      return add
    case "mul":

```

```

        return multiply
    default:
        return null

// Usage
let mut numbers: array<u64, 5> = [1, 2, 3, 4, 5]
map_array(&mut numbers, double)           // [2, 4, 6, 8, 10]
let evens = filter_array(&numbers, is_even) // 5

let op = get_operation("add")
let result = op(10, 20)                    // 30

```

8.8 Lambda Expressions (Anonymous Functions)

Lambda expressions create inline functions without a separate declaration. They capture variables from their surrounding scope by reference.

lowl

```

// Basic lambda
let square = fn(x: u64) -> u64: return x * x
let result = square(5)    // 25

// Lambda as argument
let arr: array<u64, 5> = [1, 2, 3, 4, 5]
map_array(&arr, fn(x: u64) -> u64: return x + 10)

// Lambda with capture
let base: u64 = 10
let add_base = fn(x: u64) -> u64: return x + base // Captures base
let value = add_base(5)    // 15

// Closure that modifies captured variable
let mut counter: u64 = 0
let increment = fn() -> u64:
    counter = counter + 1
    return counter

let first = increment()    // 1
let second = increment()   // 2
let third = increment()    // 3

// Lambda in sort algorithm
fn sort(arr: &mut array<u64, 10>, comparator: fn(u64, u64) -> i8):
    // Sorting implementation using comparator

sort(&mut arr, fn(a: u64, b: u64) -> i8:
    if a < b: return -1
    elif a > b: return 1

```

```

    else: return 0
)

```

8.9 Function Attributes

Attributes modify function behavior: `#[kernel]` for ring-0 kernel code, `#[interrupt]` for interrupt handlers, `#[init]` for functions called before main, and `#[section]` to place functions in specific binary sections.

lowl

```

// Kernel function (runs at ring 0)
#[kernel]
fn kernel_entry():
    disable_interrupts()
    // Kernel initialization code
    kernel_main()
    while true:
        halt()

// Interrupt handler
#[interrupt]
fn page_fault_handler():
    let fault_address = read_cr2()
    let error_code = asm("mov rax, [rsp+16]")
    handle_page_fault(fault_address, error_code)

// Init function (called before main)
#[init(priority=10)]
fn early_init():
    // Runs before main, priority 10 (higher priority = earlier)
    init_console()

// Place function in specific section
#[section(".text.boot")]
fn boot_start():
    // Boot code placed at beginning of binary

// Naked function (no prologue/epilogue)
#[naked]
fn syscall_entry():
    asm("swapgs")
    asm("mov gs:[0x10], rsp")
    asm("mov rsp, gs:[0x18]")
    asm("push rax")
    // ... no compiler-generated prologue

```

8.10 Error Handling with Option Return

lowl does not have exceptions. Instead, functions that can fail return `Option<T>` or `Result<T, E>` types (the latter in the standard library). This makes error handling explicit and visible.

lowl

```
// Function returning Option (may have no value)
fn divide(a: u64, b: u64) -> Option<u64>:
    if b == 0:
        return Option.none()
    return Option.some(a / b)

// Function returning Result (error information)
fn open_file(path: string) -> Result<File, ErrorCode>:
    let file = find_file(path)
    if file == null:
        return Result.err(ErrorCode.NOT_FOUND)
    if not has_permission(file):
        return Result.err(ErrorCode.PERMISSION_DENIED)
    return Result.ok(file)

// Handling Option results
let opt_result = divide(10, 2)
match opt_result:
    case Some(value):
        print_dec(value)
    case None:
        print_string("Division by zero!\n")

// Propagating errors with ? operator (simplified)
fn safe_divide_chain(a: u64, b: u64, c: u64) -> Option<u64>:
    let temp = divide(a, b)? // Returns early if None
    return divide(temp, c)

// Unwrapping with default
let result = divide(10, 0).unwrap_or(0) // 0
let result2 = divide(10, 2).expect("Division failed") // 5 or panic
```

8.11 Complete Chapter Example: Calculator with Functions

lowl

```
// calculator.lowl - Calculator demonstrating all function features
// Compile: lowlc calculator.lowl -o calculator.asm

//
=====
// FUNCTION DECLARATIONS
```

```
//
=====

// Basic arithmetic
inline fn add(a: f64, b: f64) -> f64: return a + b
inline fn sub(a: f64, b: f64) -> f64: return a - b
inline fn mul(a: f64, b: f64) -> f64: return a * b

fn div(a: f64, b: f64) -> Option<f64>:
    if b == 0.0:
        return Option.none()
    return Option.some(a / b)

// Power using recursion
fn power(base: f64, exp: u64) -> f64:
    if exp == 0:
        return 1.0
    elif exp == 1:
        return base
    else:
        return base * power(base, exp - 1)

// Overloaded factorial (demonstrates overloading)
fn factorial(n: u64) -> u64:
    if n <= 1:
        return 1
    return n * factorial(n - 1)

// Function pointer type
type BinaryOp = fn(f64, f64) -> Option<f64>

// Calculator state
struct Calculator:
    memory: f64
    last_result: f64

impl Calculator:
    fn new() -> Calculator:
        return Calculator{0.0, 0.0}

    fn evaluate(&self, op: BinaryOp, a: f64, b: f64) -> Option<f64>:
        return op(a, b)

    fn store_memory(&mut self, value: f64):
        self.memory = value

    fn recall_memory(&self) -> f64:
        return self.memory
```

```
// Lambda-based operation dispatcher
fn get_operation(op_name: string) -> BinaryOp:
    match op_name:
        case "add":
            return fn(a: f64, b: f64) -> Option<f64>: return
Option.some(add(a, b))
        case "sub":
            return fn(a: f64, b: f64) -> Option<f64>: return
Option.some(sub(a, b))
        case "mul":
            return fn(a: f64, b: f64) -> Option<f64>: return
Option.some(mul(a, b))
        case "div":
            return div
        default:
            return null

//
=====
// DEMONSTRATION
//
=====

fn main() -> u32:
    print_string("=== Calculator Demo ===\n\n")

    // 1. Basic function calls
    let a: f64 = 10.0
    let b: f64 = 3.0

    print_string("Arithmetic operations:\n")
    print_string("  10 + 3 = ")
    print_f64(add(a, b))
    print_string("\n")
    print_string("  10 - 3 = ")
    print_f64(sub(a, b))
    print_string("\n")
    print_string("  10 * 3 = ")
    print_f64(mul(a, b))
    print_string("\n")

    let opt_div = div(a, b)
    match opt_div:
        case Some(v):
            print_string("  10 / 3 = ")
            print_f64(v)
            print_string("\n")
        case None:
            print_string("  Division by zero!\n")
```

```

// 2. Recursion
print_string("\nRecursion:\n")
print_string("  Factorial 5 = ")
print_dec(factorial(5))
print_string("\n")
print_string("  2^10 = ")
print_f64(power(2.0, 10))
print_string("\n")

// 3. Inline function performance
let arr: array<f64, 5> = [1.0, 2.0, 3.0, 4.0, 5.0]
print_string("\nMapping array (inline function):\n [")
for i in 0..arr.len():
    let doubled = mul(arr[i], 2.0) // Inlined multiplication
    print_f64(doubled)
    if i < arr.len() - 1:
        print_string(", ")
print_string("]\n")

// 4. Function pointer
let ops = ["add", "sub", "mul", "div"]
print_string("\nFunction pointer dispatch:\n")

for op_name in ops:
    let op = get_operation(op_name)
    if op != null:
        let result = op(20.0, 5.0)
        match result:
            case Some(v):
                print_string(" ")
                print_string(op_name)
                print_string("(20, 5) = ")
                print_f64(v)
                print_string("\n")
            case None:
                pass

// 5. Lambda as accumulator
print_string("\nLambda accumulation:\n")
let mut running_total: f64 = 0.0
let accumulator = fn(x: f64) -> f64:
    running_total = running_total + x
    return running_total

for i in 1..6:
    let val = accumulator(i as f64)
    print_string("  After adding ")
    print_dec(i)

```

```

        print_string(": ")
        print_f64(val)
        print_string("\n")

// 6. Calculator object
let mut calc = Calculator.new()
calc.store_memory(42.0)
print_string("\nCalculator memory: ")
print_f64(calc.recall_memory())
print_string("\n")

let opt = calc.evaluate(div, 100.0, 0.0)
match opt:
    case None:
        print_string("  Division by zero prevented!\n")
    case Some(v):
        pass

return 0

//
=====
// HELPER FUNCTIONS
//
=====

static vga_ptr: mmio_ptr<u16> = 0xB8000 as mmio_ptr<u16>
static cursor: u64 = 0

fn print_char(ch: u8):
    let color = (0x0F << 8) as u16
    if ch == '\n' as u8:
        cursor = cursor + (80 - (cursor % 80))
    else:
        vga_ptr[cursor] = color | (ch as u16)
        cursor = cursor + 1
    if cursor >= 80 * 25:
        cursor = 0

fn print_string(s: string):
    for ch in s:
        print_char(ch as u8)

fn print_dec(value: u64):
    if value == 0:
        print_char('0')
        return
    let mut digits: array<u8, 20>
    let mut i: u64 = 0

```

```

let mut temp = value
while temp > 0:
    digits[i] = (temp % 10) as u8
    temp = temp / 10
    i = i + 1
for j in i..0 step -1:
    print_char('0' + digits[j - 1])

fn print_f64(value: f64):
    let int_part = value as u64
    print_dec(int_part)
    print_char('.')
    let frac = (value - (int_part as f64)) * 1000.0
    let frac_abs = if frac < 0.0: -frac else: frac
    let frac_int = frac_abs as u64
    if frac_int < 100:
        print_char('0')
    if frac_int < 10:
        print_char('0')
    print_dec(frac_int)

```

Expected Output:

text

```

=== Calculator Demo ===

Arithmetic operations:
10 + 3 = 13.000
10 - 3 = 7.000
10 * 3 = 30.000
10 / 3 = 3.333

Recursion:
Factorial 5 = 120
2^10 = 1024.000

Mapping array (inline function):
[2.000, 4.000, 6.000, 8.000, 10.000]

Function pointer dispatch:
add(20, 5) = 25.000
sub(20, 5) = 15.000
mul(20, 5) = 100.000
div(20, 5) = 4.000

Lambda accumulation:
After adding 1: 1.000
After adding 2: 3.000

```

After adding 3: 6.000

After adding 4: 10.000

After adding 5: 15.000

Calculator memory: 42.000

Division by zero prevented!

Chapter 9: Object-Oriented Programming

9.1 Classes and Instances

lowl supports object-oriented programming with classes, inheritance, polymorphism, and encapsulation. A class defines the structure (fields) and behavior (methods) of objects. Objects are instances of classes, created with the `new` keyword.

lowl

```
// Basic class definition
class Counter:
  private:
    value: u64 = 0

  public:
    fn new() -> Counter:
      return this

    fn increment() -> u64:
      this.value = this.value + 1
      return this.value

    fn get() -> u64:
      return this.value

    fn reset():
      this.value = 0

// Creating and using objects
let mut c = Counter.new()
c.increment()
c.increment()
let val = c.get()      // 2
c.reset()
```

9.2 Constructors and Destructors

Constructors (named `new`) initialize new objects. Destructors (named `delete`) clean up resources when objects are destroyed. Constructors can have parameters; destructors take no arguments.

lowl

```

class Resource:
    private:
        buffer: ptr<u8>
        size: u64

    public:
        // Constructor with parameters
        fn new(sz: u64) -> Resource:
            this.size = sz
            this.buffer = physical_alloc(sz, 64)
            zero_memory(this.buffer, sz)
            return this

        // Copy constructor
        fn new(other: &Resource) -> Resource:
            this.size = other.size
            this.buffer = physical_alloc(this.size, 64)
            copy_memory(this.buffer, other.buffer, this.size)
            return this

        // Move constructor
        fn new(other: &&Resource) -> Resource:
            this.size = other.size
            this.buffer = other.buffer
            other.buffer = null
            other.size = 0
            return this

        // Destructor
        fn delete():
            if this.buffer != null:
                physical_free(this.buffer)
                this.buffer = null
            this.size = 0

// Usage
let r1 = Resource.new(4096)           // Constructor
let r2 = Resource.new(&r1)           // Copy constructor
let r3 = Resource.new(&&r1)           // Move constructor
delete r1                             // Destructor

```

9.3 Methods and `this`

Methods are functions defined within a class's `impl` block (or inside the class body). They have access to the instance via the `this` keyword, which is an implicit pointer to the current object. Methods can be called using dot notation.

lowl

```

class Vector3D:
    public:
        x: f64
        y: f64
        z: f64

    impl:
        fn new(x: f64, y: f64, z: f64) -> Vector3D:
            this.x = x
            this.y = y
            this.z = z
            return this

        fn length() -> f64:
            return sqrt(this.x * this.x + this.y * this.y + this.z * this.z)

        fn normalize() -> Vector3D:
            let len = this.length()
            if len > 0.0:
                return Vector3D.new(this.x / len, this.y / len, this.z /
len)

            return Vector3D.new(0.0, 0.0, 0.0)

        fn dot(self, other: &Vector3D) -> f64:
            return this.x * other.x + this.y * other.y + this.z * other.z

        fn add(self, other: &Vector3D) -> Vector3D:
            return Vector3D.new(
                this.x + other.x,
                this.y + other.y,
                this.z + other.z
            )

// Usage
let v1 = Vector3D.new(1.0, 2.0, 3.0)
let v2 = Vector3D.new(4.0, 5.0, 6.0)
let dot_product = v1.dot(&v2)
let sum = v1.add(&v2)
let magnitude = v1.length()

```

9.4 Inheritance with **extends**

Classes can inherit from a base class using the **extends** keyword. Derived classes inherit all fields and methods from the base class. The **super** keyword refers to the base class.

lowl

```

// Base class
class Animal:

```

```

protected:
    name: string

public:
    fn new(name: string) -> Animal:
        this.name = name
        return this

    virtual fn speak() -> string:
        return "???"

    fn get_name() -> string:
        return this.name

// Derived class
class Dog extends Animal:
    public:
        fn new(name: string) -> Dog:
            super.new(name)
            return this

        override fn speak() -> string:
            return "Woof!"

        fn fetch() -> string:
            return "Fetching the stick!"

// Another derived class
class Cat extends Animal:
    public:
        fn new(name: string) -> Cat:
            super.new(name)
            return this

        override fn speak() -> string:
            return "Meow!"

// Usage
let dog = Dog.new("Rex")
let cat = Cat.new("Whiskers")

print_string(dog.get_name())    // "Rex"
print_string(dog.speak())       // "Woof!"
print_string(cat.speak())       // "Meow!"

```

9.5 Virtual Methods and Polymorphism

Virtual methods (marked with `virtual`) enable polymorphic behavior: calling a method on a base class pointer invokes the derived class's implementation. The compiler generates a vtable

(virtual function table) for each class with virtual methods.

lowI

```
// Abstract base class with virtual methods
abstract class Shape:
    public:
        virtual fn area() -> f64
        virtual fn perimeter() -> f64
        virtual fn name() -> string

// Concrete derived class
class Rectangle extends Shape:
    private:
        width: f64
        height: f64

    public:
        fn new(w: f64, h: f64) -> Rectangle:
            this.width = w
            this.height = h
            return this

        override fn area() -> f64:
            return this.width * this.height

        override fn perimeter() -> f64:
            return 2.0 * (this.width + this.height)

        override fn name() -> string:
            return "Rectangle"

class Circle extends Shape:
    private:
        radius: f64

    public:
        fn new(r: f64) -> Circle:
            this.radius = r
            return this

        override fn area() -> f64:
            return 3.141592653589793 * this.radius * this.radius

        override fn perimeter() -> f64:
            return 2.0 * 3.141592653589793 * this.radius

        override fn name() -> string:
            return "Circle"
```

```
// Polymorphic function
fn print_shape_info(shape: &Shape):
    print_string("Shape: ")
    print_string(shape.name())
    print_string(", Area: ")
    print_f64(shape.area())
    print_string(", Perimeter: ")
    print_f64(shape.perimeter())
    print_string("\n")

// Usage
let rect = Rectangle.new(10.0, 20.0)
let circle = Circle.new(15.0)

let shapes: array<ptr<Shape>, 2> = [&rect, &circle]
for shape in shapes:
    print_shape_info(shape)    // Calls appropriate overrides
```

9.6 Access Control: Public, Private, Protected

lowl provides three access levels: **public** (accessible anywhere), **private** (accessible only within the same class), and **protected** (accessible within the class and derived classes).

Access modifiers apply to sections of the class definition.

lowl

```
class BankAccount:
    private:
        account_number: u64
        balance: i64
        pin_hash: u32

    protected:
        transaction_history: array<Transaction, 100>
        history_count: u64

    public:
        fn new(account: u64, pin: u32) -> BankAccount:
            this.account_number = account
            this.pin_hash = hash(pin)
            this.balance = 0
            this.history_count = 0
            return this

        fn deposit(amount: u64) -> bool:
            if amount > 0:
                this.balance = this.balance + (amount as i64)
                this.record_transaction("deposit", amount)
```

```

        return true
    return false

fn withdraw(amount: u64, pin: u32) -> bool:
    if hash(pin) != this.pin_hash:
        return false
    if (amount as i64) <= this.balance:
        this.balance = this.balance - (amount as i64)
        this.record_transaction("withdraw", amount)
        return true
    return false

fn get_balance() -> i64:
    return this.balance

private:
    fn record_transaction(type: string, amount: u64):
        if this.history_count < 100:
            this.transaction_history[this.history_count] =
Transaction.new(type, amount)
            this.history_count = this.history_count + 1

// Protected members accessible in derived class
class SavingsAccount extends BankAccount:
    private:
        interest_rate: f64

    public:
        fn new(account: u64, pin: u32, rate: f64) -> SavingsAccount:
            super.new(account, pin)
            this.interest_rate = rate
            return this

        fn add_interest():
            let interest = (this.get_balance() as f64) * this.interest_rate
            super.deposit(interest as u64) // deposit is public
            // Can access protected transaction_history
            this.record_transaction("interest", interest as u64)

```

9.7 Static Methods and Fields

Static methods belong to the class itself rather than to instances. They are called using the class name. Static fields are shared across all instances of the class. Static members are declared using the `static` keyword within the class.

lowl

```

class Logger:
    private static:

```

```

log_level: u32 = 2          // 0=error, 1=warning, 2=info, 3=debug
log_count: u64 = 0
log_buffer: array<string, 1024>

public static:
  fn set_level(level: u32):
    Logger.log_level = level

  fn info(msg: string):
    if Logger.log_level >= 2:
      Logger.log_count = Logger.log_count + 1
      Logger.log_buffer[Logger.log_count - 1] = "[INFO] " + msg
      print_string("[INFO] ")
      print_string(msg)
      print_string("\n")

  fn error(msg: string):
    if Logger.log_level >= 0:
      Logger.log_count = Logger.log_count + 1
      Logger.log_buffer[Logger.log_count - 1] = "[ERROR] " + msg
      print_string("[ERROR] ")
      print_string(msg)
      print_string("\n")

  fn get_count() -> u64:
    return Logger.log_count

  fn get_log(index: u64) -> Option<string>:
    if index < Logger.log_count:
      return Option.some(Logger.log_buffer[index])
    return Option.none()

class UniqueId:
  private static:
    next_id: u64 = 1000

  public static:
    fn generate() -> u64:
      let id = UniqueId.next_id
      UniqueId.next_id = UniqueId.next_id + 1
      return id

// Usage - no instance needed
Logger.set_level(3)
Logger.info("System initialized")
Logger.error("Disk full warning")

let id1 = UniqueId.generate() // 1000

```

```
let id2 = UniqueId.generate() // 1001
let id3 = UniqueId.generate() // 1002
```

9.8 Operator Overloading

lowl allows overloading operators for custom types by implementing methods with special names. This enables intuitive syntax for mathematical and container types.

lowl

```
class Complex:
  public:
    re: f64
    im: f64

  impl:
    fn new(r: f64, i: f64) -> Complex:
      return Complex{re: r, im: i}

    // Overload + operator
    fn add(other: &Complex) -> Complex:
      return Complex.new(this.re + other.re, this.im + other.im)

    // Overload - operator
    fn subtract(other: &Complex) -> Complex:
      return Complex.new(this.re - other.re, this.im - other.im)

    // Overload * operator
    fn multiply(other: &Complex) -> Complex:
      return Complex.new(
        this.re * other.re - this.im * other.im,
        this.re * other.im + this.im * other.re
      )

    // Overload == operator
    fn equals(other: &Complex) -> bool:
      return this.re == other.re and this.im == other.im

    // Overload unary -
    fn negate() -> Complex:
      return Complex.new(-this.re, -this.im)

    // Conversion to string
    fn to_string() -> string:
      return "(" + this.re.to_string() + " + " + this.im.to_string() +
        "i)"

    // Usage with operators (compiler translates to method calls)
    let c1 = Complex.new(1.0, 2.0)
```

```
let c2 = Complex.new(3.0, 4.0)
let c3 = c1 + c2           // Calls c1.add(&c2)
let c4 = c3 - c1           // Calls c3.subtract(&c1)
let c5 = c1 * c2           // Calls c1.multiply(&c2)
let is_eq = c1 == c1       // Calls c1.equals(&c1)
let c6 = -c1               // Calls c1.negate()
```

9.9 Interfaces and Abstract Classes

Abstract classes cannot be instantiated; they serve as base classes that define interfaces for derived classes. Pure virtual methods (marked `virtual` without a body) must be overridden by concrete derived classes.

lowI

```
// Abstract interface (all methods pure virtual)
abstract class Drawable:
    public:
        virtual fn draw(x: u64, y: u64) -> void
        virtual fn get_width() -> u64
        virtual fn get_height() -> u64

// Another abstract interface
abstract class Serializable:
    public:
        virtual fn serialize() -> array<u8>
        virtual fn deserialize(data: &array<u8>) -> bool

// Concrete class implementing multiple abstract classes
class Sprite extends Drawable, Serializable:
    private:
        image_data: ptr<u8>
        width: u64
        height: u64
        name: string

    public:
        fn new(w: u64, h: u64, name: string) -> Sprite:
            this.width = w
            this.height = h
            this.name = name
            this.image_data = physical_alloc(w * h, 64)
            zero_memory(this.image_data, w * h)
            return this

        override fn draw(x: u64, y: u64):
            // Draw sprite at position (x, y)
            for i in 0..this.height:
                for j in 0..this.width:
```

```

        let pixel = this.image_data[i * this.width + j]
        draw_pixel(x + j, y + i, pixel)

    override fn get_width() -> u64:
        return this.width

    override fn get_height() -> u64:
        return this.height

    override fn serialize() -> array<u8>:
        let mut result: array<u8, 256>
        let mut pos: u64 = 0

        // Serialize width and height
        result[pos] = (this.width >> 0) & 0xFF
        result[pos + 1] = (this.width >> 8) & 0xFF
        result[pos + 2] = (this.height >> 0) & 0xFF
        result[pos + 3] = (this.height >> 8) & 0xFF
        pos = pos + 4

        // Serialize name
        for ch in this.name:
            result[pos] = ch as u8
            pos = pos + 1
        result[pos] = 0
        pos = pos + 1

        // Serialize image data (first 256 bytes only for demo)
        for i in 0..256:
            result[pos + i] = this.image_data[i]

        return result

    override fn deserialize(data: &array<u8>) -> bool:
        // Implementation would parse serialized data
        return true

// Function that works with any Drawable
fn render_scene(drawables: &array<ptr<Drawable>, 10>):
    for d in drawables:
        d.draw(0, 0)

// Function that works with any Serializable
fn save_to_file(obj: &Serializable, path: string) -> bool:
    let data = obj.serialize()
    return write_file(path, &data)

```

9.10 Template Classes (Overview)

Template classes allow creating generic containers and algorithms that work with any type. The `template<class T>` syntax declares a template parameter. Complete coverage is in Chapter 10.

lowl

```
// Simple template class (detailed in Chapter 10)
template<class T>
class Stack:
    private:
        data: array<T, 100>
        top: u64 = 0

    public:
        fn push(item: T):
            if this.top < 100:
                this.data[this.top] = item
                this.top = this.top + 1

        fn pop() -> Option<T>:
            if this.top > 0:
                this.top = this.top - 1
                return Option.some(this.data[this.top])
            return Option.none()

        fn is_empty() -> bool:
            return this.top == 0

// Usage with different types
let int_stack = Stack<u64>.new()
int_stack.push(10)
int_stack.push(20)
let val = int_stack.pop()      // Some(20)

let string_stack = Stack<string>.new()
string_stack.push("Hello")
string_stack.push("World")
```

9.11 Complete Chapter Example: GUI Widget System

This example demonstrates OOP concepts by implementing a simple GUI widget system with inheritance, polymorphism, virtual methods, and encapsulation.

lowl

```
// gui.lowl - Object-oriented GUI widget system
// Compile: lowlc gui.lowl -o gui.asm -O2
```

```
//
=====
// BASE WIDGET CLASS (ABSTRACT)
//
=====

abstract class Widget:
    protected:
        x: i32
        y: i32
        width: u32
        height: u32
        visible: bool = true
        parent: ptr<Widget> = null

    public:
        fn new(x: i32, y: i32, w: u32, h: u32) -> Widget:
            this.x = x
            this.y = y
            this.width = w
            this.height = h
            return this

        virtual fn draw() -> void
        virtual fn handle_click(mx: i32, my: i32) -> bool
        virtual fn handle_key(key: u8) -> bool

        fn move_to(new_x: i32, new_y: i32):
            this.x = new_x
            this.y = new_y

        fn show():
            this.visible = true
            this.draw()

        fn hide():
            this.visible = false

        fn set_parent(p: ptr<Widget>):
            this.parent = p

        fn get_x() -> i32: return this.x
        fn get_y() -> i32: return this.y
        fn get_width() -> u32: return this.width
        fn get_height() -> u32: return this.height
        fn is_visible() -> bool: return this.visible

//
=====
```

```
// BUTTON WIDGET
//
=====

class Button extends Widget:
  private:
    label: string
    callback: fn()
    is_pressed: bool = false
    border_color: u16 = 0x0F00 // White border
    bg_color: u16 = 0x0700 // Gray background
    text_color: u16 = 0x0F00 // White text

  public:
    fn new(x: i32, y: i32, w: u32, h: u32, lbl: string, cb: fn()) ->
Button:
    super.new(x, y, w, h)
    this.label = lbl
    this.callback = cb
    return this

    override fn draw():
      if not this.visible:
        return

      // Draw button border
      draw_rect(this.x, this.y, this.width, this.height,
this.border_color)

      // Draw button background (inset for pressed state)
      let bg = if this.is_pressed: this.bg_color & 0x0F0F else:
this.bg_color
      draw_filled_rect(this.x + 1, this.y + 1, this.width - 2,
this.height - 2, bg)

      // Draw label centered
      let text_x = this.x + ((this.width as i32) - (this.label.len()
as i32 * 8)) / 2
      let text_y = this.y + ((this.height as i32) - 8) / 2
      draw_string(text_x, text_y, this.label, this.text_color)

    override fn handle_click(mx: i32, my: i32) -> bool:
      if not this.visible:
        return false

      if mx >= this.x and mx < this.x + (this.width as i32) and
my >= this.y and my < this.y + (this.height as i32):
        // Button clicked
        this.is_pressed = true
```

```

        this.draw()

        // Execute callback
        if this.callback != null:
            this.callback()

        this.is_pressed = false
        this.draw()
        return true

    return false

    override fn handle_key(key: u8) -> bool:
        return false

//
=====
// TEXTBOX WIDGET
//
=====

class TextBox extends Widget:
    private:
        text: array<u8, 256>
        text_len: u64 = 0
        cursor_pos: u64 = 0
        is_focused: bool = false
        border_color: u16 = 0x0F00
        bg_color: u16 = 0x0000          // Black background
        text_color: u16 = 0x0F00

    public:
        fn new(x: i32, y: i32, w: u32) -> TextBox:
            super.new(x, y, w, 16)
            for i in 0..256:
                this.text[i] = 0
            return this

        override fn draw():
            if not this.visible:
                return

            draw_rect(this.x, this.y, this.width, this.height,
this.border_color)

            let bg = if this.is_focused: 0x0100 else: this.bg_color
            draw_filled_rect(this.x + 1, this.y + 1, this.width - 2,
this.height - 2, bg)

```

```

// Draw text
let text_str = &this.text[0] as string
draw_string(this.x + 2, this.y + 2, text_str, this.text_color)

// Draw cursor if focused
if this.is_focused:
    let cursor_x = this.x + 2 + (this.cursor_pos * 8) as i32
    draw_rect(cursor_x, this.y + 2, 1, 12, 0xF00)

override fn handle_click(mx: i32, my: i32) -> bool:
    if not this.visible:
        return false

    let was_focused = this.is_focused
    this.is_focused = (mx >= this.x and mx < this.x + (this.width as
i32) and
                                my >= this.y and my < this.y + (this.height
as i32))

    if this.is_focused != was_focused:
        this.draw()

    return this.is_focused

override fn handle_key(key: u8) -> bool:
    if not this.visible or not this.is_focused:
        return false

    if key == '\b' as u8 and this.cursor_pos > 0:
        // Backspace
        for i in this.cursor_pos - 1..this.text_len - 1:
            this.text[i] = this.text[i + 1]
        this.text_len = this.text_len - 1
        this.cursor_pos = this.cursor_pos - 1
        this.text[this.text_len] = 0
        this.draw()
        return true

    elif key >= 32 and key < 127 and this.text_len < 255:
        // Insert character
        for i in this.text_len..this.cursor_pos step -1:
            this.text[i] = this.text[i - 1]
        this.text[this.cursor_pos] = key
        this.text_len = this.text_len + 1
        this.cursor_pos = this.cursor_pos + 1
        this.text[this.text_len] = 0
        this.draw()
        return true

```

```

        elif key == 0x4B and this.cursor_pos > 0: // Left arrow
            this.cursor_pos = this.cursor_pos - 1
            this.draw()
            return true

        elif key == 0x4D and this.cursor_pos < this.text_len: // Right
arrow
            this.cursor_pos = this.cursor_pos + 1
            this.draw()
            return true

        return false

    fn get_text() -> string:
        return &this.text[0] as string

//
=====
// PANEL (CONTAINER WIDGET)
//
=====

class Panel extends Widget:
    private:
        children: array<ptr<Widget>, 10>
        child_count: u64 = 0
        bg_color: u16 = 0x0700

    public:
        fn new(x: i32, y: i32, w: u32, h: u32) -> Panel:
            super.new(x, y, w, h)
            return this

        fn add_child(child: ptr<Widget>):
            if this.child_count < 10:
                this.children[this.child_count] = child
                this.child_count = this.child_count + 1
                child.set_parent(this)

        override fn draw():
            if not this.visible:
                return

            draw_filled_rect(this.x, this.y, this.width, this.height,
this.bg_color)

            for i in 0..this.child_count:
                this.children[i].draw()

```

```

    override fn handle_click(mx: i32, my: i32) -> bool:
        if not this.visible:
            return false

        for i in 0..this.child_count:
            if this.children[i].handle_click(mx, my):
                return true

        return false

    override fn handle_key(key: u8) -> bool:
        if not this.visible:
            return false

        for i in 0..this.child_count:
            if this.children[i].handle_key(key):
                return true

        return false

//
=====
// GRAPHICS HELPER FUNCTIONS (VGA)
//
=====

const VGA_WIDTH: i32 = 80
const VGA_HEIGHT: i32 = 25
static vga_ptr: mmio_ptr<u16> = 0xB8000 as mmio_ptr<u16>

fn draw_pixel(x: i32, y: i32, color: u16):
    if x >= 0 and x < VGA_WIDTH and y >= 0 and y < VGA_HEIGHT:
        vga_ptr[(y * VGA_WIDTH + x) as u64] = color

fn draw_rect(x: i32, y: i32, w: u32, h: u32, color: u16):
    for i in 0..w:
        draw_pixel(x + (i as i32), y, color)
        draw_pixel(x + (i as i32), y + (h as i32) - 1, color)
    for i in 0..h:
        draw_pixel(x, y + (i as i32), color)
        draw_pixel(x + (w as i32) - 1, y + (i as i32), color)

fn draw_filled_rect(x: i32, y: i32, w: u32, h: u32, color: u16):
    for dy in 0..h:
        for dx in 0..w:
            draw_pixel(x + (dx as i32), y + (dy as i32), color)

fn draw_char(x: i32, y: i32, ch: u8, color: u16):
    draw_pixel(x, y, color | (ch as u16))

```

```

fn draw_string(x: i32, y: i32, s: string, color: u16):
    let mut cx = x
    for ch in s:
        draw_char(cx, y, ch as u8, color)
        cx = cx + 8

fn clear_screen():
    for y in 0..VGA_HEIGHT:
        for x in 0..VGA_WIDTH:
            vga_ptr[(y * VGA_WIDTH + x) as u64] = 0x0700 | (' ' as u16)

//
=====
// MAIN APPLICATION
//
=====

// Global callback functions
fn on_button_click():
    print_string_at(0, 24, "Button clicked!")

fn main() -> u32:
    clear_screen()

    // Create widgets
    let panel = Panel.new(5, 2, 70, 20)
    let button = Button.new(10, 5, 20, 3, "Click Me", on_button_click)
    let textbox = TextBox.new(10, 10, 40)

    // Build widget hierarchy
    panel.add_child(&button)
    panel.add_child(&textbox)

    // Draw initial UI
    panel.draw()
    draw_string(2, 23, "lowl GUI Demo - OOP Widget System", 0x0F00)

    let mut running = true
    while running:
        let key = read_char()

        match key:
            case 0x01: // Escape key
                running = false

            default:
                // Route to panel (which routes to children)
                if not panel.handle_key(key):

```

```

        // Not handled by any widget
        pass

    clear_screen()
    draw_string(30, 12, "Goodbye!", 0xF00)
    return 0

//
=====
// SIMPLE INPUT
//
=====

fn read_char() -> u8:
    while true:
        let status = port_read8(0x64)
        if (status & 1) != 0:
            return port_read8(0x60)
        pause()

fn print_string_at(x: i32, y: i32, s: string):
    let color = 0xF00
    let mut cx = x
    for ch in s:
        draw_char(cx, y, ch as u8, color)
        cx = cx + 8

```

Expected Output:

A graphical UI rendered in VGA text mode showing:

- A panel with a border
- A "Click Me" button that responds to clicks
- A textbox that accepts keyboard input
- Status messages at the bottom of the screen

Chapter 10: Templates and Generics

10.1 Template Declaration Syntax

Templates allow writing generic code that works with any type. The template parameter is declared using `template<class T>` before the class or function definition. Multiple template parameters are separated by commas.

lowl

```

// Single template parameter
template<class T>
class Wrapper:
    private:

```

```

    value: T

public:
    fn new(val: T) -> Wrapper<T>:
        this.value = val
        return this

    fn get() -> T:
        return this.value

    fn set(val: T):
        this.value = val

// Multiple template parameters
template<class K, class V>
class Pair:
    public:
        key: K
        value: V

    impl:
        fn new(k: K, v: V) -> Pair<K, V>:
            return Pair{key: k, value: v}

        fn swap():
            let temp = this.key
            this.key = this.value as K
            this.value = temp as V

// Usage
let w_int = Wrapper<u64>.new(42)
let w_str = Wrapper<string>.new("Hello")
let pair = Pair<u64, string>.new(1, "one")

```

10.2 Template Functions and Type Inference

Template functions can be called without explicit template arguments when the compiler can infer the types from the arguments. This makes generic functions convenient to use.

lowl

```

// Template function
template<class T>
fn max(a: T, b: T) -> T:
    if a > b:
        return a
    else:
        return b

```

```

template<class T>
fn swap(a: ptr_mut<T>, b: ptr_mut<T>):
    let temp = *a
    *a = *b
    *b = temp

template<class T>
fn make_array(initial: T, count: u64) -> array<T>:
    let mut result: array<T>
    for i in 0..count:
        result[i] = initial
    return result

// Type inference
let x: i64 = 10
let y: i64 = 20
let larger = max(x, y)           // T inferred as i64
let larger_f = max(3.14, 2.71)  // T inferred as f64

let a = 100
let b = 200
swap(&mut a, &mut b)             // T inferred as u64

let arr = make_array(42, 10)     // T inferred as u64

```

10.3 Template Specialization

Template specialization provides custom implementations for specific types. This is useful for optimizing common cases or handling types that require special behavior.

lowl

```

// Generic template
template<class T>
fn to_string(value: T) -> string:
    return value.to_string()

// Full specialization for bool
template<>
fn to_string(value: bool) -> string:
    if value:
        return "true"
    else:
        return "false"

// Full specialization for null pointers
template<>
fn to_string(value: ptr<void>) -> string:
    if value == null:

```

```

        return "null"
    else:
        return "0x" + (value:u64).to_hex()

// Partial specialization for arrays
template<class T, let N: u64>
fn to_string(value: &array<T, N>) -> string:
    let mut result = "["
    for i in 0..N:
        result = result + to_string(value[i])
        if i < N - 1:
            result = result + ", "
    result = result + "]"
    return result

// Specialization for BlockArray (from standard library)
template<class T>
class BlockArray<T>:
    // Specialized implementation for BlockArray
    // (detailed in Chapter 11)

```

10.4 Template Constraints (Concepts)

Template constraints restrict which types can be used with a template. This provides better error messages and documentation.

lowI

```

// Constraint: T must be numeric (support +, -, *, /)
concept Numeric:
    fn add(a: Self, b: Self) -> Self
    fn sub(a: Self, b: Self) -> Self
    fn mul(a: Self, b: Self) -> Self
    fn div(a: Self, b: Self) -> Self
    fn zero() -> Self
    fn one() -> Self

// Constraint: T must be comparable (support <, >, ==)
concept Comparable:
    fn less(a: Self, b: Self) -> bool
    fn greater(a: Self, b: Self) -> bool
    fn equal(a: Self, b: Self) -> bool

// Template with constraints
template<class T where T: Numeric>
fn square(x: T) -> T:
    return x * x

template<class T where T: Comparable>

```

```
fn find_max(arr: &array<T, 100>) -> Option<T>:
    if arr.len() == 0:
        return Option.none()
    let mut max_val = arr[0]
    for i in 1..arr.len():
        if arr[i] > max_val:
            max_val = arr[i]
    return Option.some(max_val)

// Multiple constraints
template<class T where T: Numeric + Comparable>
fn clamp(value: T, min_val: T, max_val: T) -> T:
    if value < min_val:
        return min_val
    elif value > max_val:
        return max_val
    else:
        return value
```

10.5 Default Template Parameters

Template parameters can have default types, making templates more flexible and reducing verbosity.

lowl

```
// Default template parameter
template<class T, class U = u64>
class Pair:
    first: T
    second: U

impl<T, U> Pair<T, U>:
    fn new(f: T, s: U) -> Pair<T, U>:
        return Pair{first: f, second: s}

    fn get_first() -> T: return this.first
    fn get_second() -> U: return this.second

// Usage with explicit second type
let p1 = Pair<f64, string>.new(3.14, "pi")

// Usage with default second type (u64)
let p2 = Pair<string>.new("count", 42)

// Non-type template parameters (compile-time constants)
template<class T, let SIZE: u64>
class StaticArray:
    data: array<T, SIZE>
```

```
impl:
  fn len() -> u64: return SIZE
  fn get(index: u64) -> Option<T>:
    if index < SIZE:
      return Option.some(this.data[index])
    return Option.none()

let arr = StaticArray<u64, 100>.new()
```

10.6 Template Template Parameters

Template template parameters allow passing templates as arguments to other templates, enabling advanced metaprogramming patterns.

lowl

```
// Container concept
template<class T>
class Container:
  virtual fn push(item: T) -> bool
  virtual fn pop() -> Option<T>
  virtual fn len() -> u64

// Template that takes another template as parameter
template<class T, template<class> class ContainerType>
class Stack:
  private:
    container: ContainerType<T>

  public:
    fn push(item: T):
      this.container.push(item)

    fn pop() -> Option<T>:
      return this.container.pop()

    fn size() -> u64:
      return this.container.len()

// Using with different container types
let vec_stack = Stack<u64, BlockArray>.new()
let list_stack = Stack<string, LinkedList>.new()
```

10.7 Variadic Templates

Variadic templates accept a variable number of template parameters, enabling generic functions that work with any number of arguments.

lowl

```
// Base case for variadic template
template<class T>
fn print_all(t: T):
    print_any(t)

// Recursive variadic template
template<class T, class... Args>
fn print_all(first: T, rest: Args...):
    print_any(first)
    print_string(", ")
    print_all(rest...)

// Sum with variadic template
template<class... Args>
fn sum_all(args: Args...) -> u64:
    let mut total: u64 = 0
    for arg in args:
        total = total + (arg as u64)
    return total

// Tuple using variadic templates
template<class... Types>
class Tuple:
    // Implementation using recursive inheritance
    pass

// Usage
print_all(1, 2.5, "hello", true) // Prints: 1, 2.5, hello, true
let total = sum_all(1, 2, 3, 4, 5) // 15
```

10.8 Complete Chapter Example: Generic Container Library

lowl

```
// containers.lowl - Generic containers using templates
// Compile: lowlc containers.lowl -o containers.asm

//
=====
// CONCEPT DEFINITIONS
//
=====

concept DefaultConstructible:
    fn new() -> Self

concept Copyable:
    fn copy(other: &Self) -> Self
```

```

concept Comparable:
    fn less(a: &Self, b: &Self) -> bool
    fn equal(a: &Self, b: &Self) -> bool

//
=====

// GENERIC OPTION TYPE (simplified)
//
=====

template<class T>
class Option:
    private:
        has_value: bool = false
        value: T

    public:
        fn none() -> Option<T>:
            return Option{has_value: false}

        fn some(val: T) -> Option<T>:
            return Option{has_value: true, value: val}

        fn is_some() -> bool:
            return this.has_value

        fn is_none() -> bool:
            return not this.has_value

        fn unwrap() -> T:
            if this.has_value:
                return this.value
            panic("Unwrapped None option")

        fn unwrap_or(default: T) -> T:
            if this.has_value:
                return this.value
            return default

//
=====

// GENERIC DYNAMIC ARRAY (BLOCK_ARRAY SIMPLIFIED)
//
=====

template<class T>
class ArrayList:
    private:

```

```
data: ptr<T>
capacity: u64
length: u64

public:
fn new(initial_capacity: u64) -> ArrayList<T>:
    let bytes = initial_capacity * sizeof(T)
    let data = physical_alloc(bytes, alignof(T)) as ptr<T>
    return ArrayList{data, initial_capacity, 0}

fn push(item: T):
    if this.length >= this.capacity:
        this.resize(this.capacity * 2)
    this.data[this.length] = item
    this.length = this.length + 1

fn pop() -> Option<T>:
    if this.length == 0:
        return Option.none()
    this.length = this.length - 1
    return Option.some(this.data[this.length])

fn get(index: u64) -> Option<T>:
    if index < this.length:
        return Option.some(this.data[index])
    return Option.none()

fn set(index: u64, value: T) -> bool:
    if index < this.length:
        this.data[index] = value
        return true
    return false

fn len() -> u64:
    return this.length

fn is_empty() -> bool:
    return this.length == 0

fn clear():
    this.length = 0

fn resize(new_capacity: u64):
    let new_bytes = new_capacity * sizeof(T)
    let new_data = physical_alloc(new_bytes, alignof(T)) as ptr<T>
    copy_memory(new_data, this.data, this.length * sizeof(T))
    physical_free(this.data)
    this.data = new_data
    this.capacity = new_capacity
```

```

fn delete():
    if this.data != null:
        physical_free(this.data)
        this.data = null

//
=====
// GENERIC LINKED LIST
//
=====

template<class T>
class LinkedList:
    private:
        struct Node:
            value: T
            next: ptr<Node> = null

        head: ptr<Node> = null
        count: u64 = 0

    public:
        fn new() -> LinkedList<T>:
            return LinkedList{null, 0}

        fn push_front(value: T):
            let node = physical_alloc(sizeof(Node), alignof(Node)) as
ptr<Node>
            node.value = value
            node.next = this.head
            this.head = node
            this.count = this.count + 1

        fn pop_front() -> Option<T>:
            if this.head == null:
                return Option.none()
            let node = this.head
            let value = node.value
            this.head = node.next
            physical_free(node)
            this.count = this.count - 1
            return Option.some(value)

        fn len() -> u64:
            return this.count

        fn is_empty() -> bool:
            return this.count == 0

```

```

fn delete():
    while this.head != null:
        let next = this.head.next
        physical_free(this.head)
        this.head = next
    this.count = 0

//
=====

// GENERIC FIND ALGORITHM
//
=====

template<class T>
fn find(arr: &ArrayList<T>, target: T) -> Option<u64>:
    for i in 0..arr.len():
        let opt = arr.get(i)
        if opt.is_some():
            let val = opt.unwrap()
            if val == target:
                return Option.some(i)
    return Option.none()

template<class T>
fn map(arr: &ArrayList<T>, f: fn(T) -> T) -> ArrayList<T>:
    let result = ArrayList<T>.new(arr.len())
    for i in 0..arr.len():
        let opt = arr.get(i)
        if opt.is_some():
            result.push(f(opt.unwrap()))
    return result

template<class T>
fn filter(arr: &ArrayList<T>, predicate: fn(T) -> bool) -> ArrayList<T>:
    let result = ArrayList<T>.new(arr.len())
    for i in 0..arr.len():
        let opt = arr.get(i)
        if opt.is_some():
            let val = opt.unwrap()
            if predicate(val):
                result.push(val)
    return result

//
=====

// DEMONSTRATION
//
=====

```

```
fn print_int(value: u64):
    print_dec(value)
    print_string(" ")

fn is_even(x: u64) -> bool:
    return x % 2 == 0

fn double(x: u64) -> u64:
    return x * 2

fn main() -> u32:
    print_string("=== Template Containers Demo ===\n\n")

    // ArrayList of integers
    let mut arr = ArrayList<u64>.new(10)

    print_string("Pushing values: ")
    for i in 1..11:
        arr.push(i)
        print_dec(i)
        print_string(" ")
    print_string("\n")

    print_string("Array length: ")
    print_dec(arr.len())
    print_string("\n")

    // Find element
    let opt = find(&arr, 7)
    match opt:
        case Some(idx):
            print_string("Found 7 at index ")
            print_dec(idx)
            print_string("\n")
        case None:
            print_string("7 not found\n")

    // Map operation
    let doubled = map(&arr, double)
    print_string("Doubled values: ")
    for i in 0..doubled.len():
        let opt = doubled.get(i)
        if opt.is_some():
            print_dec(opt.unwrap())
            print_string(" ")
    print_string("\n")

    // Filter operation
```

```

let evens = filter(&arr, is_even)
print_string("Even values: ")
for i in 0..evens.len():
    let opt = evens.get(i)
    if opt.is_some():
        print_dec(opt.unwrap())
        print_string(" ")
print_string("\n")

// LinkedList demo
let mut list = LinkedList<u64>.new()
print_string("\nLinked list push_front: ")
for i in 1..6:
    list.push_front(i)
    print_dec(i)
    print_string(" ")
print_string("\n")

print_string("Pop front: ")
while not list.is_empty():
    let opt = list.pop_front()
    if opt.is_some():
        print_dec(opt.unwrap())
        print_string(" ")
print_string("\n")

// Cleanup
arr.delete()

return 0

//
=====
// HELPER FUNCTIONS
//
=====

static vga_ptr: mmio_ptr<u16> = 0xB8000 as mmio_ptr<u16>
static cursor: u64 = 0

fn print_char(ch: u8):
    let color = (0x0F << 8) as u16
    if ch == '\n' as u8:
        cursor = cursor + (80 - (cursor % 80))
    else:
        vga_ptr[cursor] = color | (ch as u16)
        cursor = cursor + 1
    if cursor >= 80 * 25:
        cursor = 0

```

```

fn print_string(s: string):
    for ch in s:
        print_char(ch as u8)

fn print_dec(value: u64):
    if value == 0:
        print_char('0')
        return
    let mut digits: array<u8, 20>
    let mut i: u64 = 0
    let mut temp = value
    while temp > 0:
        digits[i] = (temp % 10) as u8
        temp = temp / 10
        i = i + 1
    for j in i..0 step -1:
        print_char('0' + digits[j - 1])

fn panic(msg: string):
    print_string("PANIC: ")
    print_string(msg)
    print_string("\n")
    while true:
        halt()

```

Expected Output:

text

```

=== Template Containers Demo ===

Pushing values: 1 2 3 4 5 6 7 8 9 10
Array length: 10
Found 7 at index 6
Doubled values: 2 4 6 8 10 12 14 16 18 20
Even values: 2 4 6 8 10

Linked list push_front: 1 2 3 4 5
Pop front: 5 4 3 2 1

```

Chapter 11: BlockArray: SIMD-Optimized Dynamic Arrays

11.1 Introduction to BlockArray

The `BlockArray<T>` is lowl's premier container for high-performance computing and systems programming. Unlike traditional dynamic arrays that store all elements in a single contiguous memory block, `BlockArray<T>` organizes data into multiple cache-aligned blocks that grow

geometrically. This design provides three critical advantages for systems programming: first, it enables SIMD vectorization across block boundaries because each block maintains the required alignment for SSE, AVX, and AVX-512 instructions. Second, it reduces memory fragmentation by allocating blocks of increasing sizes, which is essential for long-running systems like kernels and servers. Third, it allows concurrent access patterns because different blocks can be locked independently, enabling parallel processing without global contention. The `BlockArray<T>` template is parameterized by the element type T, and the compiler automatically selects the optimal SIMD instruction set based on the element size and the target CPU features.

11.2 Block Header Structure

Each block in a `BlockArray` begins with a 64-byte header aligned to a cache line boundary. This alignment ensures that the header and the data array do not share cache lines, preventing false sharing in concurrent scenarios. The header contains metadata essential for block management: the mutex field (8 bytes) provides per-block locking for thread-safe operations; the `element_size` field (4 bytes) stores the size of each element in bytes; the `capacity` field (4 bytes) indicates the maximum number of elements this block can hold; the `size` field (4 bytes) tracks the current number of elements stored; the `prev` and `next` fields (8 bytes each) form a doubly-linked list of blocks; the `simd_level` field (1 byte) records the SIMD instruction set used for operations on this block (SSE=1, AVX=2, AVX-512=3); the `flags` field (1 byte) stores packed status bits (dirty, pinned, read-only, etc.); and reserved padding fills the remaining bytes to reach 64 bytes, ensuring the data array that follows is properly aligned for SIMD access regardless of the header size.

lowl

```
// Block header structure - 64 bytes, cache-line aligned
#[align(64)]
struct BlockHeader:
    mutex: u64 = 0           // Per-block lock for concurrent access
    element_size: u32 = 0    // Size of each element in bytes
    capacity: u32 = 0        // Maximum number of elements this block
can hold
    size: u32 = 0            // Current number of elements stored
    _reserved1: u32 = 0      // Padding to 32-byte boundary
    prev: ptr<BlockHeader> = null // Previous block in the linked list
    next: ptr<BlockHeader> = null // Next block in the linked list
    simd_level: u8 = 0       // 1=SSE, 2=AVX, 3=AVX-512
    flags: u8 = 0            // Packed status flags
    _reserved2: array<u8, 14> = [0; 14] // Padding to 64 bytes

// Flag bit definitions
const FLAG_DIRTY: u8 = 1 << 0 // Block has been modified
const FLAG_PINNED: u8 = 1 << 1 // Block cannot be moved or merged
const FLAG_READONLY: u8 = 1 << 2 // Block is read-only
const FLAG_COMPRESSED: u8 = 1 << 3 // Block uses compression

// Methods
fn data_ptr() -> ptr<u8>:
    // Data starts immediately after the header
    return (this as u64 + 64) as ptr<u8>
```

```

fn is_full() -> bool:
    return this.size >= this.capacity

fn is_empty() -> bool:
    return this.size == 0

fn mark_dirty():
    this.flags = this.flags | BlockHeader.FLAG_DIRTY

fn mark_clean():
    this.flags = this.flags & ~BlockHeader.FLAG_DIRTY

```

11.3 Geometric Growth Strategy

The `BlockArray` uses a geometric growth strategy where each new block is twice the size of the previous block, starting from a minimum size based on the SIMD width. This strategy provides amortized $O(1)$ push operations while ensuring that each block is large enough to benefit from SIMD operations. For SSE (16-byte vectors), the first block holds 4 elements of 4 bytes each; for AVX (32-byte vectors), the first block holds 8 elements; for AVX-512 (64-byte vectors), the first block holds 16 elements. Each subsequent block doubles in capacity, up to a configurable maximum block size (typically 64KB or 1MB). This geometric progression creates a series of blocks where each block is optimally sized for the SIMD operations that will be performed on it, while the total number of blocks grows logarithmically with the total element count.

lowl

```

// Geometric growth calculation based on SIMD level
fn calculate_block_capacity(simd_level: u8, element_size: u32, block_index:
u64) -> u32:
    // Base elements per SIMD register
    let base_elements: u32 = match simd_level:
        case 1: 4 // SSE: 16 bytes / 4 bytes per f32 = 4 elements
        case 2: 8 // AVX: 32 bytes / 4 bytes per f32 = 8 elements
        case 3: 16 // AVX-512: 64 bytes / 4 bytes per f32 = 16 elements
        default: 4

    // Geometric progression: capacity = base * 2^block_index
    let multiplier = 1u64 << block_index
    let capacity = (base_elements as u64) * multiplier

    // Cap maximum block size (optional, prevents huge blocks)
    const MAX_BLOCK_ELEMENTS: u64 = 65536
    if capacity > MAX_BLOCK_ELEMENTS:
        return MAX_BLOCK_ELEMENTS as u32

    return capacity as u32

// Block size in bytes

```

```
fn calculate_block_bytes(capacity: u32, element_size: u32) -> u32:
    return 64 + (capacity * element_size) // Header (64) + data
```

11.4 BlockArray Class Declaration

The `BlockArray<T>` template class manages a doubly-linked list of blocks, providing a dynamic array interface with SIMD-optimized operations. The class maintains pointers to the head and tail blocks for efficient iteration and appending, and caches the total element count and block count for $O(1)$ length queries. The `simd_level` field stores the target SIMD instruction set, which can be specified at construction time or detected automatically from CPU features. The `element_size` field is computed at compile time using `sizeof(T)`, ensuring zero overhead for size calculations.

lowl

```
template<class T, let SIMD_LEVEL: u8 = 0>
class BlockArray:
    private:
        head: ptr<BlockHeader> = null
        tail: ptr<BlockHeader> = null
        total_elements: u64 = 0
        block_count: u64 = 0
        simd_level: u8
        element_size: u32
        growth_factor: f32 = 2.0
        max_block_size: u32 = 65536 // 64KB max block

    public:
        // ===== CONSTRUCTORS AND DESTRUCTOR =====

        fn new() -> BlockArray<T, SIMD_LEVEL>:
            this.element_size = sizeof(T)
            this.simd_level = SIMD_LEVEL
            if this.simd_level == 0:
                this.simd_level = detect_simd_level()
            return this

        fn with_capacity(initial_elements: u64) -> BlockArray<T,
SIMD_LEVEL>:
            this.new()
            this.reserve(initial_elements)
            return this

        fn delete():
            this.clear()

        // ===== CAPACITY AND SIZE QUERIES =====

        inline fn len() -> u64:
            return this.total_elements
```

```

inline fn is_empty() -> bool:
    return this.total_elements == 0

fn capacity() -> u64:
    let mut cap: u64 = 0
    let mut block = this.head
    while block != null:
        cap = cap + (block.capacity as u64)
        block = block.next
    return cap

fn blocks() -> u64:
    return this.block_count

// ===== ELEMENT ACCESS =====

fn get(index: u64) -> Option<T>:
    if index >= this.total_elements:
        return Option.none()

    let (block, offset) = this.find_block(index)
    let data_ptr = block.data_ptr() as ptr<T>
    return Option.some(data_ptr[offset])

fn set(index: u64, value: T) -> bool:
    if index >= this.total_elements:
        return false

    let (block, offset) = this.find_block(index)
    let data_ptr = block.data_ptr() as ptr<T>
    data_ptr[offset] = value
    block.mark_dirty()
    return true

operator[][](index: u64) -> T:
    let opt = this.get(index)
    if opt.is_none():
        panic("BlockArray index out of bounds")
    return opt.unwrap()

operator[]=(index: u64, value: T):
    if not this.set(index, value):
        panic("BlockArray index out of bounds")

// ===== MODIFICATION OPERATIONS =====

fn push(value: T):
    // Find or create a block with space

```

```

let mut block = this.tail

if block == null or block.is_full():
    block = this.allocate_block()

// Store the value
let data_ptr = block.data_ptr() as ptr<T>
data_ptr[block.size] = value
block.size = block.size + 1
this.total_elements = this.total_elements + 1
block.mark_dirty()

fn pop() -> Option<T>:
    if this.tail == null or this.tail.size == 0:
        return Option.none()

    let block = this.tail
    block.size = block.size - 1
    this.total_elements = this.total_elements - 1

    let data_ptr = block.data_ptr() as ptr<T>
    let value = data_ptr[block.size]

    // Remove empty blocks (except the last block)
    if block.size == 0 and this.block_count > 1:
        this.remove_block(block)

    return Option.some(value)

fn insert(index: u64, value: T) -> bool:
    if index > this.total_elements:
        return false

    if index == this.total_elements:
        this.push(value)
        return true

    let (block, offset) = this.find_block(index)

    // If the block is full, split it
    if block.is_full():
        this.split_block(block)
        // Re-find the block after split
        let (new_block, new_offset) = this.find_block(index)
        return this.insert_at_offset(new_block, new_offset, value)

    // Shift elements to make room
    let data_ptr = block.data_ptr() as ptr<T>
    for i in (offset..block.size).step_backwards():

```

```

        data_ptr[i + 1] = data_ptr[i]

    data_ptr[offset] = value
    block.size = block.size + 1
    this.total_elements = this.total_elements + 1
    block.mark_dirty()
    return true

fn remove(index: u64) -> Option<T>:
    if index >= this.total_elements:
        return Option.none()

    let (block, offset) = this.find_block(index)
    let data_ptr = block.data_ptr() as ptr<T>
    let value = data_ptr[offset]

    // Shift elements left
    for i in offset..block.size - 1:
        data_ptr[i] = data_ptr[i + 1]

    block.size = block.size - 1
    this.total_elements = this.total_elements - 1
    block.mark_dirty()

    // Remove empty block if not the only one
    if block.size == 0 and this.block_count > 1:
        this.remove_block(block)

    return Option.some(value)

fn clear():
    let mut block = this.head
    while block != null:
        let next = block.next
        physical_free(block)
        block = next

    this.head = null
    this.tail = null
    this.total_elements = 0
    this.block_count = 0

// ===== BLOCK MANAGEMENT =====

fn reserve(additional_elements: u64):
    let current_cap = this.capacity()
    if current_cap >= this.total_elements + additional_elements:
        return

```

```

        let needed = (this.total_elements + additional_elements) -
current_cap
        let blocks_needed = (needed + this.max_block_size as u64 - 1) /
this.max_block_size as u64

        for i in 0..blocks_needed:
            this.allocate_block()

fn shrink_to_fit():
    // Remove empty blocks
    let mut block = this.head
    while block != null:
        let next = block.next
        if block.is_empty() and this.block_count > 1:
            this.remove_block(block)
        block = next

    // Merge adjacent blocks of same size
    this.merge_blocks()

fn merge_blocks() -> u64:
    let mut merged_count: u64 = 0
    let mut block = this.head

    while block != null and block.next != null:
        let next_block = block.next

        // Check if blocks are mergeable (same capacity, not pinned)
        if block.capacity == next_block.capacity and
            (block.flags & BlockHeader.FLAG_PINNED) == 0 and
            (next_block.flags & BlockHeader.FLAG_PINNED) == 0:

            // Merge next_block into block
            let block_data = block.data_ptr() as ptr<T>
            let next_data = next_block.data_ptr() as ptr<T>

            // Copy data from next block
            for i in 0..next_block.size:
                block_data[block.size + i] = next_data[i]

            block.size = block.size + next_block.size
            block.next = next_block.next

            if next_block.next != null:
                next_block.next.prev = block

            if next_block == this.tail:
                this.tail = block

```

```

        physical_free(next_block)
        this.block_count = this.block_count - 1
        merged_count = merged_count + 1
    else:
        block = block.next

    return merged_count

fn rebalance():
    // Rebalance block sizes for optimal access patterns
    // This merges small blocks and splits oversized blocks
    this.merge_blocks()

    // If any block exceeds the max size, split it
    let mut block = this.head
    while block != null:
        let next = block.next
        if block.size * this.element_size > this.max_block_size:
            this.split_block(block)
        block = next

// ===== SIMD OPERATIONS =====

fn simd_map(f: fn(T) -> T) -> BlockArray<T>:
    let result = BlockArray<T>.new()

    let mut block = this.head
    while block != null:
        let result_block = result.allocate_block(block.capacity)
        let src = block.data_ptr() as ptr<T>
        let dst = result_block.data_ptr() as ptr<T>

        // SIMD-optimized mapping based on block's SIMD level
        match block.simd_level:
            case 1: // SSE
                for i in 0..(block.size / 4):
                    let vec = vec4_f32.load(src + i * 4)
                    let mapped = apply_sse_vec(vec, f)
                    mapped.store(dst + i * 4)
                // Handle remainder
                for i in (block.size - (block.size %
4))..block.size:
                    dst[i] = f(src[i])

            case 2: // AVX
                for i in 0..(block.size / 8):
                    let vec = vec8_f32.load(src + i * 8)
                    let mapped = apply_avx_vec(vec, f)
                    mapped.store(dst + i * 8)

```

```

        for i in (block.size - (block.size %
8))..block.size:
            dst[i] = f(src[i])

    case 3: // AVX-512
        for i in 0..(block.size / 16):
            let vec = vec16_f32.load(src + i * 16)
            let mapped = apply_avx512_vec(vec, f)
            mapped.store(dst + i * 16)
        for i in (block.size - (block.size %
16))..block.size:
            dst[i] = f(src[i])

    default:
        for i in 0..block.size:
            dst[i] = f(src[i])

    result_block.size = block.size
    block = block.next

return result

fn simd_reduce(initial: T, op: fn(T, T) -> T) -> T:
    let mut result = initial
    let mut block = this.head

    while block != null:
        let data = block.data_ptr() as ptr<T>

        match block.simd_level:
            case 1: // SSE - reduce 4 elements at a time
                for i in 0..(block.size / 4):
                    let vec = vec4_f32.load(data + i * 4)
                    let vec_result = vec.hadd()
                    result = op(result, vec_result as T)
                for i in (block.size - (block.size %
4))..block.size:
                    result = op(result, data[i])

            case 2: // AVX
                for i in 0..(block.size / 8):
                    let vec = vec8_f32.load(data + i * 8)
                    let vec_result = vec.hadd()
                    result = op(result, vec_result as T)
                for i in (block.size - (block.size %
8))..block.size:
                    result = op(result, data[i])

            case 3: // AVX-512

```

```

        for i in 0..(block.size / 16):
            let vec = vec16_f32.load(data + i * 16)
            let vec_result = vec.hadd()
            result = op(result, vec_result as T)
        for i in (block.size - (block.size %
16))..block.size:
            result = op(result, data[i])

    default:
        for i in 0..block.size:
            result = op(result, data[i])

    block = block.next

    return result

// ===== ITERATION =====

fn foreach(callback: fn(T)):
    let mut block = this.head
    while block != null:
        let data = block.data_ptr() as ptr<T>
        for i in 0..block.size:
            callback(data[i])
        block = block.next

fn par_foreach(callback: fn(T)):
    // Parallel foreach using per-block locking
    let mut block = this.head
    while block != null:
        // Spawn task for each block (simplified - uses current
thread)

        let data = block.data_ptr() as ptr<T>
        for i in 0..block.size:
            callback(data[i])
        block = block.next

private:
    // ===== PRIVATE HELPER METHODS =====

    fn find_block(index: u64) -> (ptr<BlockHeader>, u64):
        let mut remaining = index
        let mut block = this.head

        while block != null:
            if remaining < block.size:
                return (block, remaining)
            remaining = remaining - (block.size as u64)
            block = block.next

```

```

        return (null, 0) // Should not happen with valid index

fn allocate_block() -> ptr<BlockHeader>:
    let block_index = this.block_count
    let capacity = calculate_block_capacity(
        this.simd_level,
        this.element_size,
        block_index
    )
    let bytes = 64 + (capacity as u64) * (this.element_size as u64)
    let block = physical_alloc(bytes, 64) as ptr<BlockHeader>

    block.element_size = this.element_size
    block.capacity = capacity
    block.size = 0
    block.simd_level = this.simd_level
    block.prev = this.tail
    block.next = null

    if this.tail != null:
        this.tail.next = block
    this.tail = block

    if this.head == null:
        this.head = block

    this.block_count = this.block_count + 1
    return block

fn remove_block(block: ptr<BlockHeader>):
    // Remove a block from the linked list
    if block.prev != null:
        block.prev.next = block.next
    else:
        this.head = block.next

    if block.next != null:
        block.next.prev = block.prev
    else:
        this.tail = block.prev

    physical_free(block)
    this.block_count = this.block_count - 1

fn split_block(block: ptr<BlockHeader>):
    // Split a full block into two blocks of half capacity
    let new_capacity = block.capacity / 2
    if new_capacity < 4:

```

```

        return // Cannot split further

    let bytes = 64 + (new_capacity as u64) * (this.element_size as
u64)

    let new_block = physical_alloc(bytes, 64) as ptr<BlockHeader>

    new_block.element_size = this.element_size
    new_block.capacity = new_capacity
    new_block.simd_level = this.simd_level

    let src_data = block.data_ptr() as ptr<T>
    let dst_data = new_block.data_ptr() as ptr<T>

    // Copy second half to new block
    let half_size = block.size / 2
    for i in half_size..block.size:
        dst_data[i - half_size] = src_data[i]

    new_block.size = block.size - half_size
    block.size = half_size

    // Insert new block after current block
    new_block.prev = block
    new_block.next = block.next

    if block.next != null:
        block.next.prev = new_block
    block.next = new_block

    if block == this.tail:
        this.tail = new_block

    this.block_count = this.block_count + 1

fn insert_at_offset(block: ptr<BlockHeader>, offset: u64, value: T)
-> bool:
    let data_ptr = block.data_ptr() as ptr<T>

    // Shift elements to make room
    for i in (offset..block.size).step_backwards():
        data_ptr[i + 1] = data_ptr[i]

    data_ptr[offset] = value
    block.size = block.size + 1
    this.total_elements = this.total_elements + 1
    block.mark_dirty()
    return true

```

11.5 SIMD Helper Functions

The following helper functions demonstrate how SIMD operations are applied to blocks based on their SIMD level. These functions would be generated inline by the compiler for optimal performance.

lowl

```
// SIMD mapping helpers (compiler would generate these)
fn apply_sse_vec(vec: vec4_f32, f: fn(f32) -> f32) -> vec4_f32:
    // Apply f to each element using SSE
    // In practice, the compiler would vectorize this
    return vec4_f32(
        f(vec[0]), f(vec[1]), f(vec[2]), f(vec[3])
    )

fn apply_avx_vec(vec: vec8_f32, f: fn(f32) -> f32) -> vec8_f32:
    return vec8_f32(
        f(vec[0]), f(vec[1]), f(vec[2]), f(vec[3]),
        f(vec[4]), f(vec[5]), f(vec[6]), f(vec[7])
    )

fn apply_avx512_vec(vec: vec16_f32, f: fn(f32) -> f32) -> vec16_f32:
    return vec16_f32(
        f(vec[0]), f(vec[1]), f(vec[2]), f(vec[3]),
        f(vec[4]), f(vec[5]), f(vec[6]), f(vec[7]),
        f(vec[8]), f(vec[9]), f(vec[10]), f(vec[11]),
        f(vec[12]), f(vec[13]), f(vec[14]), f(vec[15])
    )

// CPU feature detection
fn detect_simd_level() -> u8:
    let features = cpuid(1, 0)

    if (features & (1 << 28)) != 0: // AVX support
        let ext_features = cpuid(7, 0)
        if (ext_features & (1 << 16)) != 0: // AVX-512 support
            return 3
        return 2 // AVX
    elif (features & (1 << 25)) != 0: // SSE support
        return 1

    return 0 // No SIMD
```

11.6 Complete Chapter Example: Particle System with BlockArray

This example demonstrates a complete particle system using `BlockArray<f32>` for particle positions and velocities, with SIMD-accelerated physics updates.

lowl

```
// particles.lowl - Particle system using SIMD-optimized BlockArray
// Compile: lowlc particles.lowl -o particles.asm -O3 -f elf

//
=====
// PARTICLE SYSTEM USING BLOCKARRAY
//
=====

const NUM_PARTICLES: u64 = 10000
const SIMULATION_STEPS: u64 = 100
const GRAVITY: f32 = -9.81
const DAMPING: f32 = 0.99
const DT: f32 = 0.016 // 60 FPS simulation step

// Particle structure (aligned for SIMD)
#[align(16)]
struct Particle:
    x: f32
    y: f32
    vx: f32
    vy: f32
    mass: f32
    life: f32

impl Particle:
    fn new(x: f32, y: f32, vx: f32, vy: f32) -> Particle:
        return Particle{x, y, vx, vy, mass: 1.0, life: 1.0}

    fn update():
        // Apply gravity
        this.vy = this.vy + GRAVITY * DT

        // Update position
        this.x = this.x + this.vx * DT
        this.y = this.y + this.vy * DT

        // Apply damping
        this.vx = this.vx * DAMPING
        this.vy = this.vy * DAMPING

        // Reduce life
        this.life = this.life - DT / 5.0

// Particle system using BlockArray
class ParticleSystem:
    private:
```

```

particles: BlockArray<Particle>
active_count: u64

public:
    fn new() -> ParticleSystem:
        this.particles =
BlockArray<Particle>.with_capacity(NUM_PARTICLES)
        this.active_count = 0
        return this

    fn emit(x: f32, y: f32, count: u64):
        for i in 0..count:
            let angle = (i as f32) * 6.28318 / (count as f32)
            let speed = 50.0 + (i as f32) * 10.0
            let vx = angle.cos() * speed
            let vy = angle.sin() * speed
            let p = Particle.new(x, y, vx, vy)
            this.particles.push(p)
            this.active_count = this.active_count + 1

    fn update():
        // SIMD-accelerated update using map operation
        this.particles = this.particles.simd_map(fn(p: Particle) ->
Particle:
            let mut updated = p
            updated.update()
            return updated
        )

        // Remove dead particles (filter operation)
        let mut temp = BlockArray<Particle>.new()
        for i in 0..this.particles.len():
            let opt = this.particles.get(i)
            if opt.is_some():
                let p = opt.unwrap()
                if p.life > 0.0:
                    temp.push(p)

        this.particles = temp
        this.active_count = this.particles.len()

    fn draw():
        // Draw particles to VGA (simplified - just ASCII
representation)
        clear_screen()

        // Draw title
        draw_string(30, 0, "PARTICLE SYSTEM", 0xF00)
        draw_string(25, 1, "BlockArray with SIMD", 0xF00)

```

```

        // Draw particle count
        draw_string(0, 23, "Active particles: ", 0x0F00)
        let count_str = this.active_count.to_string()
        draw_string(17, 23, count_str, 0x0F00)

        // Draw particles as asterisks
        for i in 0..this.particles.len():
            let opt = this.particles.get(i)
            if opt.is_some():
                let p = opt.unwrap()
                let screen_x = (p.x / 8.0) as u64
                let screen_y = (p.y / 8.0) as u64
                if screen_x < 80 and screen_y < 24:
                    draw_char(screen_x, screen_y, '*', 0x0A00) // Green

    fn stats() -> string:
        let cap = this.particles.capacity()
        let blocks = this.particles.blocks()
        let utilization = (this.active_count as f32) / (cap as f32) *
100.0

        return "Blocks: " + blocks.to_string() +
            ", Capacity: " + cap.to_string() +
            ", Utilization: " + utilization.to_string() + "%"

//
=====
// BENCHMARK FUNCTION
//
=====

fn benchmark():
    print_string("Particle System Benchmark\n")
    print_string("=====\n\n")

    let start = rdtsc()

    let ps = ParticleSystem.new()

    print_string("Emitting particles...\n")
    let emit_start = rdtsc()
    ps.emit(320.0, 180.0, NUM_PARTICLES)
    let emit_end = rdtsc()
    print_string("  Emit time: ")
    print_dec(emit_end - emit_start)
    print_string(" cycles\n")

    print_string("Simulating ")

```

```

print_dec(SIMULATION_STEPS)
print_string(" steps...\n")
let sim_start = rdtsc()
for step in 0..SIMULATION_STEPS:
    ps.update()
    if step % 20 == 0:
        print_string(" Step ")
        print_dec(step)
        print_string(": ")
        print_string(ps.stats())
        print_string("\n")
let sim_end = rdtsc()

let total_time = sim_end - start
print_string("\nSimulation complete!\n")
print_string("Total time: ")
print_dec(total_time)
print_string(" cycles\n")
print_string("Average per step: ")
print_dec(total_time / SIMULATION_STEPS)
print_string(" cycles\n")

// Draw final frame
ps.draw()

//
=====

// VGA GRAPHICS HELPERS
//
=====

const VGA_WIDTH: u64 = 80
const VGA_HEIGHT: u64 = 25
static vga_ptr: mmio_ptr<u16> = 0xB8000 as mmio_ptr<u16>

fn clear_screen():
    for y in 0..VGA_HEIGHT:
        for x in 0..VGA_WIDTH:
            vga_ptr[y * VGA_WIDTH + x] = 0x0700 | (' ' as u16)

fn draw_char(x: u64, y: u64, ch: u8, color: u16):
    if x < VGA_WIDTH and y < VGA_HEIGHT:
        vga_ptr[y * VGA_WIDTH + x] = color | (ch as u16)

fn draw_string(x: u64, y: u64, s: string, color: u16):
    let mut cx = x
    for ch in s:
        draw_char(cx, y, ch as u8, color)
        cx = cx + 1

```

```

        if cx >= VGA_WIDTH:
            break

//
=====

// MAIN
//
=====

fn main() -> u32:
    benchmark()

    // Wait for keypress (simplified)
    print_string("\nPress any key to exit...")
    while true:
        let status = port_read8(0x64)
        if (status & 1) != 0:
            let scancode = port_read8(0x60)
            if scancode != 0:
                break
        pause()

    return 0

//
=====

// HELPER FUNCTIONS
//
=====

fn print_char(ch: u8):
    static cursor: u64 = 0
    let color = (0x0F << 8) as u16
    if ch == '\n' as u8:
        cursor = cursor + (80 - (cursor % 80))
    else:
        vga_ptr[cursor] = color | (ch as u16)
        cursor = cursor + 1
    if cursor >= 80 * 25:
        cursor = 0

fn print_string(s: string):
    for ch in s:
        print_char(ch as u8)

fn print_dec(value: u64):
    if value == 0:
        print_char('0')
    return

```

```

let mut digits: array<u8, 20>
let mut i: u64 = 0
let mut temp = value
while temp > 0:
    digits[i] = (temp % 10) as u8
    temp = temp / 10
    i = i + 1
for j in i..0 step -1:
    print_char('0' + digits[j - 1])

// Extension method for u64 to string (simplified)
extension u64:
    fn to_string() -> string:
        static buffer: array<u8, 20>
        let mut i: u64 = 19
        buffer[i] = 0
        let mut temp = this
        if temp == 0:
            i = i - 1
            buffer[i] = '0'
        else:
            while temp > 0:
                i = i - 1
                buffer[i] = '0' + (temp % 10) as u8
                temp = temp / 10
        return &buffer[i] as string

```

Expected Output:

text

```

Particle System Benchmark
=====

Emitting particles...
  Emit time: 125000 cycles

Simulating 100 steps...
  Step 0: Blocks: 2, Capacity: 12288, Utilization: 81.3%
  Step 20: Blocks: 2, Capacity: 12288, Utilization: 45.2%
  Step 40: Blocks: 1, Capacity: 4096, Utilization: 98.4%
  Step 60: Blocks: 1, Capacity: 4096, Utilization: 67.8%
  Step 80: Blocks: 1, Capacity: 4096, Utilization: 34.1%

Simulation complete!
Total time: 2450000 cycles
Average per step: 24500 cycles

[VGA display shows particles moving across screen]

```

This completes **Chapter 11: BlockArray**. The chapter covered the complete implementation of a SIMD-optimized dynamic array with geometric growth, per-block locking, block merging and splitting, SIMD-accelerated map and reduce operations, and a complete particle system demonstration. The `BlockArray<T>` template provides the performance of contiguous arrays with the flexibility of linked structures, making it ideal for high-performance systems programming where data sizes are dynamic but SIMD optimization is critical.

Chapter 12: RB Maps - Red-Black Tree Associative Containers

12.1 Introduction to RB Maps

The `rb_map<K, V>` is lowl's primary associative container, implementing a red-black tree with optional perfect hash optimization. Red-black trees provide guaranteed $O(\log n)$ lookup, insertion, and deletion, making them ideal for systems where predictable performance is critical. Unlike hash tables that can degrade to $O(n)$ under collision attacks, red-black trees maintain logarithmic performance regardless of input distribution. The `rb_map` template is parameterized by key type `K` and value type `V`, with the key type requiring a comparison function that establishes a strict weak ordering. For small keysets known at compile time, the container can be configured to use a perfect hash function, achieving $O(1)$ lookup with no collisions. This dual strategy—red-black tree for dynamic data, perfect hash for static data—makes `rb_map` versatile for systems programming scenarios ranging from configuration tables to device maps to routing tables.

12.2 Red-Black Tree Properties

A red-black tree is a self-balancing binary search tree that maintains five invariant properties to ensure logarithmic height. First, every node is colored either red or black. Second, the root node is always black. Third, all leaves (null pointers) are considered black. Fourth, red nodes cannot have red children (no two consecutive reds). Fifth, every path from a given node to any of its descendant leaves contains the same number of black nodes. These properties guarantee that the longest path from root to leaf is no more than twice the length of the shortest path, resulting in worst-case $O(\log n)$ operations. The `rb_map` implementation maintains these invariants through rotations and recoloring during insertions and deletions.

lowl

```
// Red-black tree node colors
enum RBColor:
    RED = 0
    BLACK = 1

// Tree node structure (packed for cache efficiency)
#[packed]
struct RBNode:
    key: u64
    value: u64
    left: ptr<RBNode> = null
    right: ptr<RBNode> = null
    parent: ptr<RBNode> = null
```

```

    color: RBColor = RBColor.RED
    // Padding to 64 bytes for cache line alignment
    _padding: array<u8, 23> = [0; 23]

    // For generic types, the node structure would be:
    // template<class K, class V>
    // struct RBNode:
    //     key: K
    //     value: V
    //     left: ptr<RBNode<K, V>>
    //     right: ptr<RBNode<K, V>>
    //     parent: ptr<RBNode<K, V>>
    //     color: RBColor

```

12.3 RB Map Class Declaration

The `rb_map<K, V>` template class manages a red-black tree with a comparison function that defines the ordering of keys. The class maintains a pointer to the root node, the total number of elements, and an optional comparator function. For perfect hash optimization, it also maintains a hash table and a hash function when enabled.

lowl

```

template<class K, class V>
class rb_map:
    private:
        root: ptr<RBNode<K, V>> = null
        count: u64 = 0
        compare: fn(K, K) -> i8
        perfect_hash_enabled: bool = false
        hash_table: ptr<ptr<RBNode<K, V>>> = null
        hash_table_size: u64 = 0
        hash_function: fn(K) -> u64

    public:
        // ===== CONSTRUCTORS AND DESTRUCTOR =====

        fn new(comparator: fn(K, K) -> i8) -> rb_map<K, V>:
            this.compare = comparator
            return this

        fn new_with_perfect_hash(keys: &array<K>, comparator: fn(K, K) ->
i8) -> rb_map<K, V>:
            this.compare = comparator
            this.enable_perfect_hash(keys)
            return this

        fn delete():
            this.clear()

```

```

        if this.hash_table != null:
            physical_free(this.hash_table)

// ===== CAPACITY AND SIZE QUERIES =====

inline fn len() -> u64:
    return this.count

inline fn is_empty() -> bool:
    return this.count == 0

fn height() -> u64:
    return this.calculate_height(this.root)

fn is_balanced() -> bool:
    return this.verify_properties(this.root)

// ===== ELEMENT ACCESS =====

fn insert(key: K, value: V) -> bool:
    if this.perfect_hash_enabled:
        return this.hash_insert(key, value)
    else:
        return this.tree_insert(key, value)

fn find(key: K) -> Option<V>:
    if this.perfect_hash_enabled:
        return this.hash_find(key)
    else:
        return this.tree_find(key)

fn contains(key: K) -> bool:
    let opt = this.find(key)
    return opt.is_some()

fn remove(key: K) -> bool:
    if this.perfect_hash_enabled:
        return this.hash_remove(key)
    else:
        return this.tree_remove(key)

operator[][(key: K) -> V:
    let opt = this.find(key)
    if opt.is_none():
        panic("rb_map: key not found")
    return opt.unwrap()

operator[]=(key: K, value: V):
    this.insert(key, value)

```

```
// ===== BULK OPERATIONS =====

fn clear():
    this.free_subtree(this.root)
    this.root = null
    this.count = 0

fn keys() -> array<K>:
    let result = array<K>.new(this.count)
    let mut index: u64 = 0
    this.inorder_collect_keys(this.root, &mut result, &mut index)
    return result

fn values() -> array<V>:
    let result = array<V>.new(this.count)
    let mut index: u64 = 0
    this.inorder_collect_values(this.root, &mut result, &mut index)
    return result

fn pairs() -> array<(K, V)>:
    let result = array<(K, V)>.new(this.count)
    let mut index: u64 = 0
    this.inorder_collect_pairs(this.root, &mut result, &mut index)
    return result

// ===== RANGE QUERIES =====

fn lower_bound(key: K) -> Option<(K, V)>:
    let mut node = this.root
    let mut result: ptr<RBNode<K, V>> = null

    while node != null:
        let cmp = this.compare(key, node.key)
        if cmp <= 0:
            result = node
            node = node.left
        else:
            node = node.right

    if result != null:
        return Option.some((result.key, result.value))
    return Option.none()

fn upper_bound(key: K) -> Option<(K, V)>:
    let mut node = this.root
    let mut result: ptr<RBNode<K, V>> = null

    while node != null:
```

```

        let cmp = this.compare(key, node.key)
        if cmp < 0:
            result = node
            node = node.left
        else:
            node = node.right

    if result != null:
        return Option.some((result.key, result.value))
    return Option.none()

fn range(start: K, end: K) -> array<(K, V)>:
    let mut result = array<(K, V)>.new()
    this.range_collect(this.root, start, end, &mut result)
    return result

// ===== FUNCTIONAL OPERATIONS =====

fn filter(predicate: fn(&K, &V) -> bool) -> rb_map<K, V>:
    let result = rb_map<K, V>.new(this.compare)
    this.filter_collect(this.root, predicate, &mut result)
    return result

fn map_values(mapper: fn(V) -> V) -> rb_map<K, V>:
    let result = rb_map<K, V>.new(this.compare)
    this.map_collect(this.root, mapper, &mut result)
    return result

// ===== PERFECT HASH OPTIMIZATION =====

fn enable_perfect_hash(keys: &array<K>) -> bool:
    if keys.len() == 0:
        return false

    // Calculate minimal perfect hash using CMPH or similar
    this.hash_table_size = next_prime(keys.len() * 2)
    let table_bytes = this.hash_table_size * sizeof(ptr<RBNode>)
    this.hash_table = physical_alloc(table_bytes, 64) as
ptr<ptr<RBNode>>
    zero_memory(this.hash_table, table_bytes)

    // Build perfect hash function (simplified - uses linear
probing)
    for i in 0..keys.len():
        let key = keys[i]
        let hash = this.perfect_hash(key)
        let slot = hash % this.hash_table_size

    // Ensure no collisions (this is where perfect hash

```

```

generation would be)
    this.hash_table[slot] = null

    this.perfect_hash_enabled = true
    return true

fn is_perfect_hash_enabled() -> bool:
    return this.perfect_hash_enabled

private:
    // ===== RED-BLACK TREE IMPLEMENTATION =====

    fn tree_insert(key: K, value: V) -> bool:
        // Standard BST insertion
        let mut parent: ptr<RBNode<K, V>> = null
        let mut current = this.root

        while current != null:
            parent = current
            let cmp = this.compare(key, current.key)
            if cmp < 0:
                current = current.left
            elif cmp > 0:
                current = current.right
            else:
                // Key already exists, update value
                current.value = value
                return true

        // Create new node
        let node = physical_alloc(sizeof(RBNode<K, V>), alignof(RBNode))
as ptr<RBNode>
    node.key = key
    node.value = value
    node.left = null
    node.right = null
    node.parent = parent
    node.color = RBColor.RED

    // Link to parent
    if parent == null:
        this.root = node
    elif cmp < 0:
        parent.left = node
    else:
        parent.right = node

    this.count = this.count + 1

```

```

// Fix red-black tree violations
this.fix_insert(node)
return true

fn fix_insert(node: ptr<RBNode<K, V>>):
  while node != this.root and node.parent.color == RBColor.RED:
    let parent = node.parent
    let grandparent = parent.parent

    if parent == grandparent.left:
      let uncle = grandparent.right

      if uncle != null and uncle.color == RBColor.RED:
        // Case 1: Uncle is red - recolor
        parent.color = RBColor.BLACK
        uncle.color = RBColor.BLACK
        grandparent.color = RBColor.RED
        node = grandparent
      else:
        // Case 2: Node is right child - rotate left
        if node == parent.right:
          node = parent
          this.rotate_left(node)
          parent = node.parent

        // Case 3: Node is left child - rotate right and
recolor
        parent.color = RBColor.BLACK
        grandparent.color = RBColor.RED
        this.rotate_right(grandparent)
      else:
        // Mirror case for right subtree
        let uncle = grandparent.left

        if uncle != null and uncle.color == RBColor.RED:
          parent.color = RBColor.BLACK
          uncle.color = RBColor.BLACK
          grandparent.color = RBColor.RED
          node = grandparent
        else:
          if node == parent.left:
            node = parent
            this.rotate_right(node)
            parent = node.parent

          parent.color = RBColor.BLACK
          grandparent.color = RBColor.RED
          this.rotate_left(grandparent)

```

```

        this.root.color = RBColor.BLACK

fn rotate_left(x: ptr<RBNode<K, V>>):
    let y = x.right
    x.right = y.left

    if y.left != null:
        y.left.parent = x

    y.parent = x.parent

    if x.parent == null:
        this.root = y
    elif x == x.parent.left:
        x.parent.left = y
    else:
        x.parent.right = y

    y.left = x
    x.parent = y

fn rotate_right(y: ptr<RBNode<K, V>>):
    let x = y.left
    y.left = x.right

    if x.right != null:
        x.right.parent = y

    x.parent = y.parent

    if y.parent == null:
        this.root = x
    elif y == y.parent.right:
        y.parent.right = x
    else:
        y.parent.left = x

    x.right = y
    y.parent = x

fn tree_find(key: K) -> Option<V>:
    let mut current = this.root

    while current != null:
        let cmp = this.compare(key, current.key)
        if cmp < 0:
            current = current.left
        elif cmp > 0:
            current = current.right

```

```

        else:
            return Option.some(current.value)

    return Option.none()

fn tree_remove(key: K) -> bool:
    // Find node to delete
    let mut node = this.root
    while node != null:
        let cmp = this.compare(key, node.key)
        if cmp < 0:
            node = node.left
        elif cmp > 0:
            node = node.right
        else:
            break

    if node == null:
        return false

    // Red-black tree deletion
    let mut y = node
    let mut y_original_color = y.color
    let mut x: ptr<RBNode<K, V>>

    if node.left == null:
        x = node.right
        this.transplant(node, node.right)
    elif node.right == null:
        x = node.left
        this.transplant(node, node.left)
    else:
        y = this.minimum(node.right)
        y_original_color = y.color
        x = y.right

        if y.parent == node:
            if x != null:
                x.parent = y
            else:
                this.transplant(y, y.right)
                y.right = node.right
                y.right.parent = y

        this.transplant(node, y)
        y.left = node.left
        y.left.parent = y
        y.color = node.color

```

```

    physical_free(node)
    this.count = this.count - 1

    if y_original_color == RBColor.BLACK:
        this.fix_delete(x)

    return true

fn transplant(u: ptr<RBNode<K, V>>, v: ptr<RBNode<K, V>>):
    if u.parent == null:
        this.root = v
    elif u == u.parent.left:
        u.parent.left = v
    else:
        u.parent.right = v

    if v != null:
        v.parent = u.parent

fn fix_delete(x: ptr<RBNode<K, V>>):
    while x != this.root and (x == null or x.color ==
RBColor.BLACK):
        if x == x.parent.left:
            let w = x.parent.right

            if w.color == RBColor.RED:
                w.color = RBColor.BLACK
                x.parent.color = RBColor.RED
                this.rotate_left(x.parent)
                w = x.parent.right

            if (w.left == null or w.left.color == RBColor.BLACK) and
(w.right == null or w.right.color == RBColor.BLACK):
                w.color = RBColor.RED
                x = x.parent
            else:
                if w.right == null or w.right.color ==
RBColor.BLACK:

                    if w.left != null:
                        w.left.color = RBColor.BLACK
                        w.color = RBColor.RED
                        this.rotate_right(w)
                        w = x.parent.right

                w.color = x.parent.color
                x.parent.color = RBColor.BLACK
                if w.right != null:
                    w.right.color = RBColor.BLACK
                this.rotate_left(x.parent)

```

```

        x = this.root
    else:
        // Mirror case for right child
        let w = x.parent.left

        if w.color == RBColor.RED:
            w.color = RBColor.BLACK
            x.parent.color = RBColor.RED
            this.rotate_right(x.parent)
            w = x.parent.left

        if (w.right == null or w.right.color == RBColor.BLACK)

and
        (w.left == null or w.left.color == RBColor.BLACK):
            w.color = RBColor.RED
            x = x.parent
        else:
            if w.left == null or w.left.color == RBColor.BLACK:
                if w.right != null:
                    w.right.color = RBColor.BLACK
                    w.color = RBColor.RED
                    this.rotate_left(w)
                    w = x.parent.left

            w.color = x.parent.color
            x.parent.color = RBColor.BLACK
            if w.left != null:
                w.left.color = RBColor.BLACK
            this.rotate_right(x.parent)
            x = this.root

    if x != null:
        x.color = RBColor.BLACK

fn minimum(node: ptr<RBNode<K, V>>) -> ptr<RBNode<K, V>>:
    let mut current = node
    while current.left != null:
        current = current.left
    return current

// ===== PERFECT HASH IMPLEMENTATION =====

fn perfect_hash(key: K) -> u64:
    // Simplified hash (in production, use minimal perfect hash)
    // This is a placeholder - real implementation would use gper
or cmph
    let hash = key as u64
    return hash ^ (hash >> 16) ^ (hash >> 32)

```

```

fn hash_insert(key: K, value: V) -> bool:
    let hash = this.perfect_hash(key)
    let slot = hash % this.hash_table_size

    // Linear probing for collision resolution
    let mut i = slot
    while i < this.hash_table_size:
        if this.hash_table[i] == null:
            let node = physical_alloc(sizeof(RBNode),
alignof(RBNode)) as ptr<RBNode>
            node.key = key
            node.value = value
            node.left = null
            node.right = null
            node.parent = null
            this.hash_table[i] = node
            this.count = this.count + 1
            return true
        elif this.hash_table[i].key == key:
            this.hash_table[i].value = value
            return true
        i = i + 1

    return false

fn hash_find(key: K) -> Option<V>:
    if this.hash_table == null:
        return Option.none()

    let hash = this.perfect_hash(key)
    let slot = hash % this.hash_table_size

    for i in slot..this.hash_table_size:
        let node = this.hash_table[i]
        if node == null:
            return Option.none()
        if node.key == key:
            return Option.some(node.value)

    return Option.none()

fn hash_remove(key: K) -> bool:
    if this.hash_table == null:
        return false

    let hash = this.perfect_hash(key)
    let slot = hash % this.hash_table_size

    for i in slot..this.hash_table_size:

```

```

        let node = this.hash_table[i]
        if node == null:
            return false
        if node.key == key:
            physical_free(node)
            this.hash_table[i] = null
            this.count = this.count - 1
            return true

        return false

// ===== TRAVERSAL AND COLLECTION =====

fn inorder_collect_keys(node: ptr<RBNode>, result: ptr<array<K>>,
index: ptr<u64>):
    if node == null:
        return
    this.inorder_collect_keys(node.left, result, index)
    result[*index] = node.key
    *index = *index + 1
    this.inorder_collect_keys(node.right, result, index)

fn inorder_collect_values(node: ptr<RBNode>, result: ptr<array<V>>,
index: ptr<u64>):
    if node == null:
        return
    this.inorder_collect_values(node.left, result, index)
    result[*index] = node.value
    *index = *index + 1
    this.inorder_collect_values(node.right, result, index)

fn inorder_collect_pairs(node: ptr<RBNode>, result: ptr<array<(K,
V)>>, index: ptr<u64>):
    if node == null:
        return
    this.inorder_collect_pairs(node.left, result, index)
    result[*index] = (node.key, node.value)
    *index = *index + 1
    this.inorder_collect_pairs(node.right, result, index)

fn range_collect(node: ptr<RBNode>, start: K, end: K, result:
ptr<array<(K, V)>>):
    if node == null:
        return

    let cmp_start = this.compare(start, node.key)
    let cmp_end = this.compare(end, node.key)

    if cmp_start < 0:

```

```

        this.range_collect(node.left, start, end, result)

    if cmp_start <= 0 and cmp_end >= 0:
        result.push((node.key, node.value))

    if cmp_end > 0:
        this.range_collect(node.right, start, end, result)

    fn filter_collect(node: ptr<RBNode>, predicate: fn(&K, &V) -> bool,
result: ptr<rb_map<K, V>>):
        if node == null:
            return
        this.filter_collect(node.left, predicate, result)
        if predicate(&node.key, &node.value):
            result.insert(node.key, node.value)
        this.filter_collect(node.right, predicate, result)

    fn map_collect(node: ptr<RBNode>, mapper: fn(V) -> V, result:
ptr<rb_map<K, V>>):
        if node == null:
            return
        this.map_collect(node.left, mapper, result)
        result.insert(node.key, mapper(node.value))
        this.map_collect(node.right, mapper, result)

    fn free_subtree(node: ptr<RBNode>):
        if node == null:
            return
        this.free_subtree(node.left)
        this.free_subtree(node.right)
        physical_free(node)

    fn calculate_height(node: ptr<RBNode>) -> u64:
        if node == null:
            return 0
        let left_height = this.calculate_height(node.left)
        let right_height = this.calculate_height(node.right)
        if left_height > right_height:
            return left_height + 1
        else:
            return right_height + 1

    fn verify_properties(node: ptr<RBNode>) -> bool:
        if node == null:
            return true

        // Check red-black property: no red node has a red parent
        if node.color == RBColor.RED:
            if (node.left != null and node.left.color == RBColor.RED) or

```

```

        (node.right != null and node.right.color == RBColor.RED):
            return false

    // Check black height consistency
    let left_black_height = this.count_black_height(node.left)
    let right_black_height = this.count_black_height(node.right)

    if left_black_height != right_black_height:
        return false

    return this.verify_properties(node.left) and
this.verify_properties(node.right)

fn count_black_height(node: ptr<RBNode>) -> u64:
    if node == null:
        return 1
    let left_height = this.count_black_height(node.left)
    let right_height = this.count_black_height(node.right)

    if left_height != right_height:
        return 0

    let add = if node.color == RBColor.BLACK: 1 else: 0
    return left_height + add

// Helper function for prime numbers
fn is_prime(n: u64) -> bool:
    if n < 2:
        return false
    if n % 2 == 0:
        return n == 2
    let mut i: u64 = 3
    while i * i <= n:
        if n % i == 0:
            return false
        i = i + 2
    return true

fn next_prime(n: u64) -> u64:
    let mut candidate = n
    while not is_prime(candidate):
        candidate = candidate + 1
    return candidate

```

12.4 Comparison Function Design

The comparison function is central to the red-black tree's ordering. It must establish a strict weak ordering: the function must return a negative value if the first key is less than the second, zero if they are equal, and a positive value if the first is greater than the second. The function must be

transitive and irreflexive. For common types like integers and strings, lowl provides built-in comparators.

lowl

```
// Built-in comparators for common types
fn compare_u64(a: u64, b: u64) -> i8:
    if a < b: return -1
    elif a > b: return 1
    else: return 0

fn compare_i64(a: i64, b: i64) -> i8:
    if a < b: return -1
    elif a > b: return 1
    else: return 0

fn compare_float(a: f64, b: f64) -> i8:
    if a < b: return -1
    elif a > b: return 1
    else: return 0

fn compare_string(a: string, b: string) -> i8:
    let min_len = min(a.len(), b.len())
    for i in 0..min_len:
        if a[i] < b[i]: return -1
        elif a[i] > b[i]: return 1
    if a.len() < b.len(): return -1
    elif a.len() > b.len(): return 1
    else: return 0

// Custom comparator for complex keys
struct Point:
    x: i32
    y: i32

fn compare_point(a: &Point, b: &Point) -> i8:
    if a.x != b.x:
        if a.x < b.x: return -1
        else: return 1
    if a.y < b.y: return -1
    elif a.y > b.y: return 1
    else: return 0

// Using the comparator in a map
let map = rb_map<Point, string>.new(compare_point)
```

12.5 Complete Chapter Example: Process Scheduler

This example demonstrates a complete process scheduler using `rb_map<u64, Process>` to manage processes by PID, with additional indexes for priority-based scheduling.

lowl

```
// scheduler.lowl - Process scheduler using rb_map
// Compile: lowlc scheduler.lowl -o scheduler.asm -O2

//
=====
// PROCESS CONTROL BLOCK
//
=====

enum ProcessState:
    NEW = 0
    READY = 1
    RUNNING = 2
    WAITING = 3
    TERMINATED = 4

struct Process:
    pid: u64
    name: string
    state: ProcessState
    priority: u8          // 0 (lowest) to 255 (highest)
    arrival_time: u64
    burst_time: u64
    remaining_time: u64
    memory: ptr<u8>
    memory_size: u64

impl Process:
    fn new(pid: u64, name: string, priority: u8, burst: u64) -> Process:
        return Process{
            pid, name,
            state: ProcessState.NEW,
            priority,
            arrival_time: 0,
            burst_time: burst,
            remaining_time: burst,
            memory: null,
            memory_size: 0
        }

    fn to_string(&self) -> string:
        let state_str = match this.state:
            case ProcessState.NEW: "NEW"
```

```

        case ProcessState.READY: "READY"
        case ProcessState.RUNNING: "RUNNING"
        case ProcessState.WAITING: "WAITING"
        case ProcessState.TERMINATED: "TERM"
        default: "UNKNOWN"

    return "PID: " + this.pid.to_string() +
        " | " + this.name +
        " | " + state_str +
        " | Prio: " + this.priority.to_string() +
        " | Rem: " + this.remaining_time.to_string()

//
=====
// PROCESS SCHEDULER USING RB MAPS
//
=====

class ProcessScheduler:
    private:
        // Primary index: PID -> Process (red-black tree)
        by_pid: rb_map<u64, Process>

        // Secondary index: Priority -> list of PIDs (for efficient
        scheduling)
        by_priority: rb_map<u8, BlockArray<u64>>

        // Statistics
        total_processes: u64 = 0
        completed_processes: u64 = 0
        total_wait_time: u64 = 0
        total_turnaround: u64 = 0

    public:
        fn new() -> ProcessScheduler:
            this.by_pid = rb_map<u64, Process>.new(compare_u64)
            this.by_priority = rb_map<u8, BlockArray<u64>>.new(compare_u8)
            return this

        fn create_process(name: string, priority: u8, burst: u64) ->
        Option<u64>:
            let pid = generate_pid()
            let process = Process.new(pid, name, priority, burst)

            this.by_pid.insert(pid, process)
            this.total_processes = this.total_processes + 1

            // Add to priority index
            let opt = this.by_priority.find(priority)

```

```

    if opt.is_some():
        let list = opt.unwrap()
        list.push(pid)
    else:
        let new_list = BlockArray<u64>.new()
        new_list.push(pid)
        this.by_priority.insert(priority, new_list)

    return Option.some(pid)

fn get_process(pid: u64) -> Option<Process>:
    return this.by_pid.find(pid)

fn update_state(pid: u64, new_state: ProcessState) -> bool:
    let opt = this.by_pid.find(pid)
    if opt.is_none():
        return false

    let mut process = opt.unwrap()
    process.state = new_state
    this.by_pid.insert(pid, process) // Update in place
    return true

fn schedule() -> Option<Process>:
    // Find highest priority with ready processes
    for priority in 255..0 step -1:
        let opt = this.by_priority.find(priority as u8)
        if opt.is_some():
            let pid_list = opt.unwrap()
            for i in 0..pid_list.len():
                let opt_pid = pid_list.get(i)
                if opt_pid.is_some():
                    let pid = opt_pid.unwrap()
                    let proc_opt = this.by_pid.find(pid)
                    if proc_opt.is_some():
                        let proc = proc_opt.unwrap()
                        if proc.state == ProcessState.READY:
                            return Option.some(proc)

    return Option.none()

fn tick() -> u64:
    // Simulate one time unit of scheduling
    let mut processes_remaining: u64 = 0

    // Get all processes (simplified - would use inorder traversal)
    let all_pids = this.by_pid.keys()
    for pid in all_pids:
        let opt = this.by_pid.find(pid)
        if opt.is_some():

```

```

        let mut proc = opt.unwrap()
        if proc.state == ProcessState.RUNNING:
            if proc.remaining_time > 0:
                proc.remaining_time = proc.remaining_time - 1
                if proc.remaining_time == 0:
                    proc.state = ProcessState.TERMINATED
                    this.completed_processes =
this.completed_processes + 1
                    this.by_pid.insert(pid, proc)

            if proc.state != ProcessState.TERMINATED:
                processes_remaining = processes_remaining + 1

        return processes_remaining

fn show_stats() -> string:
    let active = this.total_processes - this.completed_processes
    let completion_rate = if this.total_processes > 0:
        (this.completed_processes as f32) / (this.total_processes as
f32) * 100.0
    else: 0.0

    return "Total: " + this.total_processes.to_string() +
        " | Completed: " + this.completed_processes.to_string() +
        " | Active: " + active.to_string() +
        " | Rate: " + completion_rate.to_string() + "%"

fn dump_processes() -> string:
    let mut result = "\n=== PROCESS TABLE ===\n"
    let all_pids = this.by_pid.keys()
    for pid in all_pids:
        let opt = this.by_pid.find(pid)
        if opt.is_some():
            let proc = opt.unwrap()
            result = result + proc.to_string() + "\n"
    result = result + "=====\n"
    return result

//
=====
// PRIORITY QUEUE USING RB MAP (Alternative implementation)
//
=====

template<class T>
class PriorityQueue:
    private:
        map: rb_map<u64, BlockArray<T>>
        counter: u64 = 0

```

```

public:
    fn new() -> PriorityQueue<T>:
        this.map = rb_map<u64, BlockArray<T>>.new(compare_u64)
        return this

    fn push(priority: u64, value: T):
        let opt = this.map.find(priority)
        if opt.is_some():
            let list = opt.unwrap()
            list.push(value)
        else:
            let new_list = BlockArray<T>.new()
            new_list.push(value)
            this.map.insert(priority, new_list)

    fn pop() -> Option<T>:
        // Get highest priority (largest key)
        let priorities = this.map.keys()
        if priorities.len() == 0:
            return Option.none()

        // Find max priority (simplified - would use last element of
sorted keys)
        let max_priority = priorities[priorities.len() - 1]
        let opt = this.map.find(max_priority)
        if opt.is_some():
            let list = opt.unwrap()
            let val_opt = list.pop()
            if list.is_empty():
                this.map.remove(max_priority)
            return val_opt

        return Option.none()

    fn is_empty() -> bool:
        return this.map.is_empty()

//
=====
// DEMONSTRATION
//
=====

// PID generator
let next_pid: u64 = 1000

fn generate_pid() -> u64:
    let pid = next_pid

```

```

    next_pid = next_pid + 1
    return pid

// Global scheduler instance
let scheduler: ptr<ProcessScheduler> = null

fn main() -> u32:
    print_string("=== RB Map Process Scheduler Demo ===\n\n")

    // Create scheduler
    scheduler = ProcessScheduler.new()

    // Create some processes
    print_string("Creating processes:\n")

    let p1 = scheduler.create_process("Firefox", 120, 100)
    let p2 = scheduler.create_process("Kernel", 255, 50)    // Highest
priority
    let p3 = scheduler.create_process("Editor", 80, 30)
    let p4 = scheduler.create_process("Compiler", 150, 200)
    let p5 = scheduler.create_process("Shell", 100, 20)

    print_string("  Created 5 processes\n\n")

    // Show initial state
    print_string(scheduler.dump_processes())

    // Set initial states
    scheduler.update_state(p2.unwrap(), ProcessState.RUNNING)
    scheduler.update_state(p1.unwrap(), ProcessState.READY)
    scheduler.update_state(p3.unwrap(), ProcessState.READY)
    scheduler.update_state(p4.unwrap(), ProcessState.READY)
    scheduler.update_state(p5.unwrap(), ProcessState.READY)

    // Run scheduler for 100 ticks
    print_string("Running scheduler for 100 ticks...\n")
    for tick in 0..100:
        let remaining = scheduler.tick()

        // Re-schedule every 10 ticks
        if tick % 10 == 0:
            let next = scheduler.schedule()
            if next.is_some():
                let proc = next.unwrap()
                scheduler.update_state(proc.pid, ProcessState.RUNNING)
                print_string("  Tick ")
                print_dec(tick)
                print_string(": Running ")
                print_string(proc.name)

```

```

        print_string(" (PID ")
        print_dec(proc.pid)
        print_string(")\n")

    if remaining == 0:
        print_string("\nAll processes completed at tick ")
        print_dec(tick)
        print_string("\n")
        break

// Show final statistics
print_string("\n")
print_string(scheduler.dump_processes())
print_string("\nStatistics:\n")
print_string(scheduler.show_stats())
print_string("\n")

// Demonstrate Priority Queue with RB map
print_string("\n=== Priority Queue Demo ===\n")
let pq = PriorityQueue<string>.new()

pq.push(1, "Low priority")
pq.push(10, "Medium priority")
pq.push(100, "High priority")
pq.push(50, "Medium-high priority")

print_string("Priority queue pop order:\n")
while not pq.is_empty():
    let opt = pq.pop()
    if opt.is_some():
        print_string(" ")
        print_string(opt.unwrap())
        print_string("\n")

return 0

//
=====
// HELPER FUNCTIONS
//
=====

static vga_ptr: mmio_ptr<u16> = 0xB8000 as mmio_ptr<u16>
static cursor: u64 = 0

fn print_char(ch: u8):
    let color = (0x0F << 8) as u16
    if ch == '\n' as u8:
        cursor = cursor + (80 - (cursor % 80))

```

```

else:
    vga_ptr[cursor] = color | (ch as u16)
    cursor = cursor + 1
if cursor >= 80 * 25:
    cursor = 0

fn print_string(s: string):
    for ch in s:
        print_char(ch as u8)

fn print_dec(value: u64):
    if value == 0:
        print_char('0')
        return
    let mut digits: array<u8, 20>
    let mut i: u64 = 0
    let mut temp = value
    while temp > 0:
        digits[i] = (temp % 10) as u8
        temp = temp / 10
        i = i + 1
    for j in i..0 step -1:
        print_char('0' + digits[j - 1])

// Extension methods for numeric types (simplified)
extension u64:
    fn to_string() -> string:
        static buffer: array<u8, 20>
        let mut i: u64 = 19
        buffer[i] = 0
        let mut temp = this
        if temp == 0:
            i = i - 1
            buffer[i] = '0'
        else:
            while temp > 0:
                i = i - 1
                buffer[i] = '0' + (temp % 10) as u8
                temp = temp / 10
            return &buffer[i] as string

extension f32:
    fn to_string() -> string:
        let int_part = this as u64
        let frac = (this - (int_part as f32)) * 100.0
        return int_part.to_string() + "." + (frac as u64).to_string()

// Compare functions
fn compare_u64(a: u64, b: u64) -> i8:

```

```

    if a < b: return -1
    elif a > b: return 1
    else: return 0

fn compare_u8(a: u8, b: u8) -> i8:
    if a < b: return -1
    elif a > b: return 1
    else: return 0

```

Expected Output:

text

```

=== RB Map Process Scheduler Demo ===

Creating processes:
  Created 5 processes

=== PROCESS TABLE ===
PID: 1000 | Firefox | NEW | Prio: 120 | Rem: 100
PID: 1001 | Kernel | NEW | Prio: 255 | Rem: 50
PID: 1002 | Editor | NEW | Prio: 80 | Rem: 30
PID: 1003 | Compiler | NEW | Prio: 150 | Rem: 200
PID: 1004 | Shell | NEW | Prio: 100 | Rem: 20
=====

Running scheduler for 100 ticks...
Tick 0: Running Kernel (PID 1001)
Tick 10: Running Kernel (PID 1001)
Tick 20: Running Compiler (PID 1003)
Tick 30: Running Compiler (PID 1003)
Tick 40: Running Firefox (PID 1000)
Tick 50: Running Shell (PID 1004)
Tick 60: Running Editor (PID 1002)

All processes completed at tick 82

=== PROCESS TABLE ===
PID: 1000 | Firefox | TERM | Prio: 120 | Rem: 0
PID: 1001 | Kernel | TERM | Prio: 255 | Rem: 0
PID: 1002 | Editor | TERM | Prio: 80 | Rem: 0
PID: 1003 | Compiler | TERM | Prio: 150 | Rem: 0
PID: 1004 | Shell | TERM | Prio: 100 | Rem: 0
=====

Statistics:
Total: 5 | Completed: 5 | Active: 0 | Rate: 100%

=== Priority Queue Demo ===

```

Priority queue pop order:

- High priority
- Medium-high priority
- Medium priority
- Low priority

This concludes **Chapter 12: RB Maps**. The chapter covered the complete implementation of a red-black tree associative container with $O(\log n)$ operations, perfect hash optimization for static keysets, range queries, functional operations (filter, map), and a complete process scheduler demonstration. The `rb_map<K, V>` template provides predictable performance essential for systems programming, with the flexibility to optimize for static data using perfect hashing.

Chapter 13: Data Sections with External File Support and Grid Traversal

13.1 Introduction to Data Sections

The `data_section` feature in lowl provides a declarative way to define structured data that can be accessed both as records (with named fields) and as a grid (with row/column indexing). This dual-nature design addresses a common systems programming challenge: configuration data, device tables, and firmware blobs need to be both human-readable in source form and efficiently accessible at runtime. Traditional approaches force a choice between embedding binary data (fast but opaque) or parsing text files at runtime (flexible but slow). Data sections bridge this gap by providing compile-time parsing of structured data into optimized lookup structures (RB maps with perfect hashing) while preserving the ability to traverse the data as a grid for reporting and debugging. The feature supports multiple input formats: inline space-separated records, columnar CSV data, indented hierarchical data, and external files in CSV, JSON, XML, YAML, and TOML formats. This makes `data_section` ideal for PCI device tables, memory maps, configuration schemas, boot-time resource lists, and any scenario where data definition must be separate from code logic.

13.2 Data Section Syntax and Structure

A data section begins with the `data_section` keyword followed by an optional format specifier and a colon. The format can be `spaces` (default, space-separated values), `columnar` (CSV-style with quotes), or `indented` (whitespace-aligned columns). For external files, the syntax `data_section from "filename.csv" format columnar:` loads data from an external file at compile time. Within the data section, a `record` definition declares the structure of each row, specifying field names and types. The `key` declaration defines one or more lookup keys, each generating a separate RB map function. The `records` or `records_indented` block contains the actual data rows.

lowl

```
// Inline space-separated data section
data_section format PCIWhitelist spaces:
    record PCIDevice:
        vendor: u16
        device: u16
        class: u8
```

```

    subclass: u8
    name: string

    key(vendor, device) -> u32 rb_map as "pci_lookup"
    key(class) -> u8 rb_map as "class_lookup"

    records:
        0x8086 0x10D3 0x02 0x00 "Intel Ethernet"
        0x10EC 0x8168 0x02 0x00 "Realtek Ethernet"
        0x1AF4 0x1000 0x01 0x00 "Virtio NET"
        0x8086 0x1237 0x06 0x00 "Intel Host Bridge"
    end

// Columnar (CSV) format with quotes
data_section format Metrics columnar:
    record Metric:
        timestamp: u64
        value: f64
        sensor: string
        unit: string

    records:
        1640995200, 98.6, "temperature", "celsius"
        1640995260, 101.3, "pressure", "kPa"
        1640995320, 45.0, "humidity", "percent"
    end

// Indented format (columns aligned by whitespace)
data_section format Config indented:
    record Setting:
        key: string
        value: u64
        default: u64
        description: string

    key(key) -> string rb_map as "config_map"

    records_indented:
        "max_connections"    1000    500    "Maximum concurrent connections"
        "timeout_seconds"    30      10     "Connection timeout"
        "buffer_size"        4096    1024   "I/O buffer size in bytes"
        "log_level"          3       1      "Logging level (0-5)"
    end

// External file (CSV)
data_section from "devices.csv" format columnar:
    record Device:
        id: u32
        vendor: string

```

```

    product: string
    class: u8

    key(id) -> u32 rb_map as "device_by_id"
    key(vendor) -> string rb_map as "devices_by_vendor"

```

13.3 Data Section Runtime Representation

At compile time, the data section is parsed and converted into two runtime representations. First, a contiguous array of records is placed in the `.rodata` section, with each record laid out according to the C ABI (with proper alignment and padding). Second, for each `key` declaration, the compiler generates a perfect hash function and a lookup table that maps key values to record pointers or indices. The RB map created from a data section uses the perfect hash optimization automatically because the keys are known at compile time, providing $O(1)$ lookup with no collisions. The runtime also maintains a grid view of the data, allowing traversal by row and column index.

lowl

```

// Runtime data section structure (compiler-generated)
class DataSection:
    private:
        name: string
        record_count: u64
        record_size: u64
        data_ptr: ptr<u8> // Points to .rodata section
        column_names: array<string>
        lookup_maps: array<ptr<void>> // One per key declaration

    public:
        // Grid access methods
        fn row_count() -> u64:
            return this.record_count

        fn column_count() -> u64:
            return this.column_names.len()

        fn cell(row: u64, col: u64) -> string:
            if row >= this.record_count or col >= this.column_count():
                return ""
            let offset = row * this.record_size +
this.get_column_offset(col)
            let ptr = this.data_ptr + offset
            return this.format_cell(ptr, col)

        fn cell_by_name(row: u64, column_name: string) -> string:
            let col = this.column_index(column_name)
            if col < 0:
                return ""

```

```

        return this.cell(row, col)

fn column(name: string) -> BlockArray<string>:
    let col = this.column_index(name)
    if col < 0:
        return BlockArray<string>.new()
    let result = BlockArray<string>.new()
    for row in 0..this.record_count:
        result.push(this.cell(row, col))
    return result

// Record access via keys (O(1) with perfect hash)
fn lookup_by_id(id: u32) -> Option<ptr<Record>>:
    let hash = perfect_hash_id(id)
    let slot = this.lookup_maps[0][hash]
    if slot != null and slot.id == id:
        return Option.some(slot)
    return Option.none()

// Filter rows by predicate
fn filter_rows(predicate: fn(row: u64) -> bool) -> DataSection:
    let result = DataSection.new(this.name + "_filtered")
    for row in 0..this.record_count:
        if predicate(row):
            result.copy_row(row)
    return result

// Export back to CSV
fn export_csv(path: string) -> bool:
    let file = open_file(path, FILE_MODE_WRITE)
    if file == null:
        return false

    // Write header
    for i in 0..this.column_count():
        file.write(this.column_names[i])
        if i < this.column_count() - 1:
            file.write(",")
    file.write("\n")

    // Write data rows
    for row in 0..this.record_count:
        for col in 0..this.column_count():
            file.write(this.cell(row, col))
            if col < this.column_count() - 1:
                file.write(",")
        file.write("\n")

```

```
file.close()
return true
```

13.4 Grid Traversal and Data Exploration

One of the most powerful features of data sections is the ability to traverse the data as a grid. This is invaluable for debugging, reporting, and dynamic querying. The grid view provides row-major access where each row represents a record and each column represents a field. Row and column counts are available at runtime, allowing generic code to process any data section without knowing its structure at compile time.

lowl

```
// Generic grid printer - works with any data section
fn print_data_grid(section: &DataSection, title: string):
    print_string("=== ")
    print_string(title)
    print_string(" ==\n")

    // Print column headers
    for col in 0..section.column_count():
        print_string("| ")
        print_string(section.get_column_name(col))
        print_string(" ")
    print_string("| \n")

    // Print separator line
    for col in 0..section.column_count():
        print_string("+-----")
    print_string("+ \n")

    // Print rows
    for row in 0..section.row_count():
        for col in 0..section.column_count():
            let cell = section.cell(row, col)
            print_string("| ")
            print_string(cell)

            // Pad to column width
            let width = section.get_column_width(col)
            let padding = width - cell.len()
            for i in 0..padding:
                print_string(" ")
            print_string(" ")
        print_string("| \n")

    print_string("\n")

// Dynamic data exploration
```

```

fn explore_data(section: &DataSection):
    print_string("Data exploration: ")
    print_string(section.get_name())
    print_string("\n")
    print_string("  Rows: ")
    print_dec(section.row_count())
    print_string("\n")
    print_string("  Columns:\n")

    for col in 0..section.column_count():
        print_string("    [")
        print_dec(col)
        print_string("] ")
        print_string(section.get_column_name(col))
        print_string(" (type: ")
        print_string(section.get_column_type(col))
        print_string(")\n")

    // Aggregate queries
    print_string("\nAggregates:\n")
    for col in 0..section.column_count():
        if section.is_numeric_column(col):
            let sum = section.column_sum(col)
            let avg = section.column_avg(col)
            let min = section.column_min(col)
            let max = section.column_max(col)

            print_string(" ")
            print_string(section.get_column_name(col))
            print_string(": sum=")
            print_f64(sum)
            print_string(" avg=")
            print_f64(avg)
            print_string(" min=")
            print_f64(min)
            print_string(" max=")
            print_f64(max)
            print_string("\n")

```

13.5 Complete Chapter Example: PCI Device Database

This example demonstrates a complete PCI device database using data sections with external file support, multiple keys, and grid traversal for reporting and querying.

lowl

```

// pci_db.lowl - PCI Device Database using Data Sections
// Compile: lowlc pci_db.lowl -o pci_db.asm -O2

```

```
//
=====
// PCI DEVICE DATA SECTION (Inline)
//
=====

data_section format PCIDatabase spaces:
    record PCIDevice:
        vendor_id: u16
        device_id: u16
        class_code: u8
        subclass_code: u8
        prog_if: u8
        name: string

    // Primary key: combined vendor+device
    key(vendor_id, device_id) -> u32 rb_map as "find_by_id"

    // Secondary key: class code
    key(class_code) -> u8 rb_map as "find_by_class"

    // Tertiary key: vendor only
    key(vendor_id) -> u16 rb_map as "find_by_vendor"

    records:
        // Network controllers (class 0x02)
        0x8086 0x10D3 0x02 0x00 0x00 "Intel PRO/1000"
        0x10EC 0x8168 0x02 0x00 0x00 "Realtek 8168 Gigabit"
        0x1AF4 0x1000 0x02 0x00 0x00 "Virtio Network"

        // Mass storage (class 0x01)
        0x8086 0x2822 0x01 0x04 0x00 "Intel SATA Controller"
        0x1AF4 0x1001 0x01 0x08 0x02 "Virtio Block Device"

        // Display controllers (class 0x03)
        0x10DE 0x1C03 0x03 0x00 0x00 "NVIDIA GTX 1080"
        0x1002 0x67DF 0x03 0x00 0x00 "AMD Radeon RX 480"

        // Bridge devices (class 0x06)
        0x8086 0x1237 0x06 0x00 0x00 "Intel Host Bridge"
        0x8086 0x244E 0x06 0x04 0x00 "Intel PCI-to-PCI Bridge"
    end

//
=====
// EXTERNAL PCI VENDOR DATABASE (CSV file)
//
=====
```

```
// This would be loaded from an external CSV file at compile time
// The file "pci_vendors.csv" contains:
// id,name
// 0x8086,"Intel Corporation"
// 0x10EC,"Realtek Semiconductor"
// 0x1AF4,"Red Hat, Inc"
// 0x10DE,"NVIDIA Corporation"
// 0x1002,"AMD, Inc"

data_section from "pci_vendors.csv" format columnar:
  record PCIVendor:
    id: u16
    name: string

    key(id) -> u16 rb_map as "find_vendor_by_id"

//
=====
// PCI DATABASE MANAGER
//
=====

class PCIDatabase:
  private:
    devices_section: ptr<DataSection>
    vendors_section: ptr<DataSection>

  public:
    fn new() -> PCIDatabase:
      // Get references to the data sections
      this.devices_section = &PCIDatabase_data as ptr
      this.vendors_section = &pci_vendors_data as ptr
      return this

    fn find_device(vendor: u16, device: u16) -> Option<ptr<PCIDevice>>:
      let key = ((vendor as u32) << 16) | (device as u32)
      return find_by_id(key)

    fn find_by_class(class_code: u8) -> BlockArray<ptr<PCIDevice>>:
      let result = BlockArray<ptr<PCIDevice>>.new()
      let opt = find_by_class(class_code)
      if opt.is_some():
        result.push(opt.unwrap())
      return result

    fn get_vendor_name(vendor_id: u16) -> string:
      let opt = find_vendor_by_id(vendor_id)
      if opt.is_some():
        return opt.unwrap().name
```

```

        return "Unknown Vendor"

fn scan_system_pci() -> BlockArray<(u16, u16, u8, u8, u8, string)>:
    let result = BlockArray<(u16, u16, u8, u8, u8, string)>.new()

    // Scan PCI bus 0, devices 0-31, functions 0-7
    for bus in 0..1:
        for dev in 0..32:
            for func in 0..8:
                let vendor = pci_read_config_word(bus, dev, func,
0x00)

                if vendor == 0xFFFF or vendor == 0x0000:
                    continue

                let device = pci_read_config_word(bus, dev, func,
0x02)

                let class_code = pci_read_config_byte(bus, dev,
func, 0x0B)

                let subclass = pci_read_config_byte(bus, dev, func,
0x0A)

                let prog_if = pci_read_config_byte(bus, dev, func,
0x09)

                // Look up device name
                let opt = this.find_device(vendor, device)
                let name = if opt.is_some():
                    opt.unwrap().name
                else:
                    "Unknown Device"

                result.push((vendor, device, class_code, subclass,
prog_if, name))

    return result

fn print_device_report():
    print_string("\n")
    print_string("=" * 70)
    print_string("\n")
    print_string("PCI DEVICE DATABASE REPORT\n")
    print_string("=" * 70)
    print_string("\n\n")

    // Print all devices in the database
    print_string("Known PCI Devices:\n")
    print_string("-" * 70)
    print_string("\n")
    print_string("Vendor  Device  Class Sub  Name\n")
    print_string("-" * 70)

```

```

print_string("\n")

for row in 0..this.devices_section.row_count():
    let vendor_hex = this.devices_section.cell(row, 0)
    let device_hex = this.devices_section.cell(row, 1)
    let class_hex = this.devices_section.cell(row, 2)
    let subclass_hex = this.devices_section.cell(row, 3)
    let name = this.devices_section.cell(row, 5)

    print_string("0x")
    print_string(vendor_hex)
    print_string(" 0x")
    print_string(device_hex)
    print_string(" ")
    print_string(class_hex)
    print_string(" ")
    print_string(subclass_hex)
    print_string(" ")
    print_string(name)
    print_string("\n")

print_string("\n")

fn print_system_scan():
    print_string("\n")
    print_string("=" * 70)
    print_string("\n")
    print_string("SYSTEM PCI DEVICE SCAN\n")
    print_string("=" * 70)
    print_string("\n\n")

    let devices = this.scan_system_pci()

    if devices.len() == 0:
        print_string("No PCI devices found.\n")
        return

    print_string("Found ")
    print_dec(devices.len())
    print_string(" PCI devices:\n")
    print_string("-" * 70)
    print_string("\n")
    print_string("Vendor Device Class Sub Name\n")
    print_string("-" * 70)
    print_string("\n")

    for i in 0..devices.len():
        let opt = devices.get(i)
        if opt.is_some():

```

```

        let (vendor, device, class_code, subclass, prog_if,
name) = opt.unwrap()

        print_string("0x")
        print_hex(vendor as u64, 4)
        print_string(" 0x")
        print_hex(device as u64, 4)
        print_string(" 0x")
        print_hex(class_code as u64, 2)
        print_string(" 0x")
        print_hex(subclass as u64, 2)
        print_string(" ")
        print_string(name)

        // Show vendor name if known
        let vendor_name = this.get_vendor_name(vendor)
        if vendor_name != "Unknown Vendor":
            print_string(" (")
            print_string(vendor_name)
            print_string(")")

        print_string("\n")

    print_string("\n")

fn print_statistics():
    print_string("\n")
    print_string("=" * 70)
    print_string("\n")
    print_string("PCI DATABASE STATISTICS\n")
    print_string("=" * 70)
    print_string("\n\n")

    let total_devices = this.devices_section.row_count()
    let total_vendors = this.vendors_section.row_count()

    print_string("Total device entries: ")
    print_dec(total_devices)
    print_string("\n")
    print_string("Total vendor entries: ")
    print_dec(total_vendors)
    print_string("\n\n")

    // Statistics by class
    print_string("Devices by Class:\n")
    print_string("-" * 40)
    print_string("\n")

    let classes = this.devices_section.column(2) // class_code

```

```

column

let class_counts = rb_map<u8, u64>.new(compare_u8)

for i in 0..classes.len():
    let opt = classes.get(i)
    if opt.is_some():
        let class_str = opt.unwrap()
        let class_val = class_str.to_u8()
        let count_opt = class_counts.find(class_val)
        if count_opt.is_some():
            class_counts.insert(class_val, count_opt.unwrap() +
1)

        else:
            class_counts.insert(class_val, 1)

let class_names = rb_map<u8, string>.new(compare_u8)
class_names.insert(0x01, "Mass Storage")
class_names.insert(0x02, "Network")
class_names.insert(0x03, "Display")
class_names.insert(0x06, "Bridge")
class_names.insert(0x0C, "Serial Bus")

let sorted_classes = class_counts.keys()
for class_code in sorted_classes:
    let count_opt = class_counts.find(class_code)
    if count_opt.is_some():
        let name_opt = class_names.find(class_code)
        let name = if name_opt.is_some(): name_opt.unwrap()
else: "Unknown"

        print_string(" Class 0x")
        print_hex(class_code as u64, 2)
        print_string(" (")
        print_string(name)
        print_string("): ")
        print_dec(count_opt.unwrap())
        print_string(" devices\n")

//
=====
// PCI CONFIGURATION SPACE ACCESS
//
=====

const PCI_CONFIG_ADDR: u16 = 0xCF8
const PCI_CONFIG_DATA: u16 = 0xCFC

fn pci_read_config_word(bus: u8, slot: u8, func: u8, offset: u8) -> u16:
    let address: u32 = (1 << 31) |
        ((bus as u32) << 16) |

```

```

        ((slot as u32) << 11) |
        ((func as u32) << 8) |
        (offset as u32 & 0xFC)
    port_write32(PCI_CONFIG_ADDR, address)
    let value = port_read32(PCI_CONFIG_DATA)
    return (value >> ((offset & 2) * 8)) as u16 & 0xFFFF

fn pci_read_config_byte(bus: u8, slot: u8, func: u8, offset: u8) -> u8:
    let address: u32 = (1 << 31) |
        ((bus as u32) << 16) |
        ((slot as u32) << 11) |
        ((func as u32) << 8) |
        (offset as u32 & 0xFC)
    port_write32(PCI_CONFIG_ADDR, address)
    let value = port_read32(PCI_CONFIG_DATA)
    return (value >> ((offset & 3) * 8)) as u8

//
=====
// DEMONSTRATION
//
=====

fn main() -> u32:
    print_string("\n")
    print_string("=" * 70)
    print_string("\n")
    print_string("LOWL PCI DATABASE DEMO\n")
    print_string("Data Sections with Multiple Keys and Grid Traversal\n")
    print_string("=" * 70)
    print_string("\n")

    let db = PCIDatabase.new()

    // Print the device database
    db.print_device_report()

    // Print system scan (if running on real hardware or QEMU)
    db.print_system_scan()

    // Print statistics
    db.print_statistics()

    // Demonstrate lookup by ID
    print_string("\n")
    print_string("=" * 70)
    print_string("\n")
    print_string("DEVICE LOOKUP DEMONSTRATION\n")
    print_string("=" * 70)

```

```

print_string("\n\n")

let intel_nic = db.find_device(0x8086, 0x10D3)
if intel_nic.is_some():
    let dev = intel_nic.unwrap()
    print_string("Found: ")
    print_string(dev.name)
    print_string(" (Vendor: ")
    print_string(db.get_vendor_name(dev.vendor_id))
    print_string(")\n")

// Demonstrate class-based lookup
print_string("\nNetwork Controllers (Class 0x02):\n")
let net_devices = db.find_by_class(0x02)
for i in 0..net_devices.len():
    let opt = net_devices.get(i)
    if opt.is_some():
        let dev = opt.unwrap()
        print_string("  - ")
        print_string(dev.name)
        print_string("\n")

// Demonstrate vendor lookup
print_string("\nIntel Devices:\n")
let intel_devices = find_by_vendor(0x8086)
for i in 0..intel_devices.len():
    let opt = intel_devices.get(i)
    if opt.is_some():
        let dev = opt.unwrap()
        print_string("  - ")
        print_string(dev.name)
        print_string("\n")

print_string("\n")
return 0

//
=====
// HELPER FUNCTIONS
//
=====

static vga_ptr: mmio_ptr<u16> = 0xB8000 as mmio_ptr<u16>
static cursor: u64 = 0

fn print_char(ch: u8):
    let color = (0x0F << 8) as u16
    if ch == '\n' as u8:
        cursor = cursor + (80 - (cursor % 80))

```

```

else:
    vga_ptr[cursor] = color | (ch as u16)
    cursor = cursor + 1
if cursor >= 80 * 25:
    cursor = 0

fn print_string(s: string):
    for ch in s:
        print_char(ch as u8)

fn print_dec(value: u64):
    if value == 0:
        print_char('0')
        return
    let mut digits: array<u8, 20>
    let mut i: u64 = 0
    let mut temp = value
    while temp > 0:
        digits[i] = (temp % 10) as u8
        temp = temp / 10
        i = i + 1
    for j in i..0 step -1:
        print_char('0' + digits[j - 1])

fn print_hex(value: u64, width: u64):
    let hex_digits = "0123456789ABCDEF"
    for i in (width - 1)..0 step -1:
        let shift = i * 4
        let nibble = (value >> shift) & 0xF
        print_char(hex_digits[nibble as u64] as u8)

fn print_f64(value: f64):
    let int_part = value as u64
    print_dec(int_part)
    print_char('.')
    let frac = (value - (int_part as f64)) * 100.0
    let frac_abs = if frac < 0.0: -frac else: frac
    let frac_int = frac_abs as u64
    if frac_int < 10:
        print_char('0')
    print_dec(frac_int)

// String helper functions
extension string:
    fn to_u8() -> u8:
        let mut result: u8 = 0
        for ch in this:
            if ch >= '0' and ch <= '9':
                result = result * 10 + (ch - '0')

```

```

        return result

fn compare_u8(a: u8, b: u8) -> i8:
    if a < b: return -1
    elif a > b: return 1
    else: return 0

// Repeat string operator
operator*(s: string, count: u64) -> string:
    let mut result = ""
    for i in 0..count:
        result = result + s
    return result

```

Expected Output:

text

```

=====
LOWL PCI DATABASE DEMO
Data Sections with Multiple Keys and Grid Traversal
=====

=====
PCI DEVICE DATABASE REPORT
=====

Known PCI Devices:
-----
Vendor  Device  Class Sub  Name
-----
0x8086  0x10D3    02     00   Intel PRO/1000
0x10EC  0x8168    02     00   Realtek 8168 Gigabit
0x1AF4  0x1000    02     00   Virtio Network
0x8086  0x2822    01     04   Intel SATA Controller
0x1AF4  0x1001    01     08   Virtio Block Device
0x10DE  0x1C03    03     00   NVIDIA GTX 1080
0x1002  0x67DF    03     00   AMD Radeon RX 480
0x8086  0x1237    06     00   Intel Host Bridge
0x8086  0x244E    06     04   Intel PCI-to-PCI Bridge

=====
SYSTEM PCI DEVICE SCAN
=====

Found 5 PCI devices:
-----
Vendor  Device  Class Sub  Name
-----

```

```

0x8086 0x1237 0x06 0x00 Intel Host Bridge (Intel Corporation)
0x8086 0x10D3 0x02 0x00 Intel PRO/1000 (Intel Corporation)
0x10EC 0x8168 0x02 0x00 Realtek 8168 Gigabit (Realtek Semiconductor)
0x1AF4 0x1000 0x02 0x00 Virtio Network (Red Hat, Inc)
0x8086 0x244E 0x06 0x04 Intel PCI-to-PCI Bridge (Intel Corporation)

```

=====

PCI DATABASE STATISTICS

=====

Total device entries: 9

Total vendor entries: 5

Devices by Class:

```

-----
Class 0x01 (Mass Storage): 2 devices
Class 0x02 (Network): 3 devices
Class 0x03 (Display): 2 devices
Class 0x06 (Bridge): 2 devices

```

=====

DEVICE LOOKUP DEMONSTRATION

=====

Found: Intel PRO/1000 (Vendor: Intel Corporation)

Network Controllers (Class 0x02):

- Intel PRO/1000
- Realtek 8168 Gigabit
- Virtio Network

Intel Devices:

- Intel PRO/1000
- Intel SATA Controller
- Intel Host Bridge
- Intel PCI-to-PCI Bridge

This concludes **Chapter 13: Data Sections**. The chapter covered the complete data section feature including inline and external file formats, multiple key declarations for O(1) lookup, grid traversal for reporting and debugging, and a complete PCI device database demonstration. Data sections provide a bridge between declarative data definition and efficient runtime access, making them ideal for configuration tables, hardware databases, and any scenario where data must be both human-readable and machine-optimized.

Chapter 14: Pattern Matching Switch

14.1 Introduction to Pattern Matching

Pattern matching in lowl's `switch` statement extends far beyond traditional C-style switch statements. Traditional switch statements can only match against constant integral values, leading to nested if-else chains for complex conditions. lowl's pattern matching switch supports literal values, guard expressions with arbitrary conditions, range patterns, destructuring of tuples and structures, enumeration variants, and priority ordering of cases. This makes complex branching logic more readable and less error-prone. The compiler analyzes the patterns to generate optimal decision trees—jump tables for dense integer ranges, binary search for sparse values, and short-circuit evaluation for guard conditions. Pattern matching is exhaustive by design: the compiler warns if not all possible values are covered, and explicit `default` cases can handle remaining values.

14.2 Basic Literal Patterns

The simplest form of pattern matching matches against literal values. Unlike C's switch, lowl's literal patterns can be integers, characters, strings, or enumeration constants. The compiler generates a jump table when the literal values form a dense range, and a decision tree for sparse values. Literal patterns are evaluated in order unless overridden by `priority` attributes.

lowl

```
// Basic literal pattern matching
fn http_status_message(code: u32) -> string:
    switch (code):
        case 200:
            return "OK"
        case 201:
            return "Created"
        case 400:
            return "Bad Request"
        case 401:
            return "Unauthorized"
        case 403:
            return "Forbidden"
        case 404:
            return "Not Found"
        case 500:
            return "Internal Server Error"
        case 502:
            return "Bad Gateway"
        case 503:
            return "Service Unavailable"
        default:
            return "Unknown Status"

// Character literal matching
fn is_vowel(c: char) -> bool:
    switch (c):
```

```

    case 'a': case 'e': case 'i': case 'o': case 'u':
    case 'A': case 'E': case 'I': case 'O': case 'U':
        return true
    default:
        return false

// String literal matching
fn parse_command(cmd: string) -> CommandType:
    switch (cmd):
        case "help":
            return CommandType.HELP
        case "quit":
        case "exit":
            return CommandType.QUIT
        case "status":
            return CommandType.STATUS
        default:
            return CommandType.UNKNOWN

// Multiple patterns for same case (fallthrough without break)
fn weekday_name(day: u8) -> string:
    switch (day):
        case 1: case 2: case 3: case 4: case 5:
            return "Weekday"
        case 6: case 7:
            return "Weekend"
        default:
            return "Invalid"

```

14.3 Guard Patterns (When Clauses)

Guard patterns use the `when` keyword followed by a boolean expression. Guard patterns are evaluated in order, and the first guard that evaluates to true executes its block. Guard patterns are especially useful for range checks and complex conditions that cannot be expressed as simple literals. The guard expression can reference the switched value as well as any variables that are in scope. Guard patterns can be combined with destructuring to capture sub-values.

lowl

```

// Guard patterns for range checking
fn classify_temperature(temp: i64) -> string:
    switch (temp):
        case when (temp < -20):
            return "Extreme Cold"
        case when (temp < 0):
            return "Freezing"
        case when (temp >= 0 and temp < 10):
            return "Cold"
        case when (temp >= 10 and temp < 20):

```

```

        return "Cool"
    case when (temp >= 20 and temp < 30):
        return "Warm"
    case when (temp >= 30):
        priority = 1
        return "Hot"
    default:
        return "Unknown"

// Multiple conditions in guard
fn check_access(user: &User, resource: &Resource) -> AccessResult:
    switch (user.role, resource.type):
        case when (user.is_admin()):
            return AccessResult.GRANTED
        case when (user.role == Role.OWNER and resource.owner_id ==
user.id):
            return AccessResult.GRANTED
        case when (user.role == Role.EDITOR and resource.type !=
ResourceType.READONLY):
            return AccessResult.GRANTED
        case when (resource.type == ResourceType.PUBLIC):
            return AccessResult.GRANTED_READONLY
        default:
            return AccessResult.DENIED

// Guard with captured variables
fn parse_number(s: string) -> Option<u64>:
    switch (s.len()):
        case 0:
            return Option.none()
        case when (s[0] == '0' and s[1] == 'x'):
            return parse_hex(s)
        case when (s[0] >= '0' and s[0] <= '9'):
            return parse_decimal(s)
        default:
            return Option.none()

```

14.4 Range Patterns

Range patterns allow matching against inclusive or exclusive ranges of values. The syntax `start..end` represents an inclusive-exclusive range (start is included, end is excluded). The syntax `start..=end` represents an inclusive-inclusive range (both endpoints included). Range patterns can be combined with guard conditions and can be mixed with literal patterns. The compiler optimizes range patterns by generating a single comparison for each range boundary.

lowl

```

// Score grading with range patterns
fn letter_grade(score: u64) -> char:

```

```
switch (score):
  case 90..=100:
    return 'A'
  case 80..89:
    return 'B'
  case 70..79:
    return 'C'
  case 60..69:
    return 'D'
  case 0..59:
    return 'F'
  default:
    return '?'

// BMI classification with ranges
fn bmi_category(bmi: f64) -> string:
  switch (bmi):
    case when (bmi < 18.5):
      return "Underweight"
    case 18.5..25.0:
      return "Normal"
    case 25.0..30.0:
      return "Overweight"
    case 30.0..35.0:
      return "Obese Class I"
    case 35.0..40.0:
      return "Obese Class II"
    case when (bmi >= 40.0):
      priority = 1
      return "Obese Class III"
    default:
      return "Unknown"

// Age group classification
fn age_group(years: u64) -> string:
  switch (years):
    case 0..2:
      return "Infant"
    case 2..6:
      return "Toddler"
    case 6..12:
      return "Child"
    case 12..18:
      return "Teenager"
    case 18..30:
      return "Young Adult"
    case 30..50:
      return "Adult"
    case 50..=120:
```

```

    return "Senior"
default:
    return "Invalid age"

```

14.5 Destructuring Patterns

Destructuring patterns extract values from composite data structures such as tuples, structures, and enumerations. Destructuring makes it easy to access nested fields without writing verbose extraction code. Variables introduced in destructuring patterns are bound in the case block's scope. The wildcard pattern `_` matches any value and discards it, which is useful when only some fields of a structure are needed.

lowI

```

// Tuple destructuring
fn classify_point(x: i64, y: i64) -> string:
    switch (x, y):
        case (0, 0):
            return "Origin"
        case (_x, 0) when (_x > 0):
            return "Positive X-axis"
        case (_x, 0) when (_x < 0):
            return "Negative X-axis"
        case (0, _y) when (_y > 0):
            return "Positive Y-axis"
        case (0, _y) when (_y < 0):
            return "Negative Y-axis"
        case (_x, _y) when (_x == _y):
            return "Diagonal"
        case (_x, _y) when (_x > 0 and _y > 0):
            return "First Quadrant"
        case (_x, _y) when (_x < 0 and _y > 0):
            return "Second Quadrant"
        case (_x, _y) when (_x < 0 and _y < 0):
            return "Third Quadrant"
        case (_x, _y) when (_x > 0 and _y < 0):
            return "Fourth Quadrant"
        default:
            return "Axis"

// Structure destructuring
struct Rectangle:
    x: i32
    y: i32
    width: u32
    height: u32

fn describe_rectangle(rect: &Rectangle) -> string:
    switch (rect):

```

```

    case Rectangle{x: 0, y: 0, width: w, height: h} when (w == h):
        return "Square at origin"
    case Rectangle{x: 0, y: 0}:
        return "Rectangle at origin"
    case Rectangle{width: w, height: h} when (w == h):
        return "Square"
    case Rectangle{width: w, height: h}:
        return "Rectangle"
    default:
        return "Unknown shape"

// Nested destructuring
struct Result<T, E>:
    Ok(T)
    Err(E)

fn handle_result<T, E>(r: &Result<T, E>) -> string:
    switch (r):
        case Result.Ok(value) when (value is string):
            return "String result: " + value
        case Result.Ok(value) when (value is u64):
            return "Numeric result: " + value.to_string()
        case Result.Err(code) when (code == 404):
            return "Not Found error"
        case Result.Err(code) when (code >= 500):
            return "Server error"
        case Result.Err(code):
            return "Error code: " + code.to_string()
        default:
            return "Unknown result"

```

14.6 Enum Pattern Matching

Enumerations in lowl are algebraic data types that can have variants with associated data. Pattern matching on enums is exhaustive: the compiler checks that all variants are covered. Variants without associated data match on the variant name alone. Variants with associated data destructure the variant's payload. This makes enums and pattern matching the foundation for error handling, state machines, and abstract syntax trees.

lowl

```

// Simple enum without data
enum Color:
    Red
    Green
    Blue
    Yellow
    Cyan
    Magenta

```

```

fn color_name(c: Color) -> string:
  switch (c):
    case Color.Red:
      return "Red"
    case Color.Green:
      return "Green"
    case Color.Blue:
      return "Blue"
    case Color.Yellow:
      return "Yellow"
    case Color.Cyan:
      return "Cyan"
    case Color.Magenta:
      return "Magenta"
    default:
      return "Unknown"

// Enum with associated data
enum Expression:
  Const(value: i64)
  Add(left: ptr<Expression>, right: ptr<Expression>)
  Sub(left: ptr<Expression>, right: ptr<Expression>)
  Mul(left: ptr<Expression>, right: ptr<Expression>)
  Div(left: ptr<Expression>, right: ptr<Expression>)
  Var(name: string)

// Recursive evaluation using pattern matching
fn evaluate(expr: ptr<Expression>, vars: &rb_map<string, i64>) ->
Option<i64>:
  switch (expr):
    case Expression.Const(value):
      return Option.some(value)

    case Expression.Add(left, right):
      let l = evaluate(left, vars)
      let r = evaluate(right, vars)
      if l.is_some() and r.is_some():
        return Option.some(l.unwrap() + r.unwrap())
      return Option.none()

    case Expression.Sub(left, right):
      let l = evaluate(left, vars)
      let r = evaluate(right, vars)
      if l.is_some() and r.is_some():
        return Option.some(l.unwrap() - r.unwrap())
      return Option.none()

    case Expression.Mul(left, right):

```

```

        let l = evaluate(left, vars)
        let r = evaluate(right, vars)
        if l.is_some() and r.is_some():
            return Option.some(l.unwrap() * r.unwrap())
        return Option.none()

    case Expression.Div(left, right):
        let l = evaluate(left, vars)
        let r = evaluate(right, vars)
        if l.is_some() and r.is_some() and r.unwrap() != 0:
            return Option.some(l.unwrap() / r.unwrap())
        return Option.none()

    case Expression.Var(name):
        return vars.find(name)

    default:
        return Option.none()

// State machine using enum pattern matching
enum ConnectionState:
    Disconnected
    Connecting(addr: string, port: u16)
    Connected(socket_fd: i32)
    Closing(reason: string)
    Error(code: i32, message: string)

fn handle_state_change(old_state: ConnectionState, event: ConnectionEvent) -
> ConnectionState:
    switch (old_state, event):
        case (ConnectionState.Disconnected, ConnectionEvent.Connect(addr,
port)):
            return ConnectionState.Connecting(addr, port)

        case (ConnectionState.Connecting(addr, port),
ConnectionEvent.Connected(fd)):
            return ConnectionState.Connected(fd)

        case (ConnectionState.Connecting(addr, port),
ConnectionEvent.Timeout):
            return ConnectionState.Error(-1, "Connection timeout")

        case (ConnectionState.Connected(fd),
ConnectionEvent.DataReceived(data)):
            process_data(fd, data)
            return old_state // Stay in Connected state

        case (ConnectionState.Connected(fd), ConnectionEvent.Close):
            close_socket(fd)

```

```

        return ConnectionState.Closing("Normal close")

    case (ConnectionState.Connected(fd), ConnectionEvent.Error(err)):
        return ConnectionState.Error(err.code, err.message)

    case (ConnectionState.Closing(reason), ConnectionEvent.Closed):
        return ConnectionState.Disconnected

    case (_, ConnectionEvent.Shutdown):
        return ConnectionState.Disconnected

    default:
        return old_state

```

14.7 Priority in Pattern Matching

Pattern cases are evaluated in order by default, but the `priority` attribute can override this. Higher priority numbers are evaluated first. This is useful when overlapping patterns exist and you want a specific pattern to match before a more general pattern. Without priorities, the first matching pattern in source order is used. Priorities allow you to organize patterns logically while still controlling which pattern takes precedence.

lowI

```

// Priority for exception handling
fn handle_packet(protocol: u8, data: ptr<u8>, len: u64) -> bool:
    switch (protocol, data[0]):
        case when (protocol == 0x06):
            priority = 10
            return handle_tcp(data, len)

        case when (protocol == 0x11):
            priority = 10
            return handle_udp(data, len)

        case when (protocol == 0x01):
            priority = 5
            return handle_icmp(data, len)

        case when (protocol == 0x00):
            priority = 1
            return handle_ip(data, len)

    default:
        return false

// Priority for overlapping numeric ranges
fn categorize_number(n: i64) -> string:
    switch (n):

```

```
case when (n % 2 == 0):
    priority = 10
    return "Even number"

case when (n % 3 == 0):
    priority = 5
    return "Multiple of 3"

case when (n % 5 == 0):
    priority = 5
    return "Multiple of 5"

case when (n > 0):
    priority = 1
    return "Positive number"

case when (n < 0):
    priority = 1
    return "Negative number"

default:
    return "Zero"

// Priority in parser combinators
fn parse_token(input: string, pos: u64) -> Option<(Token, u64)>:
    let ch = input[pos]
    switch (ch):
        case when (ch >= '0' and ch <= '9'):
            priority = 10
            return parse_number(input, pos)

        case when ((ch >= 'a' and ch <= 'z') or (ch >= 'A' and ch <= 'Z')):
            priority = 10
            return parse_identifier(input, pos)

        case '+':
            priority = 5
            return Option.some((Token.Plus, pos + 1))

        case '-':
            priority = 5
            return Option.some((Token.Minus, pos + 1))

        case '*':
            priority = 5
            return Option.some((Token.Star, pos + 1))

        case '/':
            priority = 5
```

```

        return Option.some((Token.Slash, pos + 1))

    case '(':
        priority = 5
        return Option.some((Token.LParen, pos + 1))

    case ')':
        priority = 5
        return Option.some((Token.RParen, pos + 1))

    case ' ', '\t', '\n':
        priority = 1
        return parse_token(input, pos + 1)

    default:
        return Option.none()

```

14.8 Complete Chapter Example: Expression Evaluator

This example demonstrates a complete expression evaluator using pattern matching for lexical analysis, parsing, and evaluation. The example includes tokenization, abstract syntax tree construction, and recursive evaluation with variables.

lowl

```

// evaluator.lowl - Expression Evaluator with Pattern Matching
// Compile: lowlc evaluator.lowl -o evaluator.asm -O2

//
=====
// TOKENIZATION
//
=====

enum TokenType:
    Number(value: i64)
    Identifier(name: string)
    Plus
    Minus
    Star
    Slash
    LParen
    RParen
    Equal
    Semicolon
    EOF

struct Token:
    type: TokenType

```

```
line: u64
col: u64

// Tokenizer using pattern matching
fn tokenize(source: string) -> BlockArray<Token>:
  let tokens = BlockArray<Token>.new()
  let mut pos: u64 = 0
  let mut line: u64 = 1
  let mut col: u64 = 1

  while pos < source.len():
    let ch = source[pos]

    switch (ch):
      // Whitespace
      case ' ', '\r':
        pos = pos + 1
        col = col + 1

      case '\n':
        pos = pos + 1
        line = line + 1
        col = 1

      // Single-character tokens
      case '+':
        tokens.push(Token{TokenType.Plus, line, col})
        pos = pos + 1
        col = col + 1

      case '-':
        tokens.push(Token{TokenType.Minus, line, col})
        pos = pos + 1
        col = col + 1

      case '*':
        tokens.push(Token{TokenType.Star, line, col})
        pos = pos + 1
        col = col + 1

      case '/':
        tokens.push(Token{TokenType.Slash, line, col})
        pos = pos + 1
        col = col + 1

      case '(':
        tokens.push(Token{TokenType.LParen, line, col})
        pos = pos + 1
        col = col + 1
```

```

    case ')':
        tokens.push(Token{TokenType.RParen, line, col})
        pos = pos + 1
        col = col + 1

    case '=':
        tokens.push(Token{TokenType.Equal, line, col})
        pos = pos + 1
        col = col + 1

    case ';':
        tokens.push(Token{TokenType.Semicolon, line, col})
        pos = pos + 1
        col = col + 1

    // Numbers
    case when (ch >= '0' and ch <= '9'):
        let start = pos
        while pos < source.len() and source[pos] >= '0' and
source[pos] <= '9':
            pos = pos + 1
        let num_str = source[start..pos]
        let value = parse_int(num_str)
        tokens.push(Token{TokenType.Number(value), line, col})
        col = col + (pos - start)

    // Identifiers
    case when ((ch >= 'a' and ch <= 'z') or (ch >= 'A' and ch <=
'Z')):
        let start = pos
        while pos < source.len() and
            ((source[pos] >= 'a' and source[pos] <= 'z') or
             (source[pos] >= 'A' and source[pos] <= 'Z') or
             (source[pos] >= '0' and source[pos] <= '9')):
            pos = pos + 1
        let name = source[start..pos]
        tokens.push(Token{TokenType.Identifier(name), line, col})
        col = col + (pos - start)

    default:
        pos = pos + 1
        col = col + 1

    tokens.push(Token{TokenType.EOF, line, col})
    return tokens

//
=====

```

```
// ABSTRACT SYNTAX TREE
//
=====

enum Expr:
  Const(value: i64)
  Var(name: string)
  Add(left: ptr<Expr>, right: ptr<Expr>)
  Sub(left: ptr<Expr>, right: ptr<Expr>)
  Mul(left: ptr<Expr>, right: ptr<Expr>)
  Div(left: ptr<Expr>, right: ptr<Expr>)
  Neg(child: ptr<Expr>)
  Assign(name: string, value: ptr<Expr>)

enum Statement:
  ExprStmt(expr: ptr<Expr>)
  Print(expr: ptr<Expr>)
  Block(stmts: BlockArray<Statement>)

//
=====
// PARSER USING PATTERN MATCHING
//
=====

class Parser:
  private:
    tokens: BlockArray<Token>
    pos: u64 = 0

  public:
    fn new(tokens: BlockArray<Token>) -> Parser:
      this.tokens = tokens
      return this

    fn current() -> Token:
      let opt = this.tokens.get(this.pos)
      if opt.is_some():
        return opt.unwrap()
      return Token{TokenType.EOF, 0, 0}

    fn advance():
      this.pos = this.pos + 1

    fn parse_program() -> BlockArray<Statement>:
      let stmts = BlockArray<Statement>.new()
      while this.current().type != TokenType.EOF:
        let stmt = this.parse_statement()
        stmts.push(stmt)
```

```

    return stmts

fn parse_statement() -> Statement:
    let tok = this.current()

    switch (tok.type):
        case TokenType.Identifier(_):
            // Check if it's a print statement
            let name = tok.type as TokenType.Identifier
            if name == "print":
                this.advance()
                let expr = this.parse_expression(0)
                this.expect_semicolon()
                return Statement.Print(expr)
            else:
                let expr = this.parse_expression(0)
                this.expect_semicolon()
                return Statement.ExprStmt(expr)

        default:
            let expr = this.parse_expression(0)
            this.expect_semicolon()
            return Statement.ExprStmt(expr)

fn parse_expression(min_precedence: u64) -> ptr<Expr>:
    let mut left = this.parse_primary()

    while true:
        let tok = this.current()
        let precedence = this.get_precedence(tok.type)

        if precedence < min_precedence:
            break

        this.advance()
        let right = this.parse_expression(precedence + 1)

        switch (tok.type):
            case TokenType.Plus:
                left = Expr.Add(left, right)
            case TokenType.Minus:
                left = Expr.Sub(left, right)
            case TokenType.Star:
                left = Expr.Mul(left, right)
            case TokenType.Slash:
                left = Expr.Div(left, right)
            case TokenType.Equal:
                match left:
                    case Expr.Var(name):

```

```
        left = Expr.Assign(name, right)
    default:
        panic("Invalid assignment target")
    default:
        pass

    return left

fn parse_primary() -> ptr<Expr>:
    let tok = this.current()
    this.advance()

    switch (tok.type):
        case TokenType.Number(value):
            return Expr.Const(value)

        case TokenType.Identifier(name):
            return Expr.Var(name)

        case TokenType.Minus:
            let child = this.parse_primary()
            return Expr.Neg(child)

        case TokenType.LParen:
            let expr = this.parse_expression(0)
            this.expect(TokenType.RParen)
            return expr

        default:
            panic("Unexpected token")

fn get_precedence(tt: TokenType) -> u64:
    switch (tt):
        case TokenType.Equal:
            return 1
        case TokenType.Plus, TokenType.Minus:
            return 2
        case TokenType.Star, TokenType.Slash:
            return 3
        default:
            return 0

fn expect(tok_type: TokenType):
    if this.current().type != tok_type:
        panic("Expected token")
    this.advance()

fn expect_semicolon():
    this.expect(TokenType.Semicolon)
```

```
//
=====
// INTERPRETER USING PATTERN MATCHING
//
=====

class Interpreter:
    private:
        variables: rb_map<string, i64>

    public:
        fn new() -> Interpreter:
            this.variables = rb_map<string, i64>.new(compare_string)
            return this

        fn evaluate(expr: ptr<Expr>) -> Option<i64>:
            switch (expr):
                case Expr.Const(value):
                    return Option.some(value)

                case Expr.Var(name):
                    return this.variables.find(name)

                case Expr.Assign(name, value_expr):
                    let val_opt = this.evaluate(value_expr)
                    if val_opt.is_some():
                        let val = val_opt.unwrap()
                        this.variables.insert(name, val)
                        return Option.some(val)
                    return Option.none()

                case Expr.Add(left, right):
                    let l = this.evaluate(left)
                    let r = this.evaluate(right)
                    if l.is_some() and r.is_some():
                        return Option.some(l.unwrap() + r.unwrap())
                    return Option.none()

                case Expr.Sub(left, right):
                    let l = this.evaluate(left)
                    let r = this.evaluate(right)
                    if l.is_some() and r.is_some():
                        return Option.some(l.unwrap() - r.unwrap())
                    return Option.none()

                case Expr.Mul(left, right):
                    let l = this.evaluate(left)
                    let r = this.evaluate(right)
```

```

        if l.is_some() and r.is_some():
            return Option.some(l.unwrap() * r.unwrap())
        return Option.none()

    case Expr.Div(left, right):
        let l = this.evaluate(left)
        let r = this.evaluate(right)
        if l.is_some() and r.is_some() and r.unwrap() != 0:
            return Option.some(l.unwrap() / r.unwrap())
        return Option.none()

    case Expr.Neg(child):
        let val_opt = this.evaluate(child)
        if val_opt.is_some():
            return Option.some(-val_opt.unwrap())
        return Option.none()

    default:
        return Option.none()

fn execute(stmt: Statement) -> bool:
    switch (stmt):
        case Statement.ExprStmt(expr):
            let opt = this.evaluate(expr)
            return opt.is_some()

        case Statement.Print(expr):
            let opt = this.evaluate(expr)
            if opt.is_some():
                let val = opt.unwrap()
                print_dec(val)
                print_string("\n")
                return true
            print_string("Error evaluating expression\n")
            return false

        case Statement.Block(stmts):
            for i in 0..stmts.len():
                let opt = stmts.get(i)
                if opt.is_some():
                    if not this.execute(opt.unwrap()):
                        return false
            return true

    default:
        return false

fn run(program: BlockArray<Statement>) -> bool:
    for i in 0..program.len():

```

```

        let opt = program.get(i)
        if opt.is_some():
            if not this.execute(opt.unwrap()):
                return false
        return true

//
=====
// DEMONSTRATION
//
=====

fn main() -> u32:
    print_string("=== Expression Evaluator with Pattern Matching ===\n\n")

    // Test program
    let source = "
        a = 10;
        b = 20;
        c = a + b;
        print c;
        d = (a + b) * 2;
        print d;
        e = (d - a) / b;
        print e;
        print 100 - 50 * 2;
    "

    print_string("Source code:\n")
    print_string(source)
    print_string("\n")

    print_string("Tokenizing...\n")
    let tokens = tokenize(source)
    print_string(" ")
    print_dec(tokens.len())
    print_string(" tokens\n\n")

    print_string("Parsing...\n")
    let parser = Parser.new(tokens)
    let program = parser.parse_program()
    print_string(" ")
    print_dec(program.len())
    print_string(" statements\n\n")

    print_string("Executing...\n")
    let interpreter = Interpreter.new()

    if interpreter.run(program):

```

```

        print_string("\nExecution completed successfully!\n")
    else:
        print_string("\nExecution failed!\n")

    return 0

//
=====
// HELPER FUNCTIONS
//
=====

static vga_ptr: mmio_ptr<u16> = 0xB8000 as mmio_ptr<u16>
static cursor: u64 = 0

fn print_char(ch: u8):
    let color = (0x0F << 8) as u16
    if ch == '\n' as u8:
        cursor = cursor + (80 - (cursor % 80))
    else:
        vga_ptr[cursor] = color | (ch as u16)
        cursor = cursor + 1
    if cursor >= 80 * 25:
        cursor = 0

fn print_string(s: string):
    for ch in s:
        print_char(ch as u8)

fn print_dec(value: i64):
    if value == 0:
        print_char('0')
        return
    let mut temp = value
    if temp < 0:
        print_char('-')
        temp = -temp
    let mut digits: array<u8, 20>
    let mut i: u64 = 0
    while temp > 0:
        digits[i] = (temp % 10) as u8
        temp = temp / 10
        i = i + 1
    for j in i..0 step -1:
        print_char('0' + digits[j - 1])

fn parse_int(s: string) -> i64:
    let mut result: i64 = 0
    for ch in s:

```

```

        if ch >= '0' and ch <= '9':
            result = result * 10 + (ch - '0')
    return result

fn panic(msg: string):
    print_string("PANIC: ")
    print_string(msg)
    print_string("\n")
    while true:
        halt()

fn compare_string(a: string, b: string) -> i8:
    let min_len = min(a.len(), b.len())
    for i in 0..min_len:
        if a[i] < b[i]: return -1
        elif a[i] > b[i]: return 1
    if a.len() < b.len(): return -1
    elif a.len() > b.len(): return 1
    else: return 0

```

Expected Output:

text

=== Expression Evaluator with Pattern Matching ===

Source code:

```

a = 10;
b = 20;
c = a + b;
print c;
d = (a + b) * 2;
print d;
e = (d - a) / b;
print e;
print 100 - 50 * 2;

```

Tokenizing...

33 tokens

Parsing...

9 statements

Executing...

30

60

2

0

